Benchmarking Agentic Code Generation Tools

Wenrui Chen (1594695)
wenruichen@student.unimelb.edu.au



Department of Computing and Information Systems, The University of Melbourne

Supervised by Prof. Richard Sinnott

I certify that this thesis is my original work, all sources have been duly acknowledged.

Signature: ***Wenrui Chen*** **Date:** October 26, 2026

Abstract

Evaluating agentic coding systems requires evidence that they can turn a debugging request into an acceptable application state through their available tools. A completion score alone can obscure failures to apply edits, unmet requirements and differences in assessment conditions. This thesis develops the Tiered System Benchmark (TSB), a controlled evaluation of complete coding-system configurations across frontend, backend, database and container dependencies.

TSB comprises 12 deterministic debugging tasks across four architectural tiers, with resettable starting states, public task contracts and evaluator-only functional and architectural checks. Cursor, Devin Desktop, GPT-5.4 and Qwen2.5-Coder-7B-Instruct are evaluated through three fresh runs per task, yielding 144 selected valid observations, including unsuccessful runs. Success requires normal submission and passing every applicable check under the declared scoring version. Timing, clarification behaviour, final diffs and available interaction evidence help explain these outcomes. Original and reviewed scoring versions are retained to examine evaluator sensitivity.

Under reviewed scoring, Cursor, GPT-5.4 and Devin Desktop succeed in 35, 34 and 33 of 36 runs respectively, while the configured Qwen system succeeds in none. Exploratory task-paired comparisons distinguish each of the first three systems from Qwen, but do not distinguish them from one another after multiplicity adjustment. Their descriptive ordering changes under original scoring. Success does not decline monotonically across the selected tiers. Qwen's final diffs were empty. A separate 72-run interface control found no successful repair with either the frozen fallback or a version allowing multiline JSON; some newly accepted requests still failed on nonexistent paths. On the single deliberately incomplete task, successful completion and use of the authorised clarification channel are distinct outcomes.

The findings show why the configured interface, task contract and scoring conditions matter when interpreting performance. The contribution is a traceable evaluation framework and bounded empirical comparison; twelve constructed tasks and incomplete native interaction traces limit generalisation and causal attribution.

Contents

1 Introduction **4**
- 1.1 Background and Motivation . . . 4
- 1.2 Problem Statement . . . 4
- 1.3 Research Objectives and Scope . . . 6
- 1.4 Contributions and Thesis Organisation . . . 7

2 Related Work **7**
- 2.1 The Evolution of AI-Assisted Software Engineering . . . 7
- 2.2 LLM-based Agents in Software Engineering . . . 8
- 2.3 Classification of Existing Code Generation Benchmarks . . . 11
- 2.4 Evaluation Methods and the Meaning of Correctness . . . 15
- 2.5 Cross-Service Debugging and Integration Faults . . . 19
- 2.6 Clarification Behaviour and Human–AI Interaction . . . 20
- 2.7 The Research Gap . . . 22

3 Technical Background **23**
- 3.1 Architecture of Agentic IDEs . . . 24
- 3.2 Evaluation Baselines: API-Based Agents . . . 24
- 3.3 Tool Actions, Execution Context and Observable Progress . . . 25
- 3.4 Application Layers and Cross-Service Contracts . . . 27
- 3.5 Requirements, Test Oracles and Architectural Checks . . . 28
- 3.6 Clarification and the Boundary of Evaluator Assistance . . . 29

4 Methodology **30**
- 4.1 Overview of the Multi-tier Sandbox Architecture . . . 30
- 4.2 System-level Execution Environment and Task Progression . . . 32
- 4.3 Human-Agent Interaction Protocol and State Machine . . . 37
- 4.4 Pilot Study and Main-Experiment Configuration Freeze . . . 41
- 4.5 Supplementary Qwen Interface Control . . . 45
- 4.6 Quantitative Evaluation Metrics . . . 45

5 Results **50**
- 5.1 End-to-End Success . . . 50
- 5.2 Paired System Comparisons . . . 53
- 5.3 Elapsed Time and Diagnostic Correctness . . . 55
- 5.4 Clarification and Success Without Evaluator Responses . . . 58
- 5.5 Observable Failure Trajectories . . . 59
- 5.6 Supplementary Qwen Interface Control . . . 63
- 5.7 Sensitivity to Scoring Review . . . 63

6 Discussion **65**
- 6.1 Interpretation of the Main Findings . . . 65
- 6.2 End-to-End Performance and the Role of the Interface . . . 66

6.3 Task Contracts and Architectural Boundaries . . . 67
6.4 Clarification as a Distinct Interaction Outcome . . . 68
6.5 Construct Validity . . . 69
6.6 Internal Validity . . . 70
6.7 External Validity . . . 72
6.8 Practical Implications and Priorities for Further Evaluation . . . 73

7 Conclusion **75**

References **76**

A Detailed Task Case Studies in the Tiered System Benchmark **87**
A.1 Level 3 Task: Cross-Origin Resource Sharing (CORS) Configuration . . . 87
A.2 Level 4 Task: Database Schema Cascading Failure . . . 87

B FSM Implementation and Logic Gating Parameters **88**
B.1 Run Limits and Inference Configuration . . . 88
B.2 Invalid Runs, Replacement Attempts, and Budget Interruptions . . . 89
B.3 Clarification Rules and Technical Fault Categories . . . 90
B.4 Abstract State Transition Logic . . . 91

C API-Agent Prompts and Action-Space Specifications **93**
C.1 Standardized System Prompt for API-Based Agents . . . 93
C.2 Common Task Brief . . . 94
C.3 Constrained Action Space for API-Based Agents . . . 95
C.4 Native IDE Protocol for Cursor and Devin Desktop . . . 95
C.5 Pilot and Main-Experiment Completion Status . . . 96

D Evaluation Metrics and Reporting Schema **97**
D.1 Evidence and Amendment Provenance . . . 97
D.2 Run-Level Success and Diagnostic Correctness . . . 98
D.3 Execution Performance . . . 98
D.4 Clarification-Seeking Behaviour . . . 98
D.5 Success Without Evaluator Clarification Responses . . . 99
D.6 Qualitative Failure-Trajectory Analysis . . . 99
D.7 Statistical Analysis and Reporting . . . 99
D.8 Uniform Evaluator Review and Score Amendments . . . 100
D.9 Failure Coding and Illustrative Evidence . . . 102

E Supplementary Qwen Interface Experiment **104**
E.1 Configuration and Execution Checks . . . 104
E.2 Task-Level Outcomes . . . 104
E.3 Attempt Accounting . . . 105

List of Tables

1 Task scope and assessment in selected benchmarks . . . 12
2 Six dimensions of evaluation design . . . 16
3 Task coverage and required repair evidence . . . 34
4 Experimental controls and remaining variation . . . 37
5 Clarification behaviour . . . 58
6 Supplementary Qwen interface results. Each arm contains three fresh runs on each of 12 tasks. Counts of calls and operations are totals across runs; task success uses the reviewed end-to-end criterion. . . . 63
7 Original and reviewed scoring of affected submissions . . . 102
8 Tool progress by task in the supplementary experiment. Each row aggregates three repetitions per arm. . . . 105

List of Figures

1 From local code completion to system-level agentic coding . . . 5
2 Native IDE and API-agent execution paths . . . 26
3 Controlled sandbox evaluation workflow . . . 31
4 Level 4 schema-cascade repair . . . 33
5 Run protocol and conjunctive success condition . . . 40
6 From pilot attempts to the main analysis dataset . . . 42
7 Original and reviewed scoring of the same runs . . . 44
8 Overall and task-level reviewed success . . . 51
9 Success by architectural tier . . . 52
10 Paired success-rate differences . . . 54
11 Elapsed time by outcome population . . . 56
12 Functional and architectural diagnostics . . . 57
13 L3/L4 success without clarification responses . . . 59
14 Observed stages of unsuccessful runs . . . 60
15 Evidence-supported failure categories . . . 62
16 Sensitivity to scoring review . . . 64
17 Three perspectives on benchmark outcomes . . . 66

1 Introduction

1.1 Background and Motivation

Software-engineering support has progressed from learning-based code completion [1, 2] toward LLM-based coding tools and software-engineering agents [3, 4]. Commercial examples include Cursor[1] and Devin Desktop[2]. Agentic coding systems extend beyond syntax prediction, using multi-file context, planning and environment interaction to support repository-scale tasks [4, 5].

LLM-based software engineering has attracted sustained research and practical interest [3]. Developers have reported productivity benefits from AI-assisted code completion [2], but assistance effects also depend on the task. In a controlled study comparing coding puzzles and bug repair, Wang et al. found a significant completion-time benefit for the puzzles but not for the studied repair task [6]. The present study therefore measures configured agentic systems directly in controlled multi-file debugging scenarios.

A debugging request calls for a working application whose behaviour satisfies the request. A plausible replacement for one expression may be insufficient. In a system with a browser interface, an API and stored data, that outcome may depend on several connected components. A change can be syntactically valid yet leave the visible problem unresolved, or remove the immediate error while violating another requirement. This thesis studies such bounded repair tasks, in which the relevant outcome is the state of the application after the agent has finished interacting with it.

The agent's interaction with the environment is part of that outcome. Before a repair can be evaluated, the system has to locate relevant code, issue usable operations and obtain feedback about their effects. SWE-agent's investigation of agent–computer interfaces provides a research basis for treating navigation, editing and execution as substantive parts of software-engineering automation [5]. Comparing deployed agent configurations therefore asks a different question from comparing language models given an already-localised code fragment.

There is also an information boundary between diagnosing a technical fault and determining what behaviour the user wants. Clarification research examines how questions can address underspecified code-generation requests [7]. In a repair setting, however, an error message is not necessarily a missing requirement: it can be evidence the agent is expected to investigate. Recording clarification therefore distinguishes requests for missing task intent from requests for help with a solvable technical problem. Without that distinction, a low request count can look autonomous while concealing a missed information gap, and a high count can mix useful clarification with unnecessary interaction.

1.2 Problem Statement

Evaluating agentic code-generation tools requires attention to the tasks and interaction settings in which they operate. A review of code-generation evaluation covers function-level benchmarks such as HumanEval and MBPP [8]. EvalPlus shows how stronger tests expose incorrect code missed by the original HumanEval test suite [9]. Repository-level issue resolution is represented by SWE-bench [10].

[1] https://www.cursor.com/
[2] https://devin.ai/

Function-level benchmarks provide controlled measures of isolated generation, and repository-level benchmarks evaluate realistic multi-file repair. This study combines deterministic cross-service faults, resettable multi-container execution, architectural invariants and a predefined evaluator-interaction protocol. Interactive benchmarks capture execution feedback [11]; the TSB extends this evaluation focus to end-to-end agent behaviour across frontend, backend, and database dependencies. The TSB measures correctness, end-to-end execution time, clarification-seeking behaviour, and L3/L4 success without evaluator clarification responses in this setting. Each system receives a fixed task brief and versioned starting repository, with tools for inspection, editing, execution and validation toward an end-to-end outcome. Figure 1 shows this progression from local code completion to system-level agentic coding.

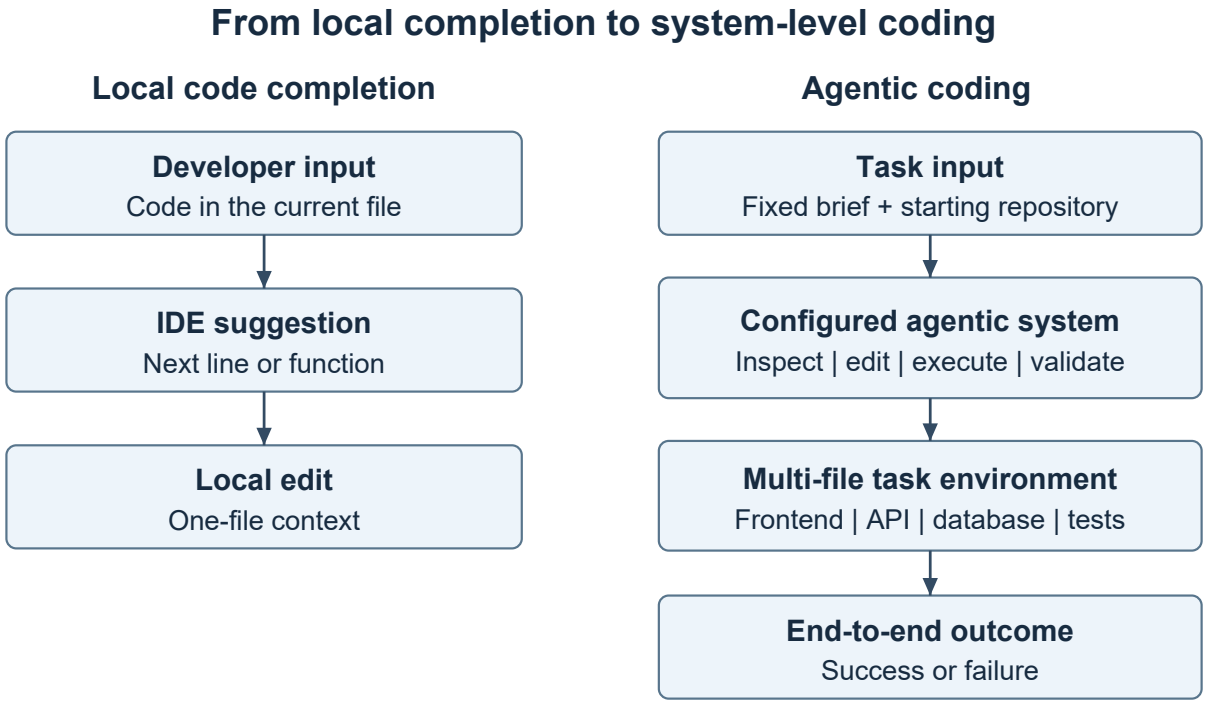


Figure 1: Shift from local code completion to agentic coding systems operating in a constrained multi-file software environment.

The unresolved question is how configured systems perform when repair completion, component dependencies and limits on evaluator assistance are assessed together. A benchmark of isolated functions cannot answer that particular question on its own, while a repository-repair score does not automatically explain performance across this thesis's architectural tiers or clarification conditions. Related Work develops this comparison of evaluation conditions.

Three measurement issues motivate reporting more than a single completion score. First, the same unsuccessful outcome can arise before any effective code change or after a substantial but incorrect submission. Second, a partially passing application can retain a fault in a requirement that matters to the complete workflow. Third, elapsed time depends on whether the run reaches a successful completion or stops unsuccessfully. Preserving these distinctions clarifies what the headline success rate measures. It can also avoid rewarding rapid failure as if it were efficient repair or reading an unchanged passing component as evidence of agent progress.

The evaluation criteria themselves are another source of uncertainty. The test-oracle literature identifies the problem of deciding whether observed behaviour corresponds to the intended correct behaviour [12]. For this thesis, that problem concerns both coverage of the task contract and acceptance of different implementations that meet it. Reproducible execution is necessary for comparing outcomes, but it does not alone show that every scoring condition is justified by the public task. The study therefore retains the evaluation version and evidence behind score amendments, allowing readers to distinguish changes in assessment from changes in the agent's execution.

Architectural tiers organise the investigation without assuming its answer. They make it possible to compare selected tasks with different component boundaries, but they do not create a pure manipulation of difficulty: requirements and fault families differ as well. The study asks whether performance changes across that progression and where task-specific failures occur.

1.3 Research Objectives and Scope

The primary objective is to develop and evaluate a thesis-scale Tiered System Benchmark (TSB) for agentic software-debugging systems. The study addresses three objectives:

- **Design a tiered evaluation sandbox:** Implement 12 deterministic debugging tasks across four architectures: (1) static websites, (2) single-service FastAPI applications, (3) Vue–FastAPI multi-service applications, and (4) Vue–FastAPI–PostgreSQL multi-container applications. The upper tiers test cross-service and cross-layer dependencies in compact synthetic systems.
- **Evaluate four complementary dimensions:** Report binary run-level success as the primary outcome, supported by functional-test and architectural-invariant pass proportions, end-to-end wall-clock time, clarification opportunity capture, request precision, unnecessary-request rate, and success without evaluator clarification responses on Level 3 and Level 4 tasks. Token usage and API cost are supplementary where provider logs expose them.
- **Compare four end-to-end systems:** Evaluate Cursor and Devin Desktop in their native IDE environments using the default agent model or mode and default settings available at the configuration-freeze date, without a manual model override. GPT-5.4 is accessed through the OpenAI Responses API using a dated GPT-5.4 snapshot, while Qwen2.5-Coder-7B-Instruct is served from a pinned official Hugging Face model revision through a managed Inference Endpoint with vLLM. The pilot established the initial configuration records before main collection. Subsequent changes are documented through amendments and replacement freezes, with each selected run linked to its applicable configuration.

The completed study comprises 144 selected valid observations across 12 tasks, four configured systems and three repetitions, with one observation per scheduled task–system–repetition slot. Valid unsuccessful runs remain in this total; invalidated attempts and pilot runs are excluded. Section 5 presents the statistical results and observable failure patterns; the discussion considers their implications and limits.

1.4 Contributions and Thesis Organisation

The thesis contributes an executable evaluation setting, a comparison of configured systems and an evidence record that connects outcomes to their interpretation. The setting provides compact tasks with explicit repair obligations and reproducible starting states. The comparison applies these tasks to native IDE systems and API-based agents under a common task protocol, while retaining the differences between their execution interfaces. The evidence record links selected runs to final-state checks, stopping reasons and available interaction records. Readers can therefore examine aggregate results at task and run levels.

These contributions address RQ1 (tiered task correctness), RQ2 (clarification-seeking behaviour) and RQ3 (comparative system performance), formally stated in Section 2.7. The tier and task decomposition addresses correctness across the chosen architectural progression. The clarification measures address behaviour under an intentionally restricted information gap. The system comparisons combine success and timing with the evidence available about unsuccessful runs. The study assesses complete configurations on these specified tasks.

The chapters have distinct roles in developing that assessment. Related Work positions the benchmark against research on task design, interaction and evaluation. Technical Background introduces the execution and application concepts needed to understand the experiment. Methodology specifies the task packages, controls, interaction rules and analysis. Results reports the observed outcomes, while Discussion interprets their implications and limits. The appendices retain detailed task examples, prompts, protocol specifications and evaluation evidence that support inspection without interrupting the main argument.

2 Related Work

This review compares six aspects of prior work that shape the interpretation of the present benchmark: task provenance, interaction feedback, environment reset, success criteria, architectural constraints, and clarification conditions. The comparison examines how these design choices determine what each benchmark measures.

2.1 The Evolution of AI-Assisted Software Engineering

Early learning-based completion work used examples from existing code to improve Eclipse recommendations [1]. White et al. investigated recurrent neural models of source code [13], Gu et al. learned mappings from natural-language queries to API sequences [14], and Iyer et al. used an attention-based neural model for code summarisation [15]. Raychev et al. studied API completion with statistical language models [16]. These systems addressed distinct assistance tasks; Hellendoorn and Devanbu's comparison of language models further highlighted the importance of vocabulary, locality, and adaptation to changing code [17].

The Transformer introduced an attention-based architecture [18], and GPT-3 demonstrated broad few-shot task performance from text prompts [19]. Studies of GitHub Copilot examined generated solution quality and suitability as a programming assistant [20], solution correctness [21], and perceived productivity [2]. They also documented gaps between user expectations and the usability and reliability of generated code [22].

The distinction between a code suggestion and a completed repair is important for interpreting this research trajectory. A suggestion is an intermediate artefact that a developer may accept, revise, or reject. In the present study, the evaluated system must act on the repository and leave an acceptable application state. These are different evaluation targets even when both involve generated code. Consequently, evidence about suggestion acceptance, perceived productivity, or isolated solution correctness cannot by itself establish how a configured agent will perform when it must locate a fault, apply a change, and submit the resulting state.

2.2 LLM-based Agents in Software Engineering

Recent LLM-based software-engineering systems combine language generation with repository search, editing, and execution, extending the range of activities considered in software-engineering research [3]. Their design choices concern both the tools made available and the control over when those tools are used. These choices distinguish an autonomous action loop from a fixed repair pipeline, even when both ultimately produce a patch.

Tool-using agents produce observable sequences of messages, actions, and environment responses [23, 11]. Such traces support analysis of whether a system located relevant code, attempted a change, or obtained execution feedback. They do not directly reveal private reasoning. For an end-to-end comparison, the distinction between an intended action and an executed action is therefore important: an explanation of a proposed repair is not equivalent to an applied change.

Tool Selection and Repository Context: Tool-learning research separates finding an appropriate operation from executing it correctly. ToolLLM builds ToolBench from real-world APIs and generated instruction–solution pairs, combining API retrieval with a search procedure for tool-use trajectories [24]. API-Bank evaluates progressively demanding combinations of API calling, retrieval and planning through executable tools and dialogue examples [25]. Their evaluation targets include deciding which operation is relevant and supplying a valid invocation. For TSB's API-based configurations, the wrapper provides a bounded set of repository operations. An agent must use those operations effectively, but it is not being tested on discovery across ToolBench's API collection. This distinction locates a shared concern, the transition from a request to an executable operation, without equating the available tool spaces.

Gorilla examines API-call generation with retrieved documentation and adaptation to changed API descriptions [26]. This adds a documentation condition to the comparison: selecting a relevant API and producing a supported call are distinct from observing that the requested operation completed. Its APIBench setting evaluates API use rather than repository repair.

Repository retrieval poses a related information-selection problem. RepoCoder iterates retrieval and generation so that the developing completion can inform a later search for relevant repository context [27]. Repoformer learns selective retrieval, motivated by evidence that retrieving additional context can be unnecessary or unhelpful for a completion [28]. Statically Contextualizing Large Language Models with Typed Holes uses expected types and relevant names from the typing context to organise information supplied at a completion site [29]. These designs expose three decisions: how retrieval is updated, when it should occur, and which program structures constrain the selected context. The Typed Holes experiments provide the location to complete, and their TypeScript study manually simulates relevant-header retrieval; they do not establish autonomous fault localisation or full-application repair. More generally, completion evidence should be

distinguished from evidence that a system can diagnose and repair an application. For TSB, recorded file reads establish exposure to particular code, but do not show that relevant type information was presented or isolate the contribution of that context to a subsequent repair.

Moving from context retrieval to issue localisation, LocAgent represents code structure and dependencies as a graph for locating entities relevant to an issue [30]. It makes localisation an explicit target, helping distinguish access to relevant code from the correctness of a later edit.

From Patch Generation to a Repair Process: Automated program repair provides a useful foundation for analysing what an agent adds to code generation. TBar applies a collection of repair templates and studies their interaction with fault localisation and the retrieval of replacement code [31]. Its distinction between perfect localisation and a practical repair pipeline shows why supplying the faulty location changes the problem being evaluated. A patch generator given the relevant statement has already received information that a repository agent may need to discover. Comparisons should therefore record the supplied localisation evidence before attributing a difference to patch-generation quality.

Neural repair systems change how candidate edits are represented and searched. CoCoNuT separately represents buggy code and its surrounding context, then combines translation models to obtain varied candidate repairs [32]. CURE combines a pre-trained code model, subword tokenisation, and a code-aware search strategy that favours compilable candidates [33]. TFix instead frames repairs of static-analyser errors as a text-to-text task [34]. These approaches expose different sources of guidance: local context, programming-language constraints, and analyser diagnostics. Their contribution is more specific than a generic ability to generate code; each connects a representation of the defect to a constrained repair procedure.

These repair settings clarify the additional responsibilities in TSB. A configured system receives a task brief and an application state, and must turn observations into an applied change under the run protocol. It may need to discover which component is responsible before deciding what to edit. Repair models evaluated with an identified location or a particular analyser warning consequently provide evidence about a component of this process. Their reported results cannot substitute for evidence that the complete navigation, editing, execution, and submission sequence works in the selected application.

Interfaces and Repair Strategies: SWE-agent and AutoCodeRover illustrate different ways to organise repository repair. SWE-agent studies the interface through which a language model searches, views, edits, and executes code; its interface analysis shows that the presentation of actions and feedback can affect behaviour [5]. AutoCodeRover uses structure-aware repository search to obtain relevant program context before patch generation, with spectrum-based fault localisation available in its test-assisted configuration [35]. The comparison highlights two design choices: how the agent obtains relevant context and how it uses available evidence to select or revise a change. CodeAct varies the action representation itself, using executable Python to compose operations and incorporate interpreter feedback [36]. Its comparison with structured action formats makes action representation another experimental variable, alongside repository search and control policy.

Feedback can guide subsequent repairs rather than only accept or reject a completed candidate. ChatRepair carries test-failure information and earlier patch attempts into a conversational repair

loop, using unsuccessful and plausible attempts to inform later generation [37]. RepairAgent exposes repair tools through a dynamically updated prompt and uses a finite state machine to guide tool invocation [38]. Reflexion adds a different retention mechanism: task feedback is converted into verbal reflections stored in episodic memory for later trials, without updating model weights [39]. The comparison separates the source of feedback, the duration for which it is retained, and the rules governing available actions. A within-run conversational history and memory carried into another trial expose different information to the next attempt. Neither determines the action space by itself, and a constrained action space can still allow adaptive decisions.

This distinction matters for comparing validation behaviour. An agent can run a test and then ignore its output, while a repair loop can explicitly incorporate the resulting failure into the next attempt. The available trace can establish whether execution occurred and what feedback was returned; attributing productive use of that feedback requires examining the subsequent action. TSB's trace evidence can support such bounded descriptions where the sequence is recorded.

Fixed Pipelines and Autonomous Control: Agentless provides a useful counterpoint to systems that delegate action selection to the language model. Xia et al. organise issue resolution into localisation, repair, and patch validation, with a predefined workflow rather than open-ended model control over the next tool action [4]. Its competitive results on SWE-bench Lite show that pipeline structure is a substantive comparison variable. The relevant contrast is where control resides: in a specified sequence of stages or in the agent's decisions during execution.

This comparison separates three questions that can otherwise become conflated. The model supplies generation and interpretation capabilities; the surrounding software exposes actions and observations; the control policy determines the order in which they are used. A system can improve at repository repair through changes to any of these components. TSB observes their combined outcome in each configured system. Its results can identify failures at observable interaction boundaries, but attribution to a particular component would require a matched intervention that holds the others fixed.

These mechanisms should be distinguished from the shared task on which systems are evaluated. A benchmark specifies the initial problem and assessment conditions, whereas a repair system chooses how to search and modify the code within those conditions.

For TSB, this motivates retaining both the final outcome and the observable interaction evidence. A failed run may leave no code change, an applied but incorrect change, or a partial repair that violates another requirement. These states require different descriptions even though they share the same binary outcome. The thesis uses recorded actions, final diffs, and evaluator outputs to distinguish them.

Generalisability also depends on the relationship between training projects and evaluation projects. Zirak and Hemmati study domain shift in neural repair and evaluate adaptation of TFix and CodeXGLUE to target projects [40]. Their use of exact match as a reported measure illustrates a separate assessment choice: reproducing a reference edit differs from satisfying an application's behavioural contract. SWE-Gym supplies executable repository tasks and feedback for training software-engineering agents, and uses trajectories to train both agents and outcome-based verifiers [41]. An environment can therefore be a source of learning data as well as a site of evaluation. TSB evaluates already-configured systems; it does not isolate their training histories or estimate the benefit of training on a particular task environment. Comparisons need to state the task

population, the role of those tasks in development, and the acceptance rule. Success on the selected web applications does not establish transfer to unfamiliar project families, and a syntactically different patch need not be invalid when the public contract permits it.

2.3 Classification of Existing Code Generation Benchmarks

Software-engineering benchmarks differ in the unit of work and in the provenance of their tasks. HumanEval assesses synthesis from function specifications, while MBPP contains short natural-language programming problems [42, 43]. SWE-bench instead pairs real repository issues with codebases that must be modified [10]. These distinctions determine what a successful result establishes: a bounded implementation, a repository repair, or an interaction trajectory in a task environment.

Task Scope and the Unit of Success: The unit of work determines what a successful outcome can establish. Function synthesis assesses an implementation against a bounded specification and test set; issue repair assesses a change in the context of an existing repository. SWE-bench explicitly places the latter problem in real GitHub repositories, making it a relevant comparator for systems that must work with existing code [10]. The difference is not simply the number of lines generated. Repository repair adds the need to preserve surrounding behaviour while changing the part of the project responsible for the issue.

TSB uses constructed faults rather than a sample of naturally occurring issue reports. This choice makes the initial fault and required preserved state explicit, but it narrows the source of realism. A successful TSB repair meets the applicable assessed requirements under the declared scoring version; it does not establish complete contract coverage or resolution of an arbitrary production incident. Conversely, repository realism alone does not specify how to measure an architectural progression or a controlled information gap. These are separate design dimensions, which is why TSB is positioned as a complementary controlled setting rather than a replacement for repository-level benchmarks.

CrossCodeEval isolates one repository-level demand by using static analysis to construct completion examples that require cross-file context across four programming languages [44]. Its assessment includes reference-code and identifier matching. This makes the availability and use of external code context central to the task, while leaving application execution outside its primary assessment unit. TSB's repair contract asks a different question: whether the resulting application satisfies specified behaviour and structural constraints. A benchmark can therefore involve several files without evaluating service interactions, and it can evaluate an executable outcome without identifying how well an agent retrieved the relevant context.

RepoBench makes the separation explicit through retrieval, code-completion and combined-pipeline tasks [45]. This provides a useful contrast with TSB: the latter observes completed application repairs but does not independently score retrieval quality. CRUXEval further isolates input and output prediction for short Python functions [46]. Predicting program behaviour, retrieving context and repairing a running application therefore require distinct assessment units.

WebArena and OSWorld extend interaction-based evaluation beyond code generation. WebArena supplies self-hosted web applications with tasks involving information seeking, navigation and website changes, using task-specific answer and state checks [47]. OSWorld provides tasks

across real desktop applications and files, with task-specific initial configurations and execution-based evaluators [48]. These environments make successful action depend on the state reached through interaction. Their relevance to TSB lies in this outcome-oriented design and the need to specify an initial state. Their task populations remain different: operating an application through its interface does not necessarily require changing its implementation. Table 1 makes these assessment units explicit alongside a dialogue-and-tool benchmark and the present repair setting.

Table 1: Selected benchmarks assess different units of work. The comparison summarises task scope, interaction conditions, and assessment rather than ranking benchmark quality or reported scores.

Benchmark	Task object	Interaction and initial state	Assessment
CrossCodeEval [44]	Code statements requiring cross-file context	Completion from repository context; static examples rather than a running application	Reference code and identifier matching
WebArena [47]	Information seeking, navigation, and website changes	Browser actions on self-hosted websites with reproducible data and reset support	Task-specific answer matching and programmatic state checks
OSWorld [48]	Tasks across desktop applications and files	Computer interaction with task-specific initial configurations and live execution feedback	Custom execution-based checks of task outcomes
τ-bench [49]	Customer-service requests subject to domain policies	API actions and LM-simulated user dialogue over a task-specific database	Goal database state and required response content; repeated-trial reliability
TSB	Faulty web applications with preserved architectural constraints	Repository edits and execution after fresh reset; fixed brief and one predefined clarification pathway	Normal submission plus all applicable hidden functional tests and architectural invariants

Library Composition and Language Coverage: BigCodeBench broadens function-level evaluation through tasks that require composing calls from diverse libraries under detailed instructions [50]. This introduces dependencies between API choices and the requested behaviour, even where the output remains a bounded implementation. Such composition is relevant to application development, but it does not automatically require locating a fault in an existing repository or preserving a deployed service boundary. The distinction helps separate the complexity of the generated implementation from the surrounding maintenance process.

Multi-SWE-bench extends issue resolution across eight programming languages and analyses human-assigned difficulty and cross-file issues [51]. Language coverage, repair span, and assessed difficulty therefore provide additional axes for describing a benchmark. They should remain distinct from TSB's architectural tiers: a cross-file repair may stay within one service, while a small edit may affect communication between services.

Task Provenance and Temporal Scope: Task provenance also determines which uncertainty is controlled by the benchmark designer. Constructed faults make the initial defect explicit, while mined issues preserve evidence of actual maintenance requests. The latter still require curation of the information available to the system. Xia et al.'s examination of SWE-bench Lite identified descriptions containing missing information, misleading suggestions, or solution information [4]. Thus, an authentic issue report is not automatically a complete and neutral experimental brief. This motivates reviewing task information separately from verifying that a reference repair passes the tests.

More recent benchmarks also extend beyond short repairs. SWE-Bench Pro includes long-horizon repository tasks and augments problem statements with requirements and, where needed, interface specifications [52]. The SWE-CI preprint evaluates evolving codebases through repeated requirement-generation and implementation cycles, using test-based progress across iterations to assess maintenance [53]. TSB instead resets each repetition to a prescribed fault state. Its architectural tiers vary the repair setting; they do not measure the accumulation of technical debt across successive changes.

Benchmark freshness introduces a further choice between a stable comparison set and continuing task collection. LiveCodeBench uses time-stamped competition problems and evaluates generation, self-repair, code execution, and test-output prediction [54]. SWE-bench-Live instead collects repository issues through an updateable curation pipeline and supplies task-specific Docker environments [55]. The former expands the capabilities tested within competition problems; the latter expands the repository issue population over time. Both make task collection part of the evaluation design, but neither makes an unspecified collection date sufficient evidence that a particular model has never encountered related material.

A fixed benchmark version and a refreshed task stream answer different longitudinal questions. Holding the tasks constant makes changes in system outcomes easier to describe against the same problems. Adding newly collected tasks provides a different test of performance, while also changing the problem population. TSB adopts a fixed set for its within-study comparison and records system configurations and run conditions. Its constructed faults support control of those selected starting states; they do not constitute a direct audit of the evaluated models' training data or establish immunity to prior exposure.

Execution Feedback and Final Assessment: InterCode models code as actions and execution feedback as observations, using self-contained environments for interactive coding in Bash, SQL, and Python [11]. AgentBench broadens the evaluation of multi-turn agents across distinct task environments [23]. These studies establish that evaluation can include interaction with an environment rather than only a single generated answer.

Feedback should be described by its source and timing, rather than by the presence of an interaction loop alone. A compiler error, an observed HTTP response, and a hidden evaluator verdict expose different information. The first two can guide debugging without disclosing the complete acceptance procedure. Teaching Large Language Models to Self-Debug illustrates this separation within a code-generation protocol: its MBPP setting supplies the first unit test in the prompt and retains the other two for complete evaluation [56]. Passing the visible test and passing the assessment suite consequently answer different questions. The paper also uses different feedback forms depending on whether tests are available, so self-debugging should not be treated as a single

information condition. For TSB, the corresponding comparison concerns which observations can influence the next repair action and which judgements are reserved for final assessment.

Test provenance adds a related constraint. Just et al. compare tests supplied in bug reports with those committed alongside fixes and find that the latter can make fault localisation and repair appear more effective [57]. Consequently, describing a test as realistic is insufficient without specifying when its information becomes available.

The unit of an interaction also depends on the interface. A shell command, an IDE action, and a structured tool request need not represent comparable amounts of work. Consequently, action counts are most interpretable within a specified interface and logging convention. Cross-system comparisons require the definition of the counted event, including whether retries, failed calls, and permission interactions are included. This is why a shared task contract can support outcome comparisons without making every route-specific trace statistic directly interchangeable.

In TSB, information available during execution and information used for final assessment are deliberately separated. The agent can inspect its repository and use permitted execution feedback, while the hidden evaluator runs after submission or suspension. This separation makes it possible to record validation attempts without exposing the complete scoring procedure during the repair. It also means that a reported success rate is conditional on the information and tools made available during execution. Rates obtained with different feedback, budgets, task sets, or submission rules should not be read as directly interchangeable measures of model ability.

Environment Reset and Experimental Independence: A reproducible execution environment and a restored task state address different sources of variation. The current SWE-bench harness uses Docker-based isolated evaluation environments [58]. InterCode's official implementation likewise provides Docker environments and an explicit reset operation in its interaction loop [59]. These facilities show that containerisation and reset interfaces already form part of established evaluation practice. The relevant comparison is what state a protocol restores and when it restores it, rather than whether a benchmark mentions Docker.

StableToolBench addresses changing tool availability and responses through a virtual API system that combines cached results, real API calls and simulated fallback responses [60]. Its model-based evaluation is a separate part of the design. Stabilising the response source therefore does not by itself establish the validity of every success judgement. TSB takes a different approach to the executable environment by restoring local application and repository state. The distinction matters when interpreting repeatability: a preserved simulated response and a freshly executed service operation can support different observations even when both reduce uncontrolled variation. Reports should identify which parts of the environment are restored, replayed or simulated, and which services remain outside that control.

For a web application, restoring source files alone need not restore the conditions under which those files execute. Persistent records, database schemas, service configuration, and browser state can affect the observable result of a repair. The existence of an isolated container does not, by itself, document the treatment of every external or persistent component. Where the reviewed description does not specify a particular reset boundary, that boundary should remain unspecified rather than be counted as an absent capability.

TSB makes the initial application state part of the task package and validates the reset before repeated trials. This supports comparison from prescribed starting conditions, including the

selected database and service dependencies. Reflexion's retention of reflections across restarted episodes illustrates why restoring an environment does not necessarily clear an agent's memory [39]. Application state and agent context therefore need separate descriptions; TSB's fresh-run protocol does not provide direct access to a commercial service's hidden state. Nor does restoration make observations independent of their shared task or eliminate changes in an external model service. Environment restoration, task sampling, and configuration provenance require separate controls. The resulting comparison concerns repeatable executions of twelve specified faults, not a random sample of production incidents.

The TSB combines deterministic fault injection, resettable multi-container web applications, frozen evaluator-only tests, architectural invariants, and a predefined clarification protocol. The Level 3 and Level 4 tasks focus specifically on dependencies across frontend, backend, and database layers. This design is motivated by the architectural and debugging challenges described in microservices research [61, 62, 63]. Table 2 summarises this difference in emphasis.

2.4 Evaluation Methods and the Meaning of Correctness

Executable tests provide evidence about behaviour, but the strength of that evidence depends on the test suite and its relationship to the specification. EvalPlus demonstrates that augmenting code-generation benchmarks with additional tests can reveal incorrect programs that previously passed, and can alter comparative rankings [9]. This distinction matters when interpreting a high pass rate: the outcome is always relative to the checks actually applied.

Test Composition and Patch Overfitting: Program-repair research shows why the assessment suite needs its own scrutiny. Kong et al. examine how repair techniques, subject programs, and test suites affect repair outcomes, including sensitivity to the selected failing and passing tests [66]. Thus, test count alone cannot describe the information available to a repair process or the strength of its final assessment. A suite can contain many redundant checks while omitting a relevant boundary condition. For this thesis, the pertinent question is which required behaviour each check distinguishes, rather than whether a larger suite necessarily implies a more reliable ranking.

DiffTGen seeks additional inputs that expose behavioural differences and uses an oracle to turn an incorrect candidate's behaviour into a distinguishing test [67]. Ye et al. assess generated patches at scale using random tests based on human-written repairs [68]. These studies connect patch evaluation to the search for counterexamples beyond the original suite. Their methods also make the source of expected behaviour explicit: generating an input and deciding the correct output are separate tasks. Hidden assessment in TSB limits access to the scoring procedure during execution, but does not remove the need to justify the oracle used by that procedure.

Automated assessment can itself reject valid behaviour. The QuixBugs study compares patch-assessment methods and reports false positives for invariant-based assessment [69]. The large-scale assessment study additionally checks whether generated inputs violate program preconditions when investigating disagreements [68]. These findings motivate examining the cause of a failed check before equating it with a defective repair. For TSB, this supports maintaining a trace from each acceptance condition to the task contract and preserving the reason for a scoring correction.

Wang et al. compare static and dynamic patch-correctness assessment and find complementary strengths that vary across projects and repair techniques [70]. This supports checking several

Table 2: Six dimensions of evaluation design: selected prior approaches and the bounded scope of TSB. Entries describe documented emphases, not an exhaustive capability audit or performance ranking.

Dimension	**Selected prior approaches**	**TSB design and boundary**
Task provenance	HumanEval/MBPP: bounded synthesis problems; SWE-bench: repository issues; Defects4REST: real REST defects [42, 43, 10, 64].	Twelve constructed faults across four architectural tiers; controlled initial conditions rather than an estimate of industrial fault prevalence.
Interaction feedback	InterCode: code actions and execution observations; SWE-CI: requirements and implementation across CI iterations [11, 53].	Permitted repository and execution feedback during a run; evaluator-only final assessment.
Environment reset	SWE-bench: isolated Docker evaluation; InterCode: Docker environments with a reset interface [58, 59].	Versioned repository and application-state restoration before each trial; shared task identity remains across repetitions.
Success criteria	EvalPlus: stronger functional tests; SWE-Bench Pro: repair and regression tests; SWE-CI: progress over iterations [9, 52, 53].	Required functional and architectural checks plus normal submission; diagnostic checks remain distinguishable from end-to-end success.
Architectural constraints	Integration-fault research characterises service-boundary failures; this differs from specifying an agent's permitted repair structure [63, 65].	Task-specific invariants preserve required service and data boundaries; the selected tiers do not cover all distributed-system faults.
Clarification conditions	CodeClarQA: questions and answers for underspecified descriptions; SWE-Bench Pro: augmented requirements supplied with tasks [7, 52].	One designated information gap with a predefined answer; complete briefs for the other eleven tasks, excluding routine permission approvals.

kinds of evidence while retaining the limitations of each assessment method.

The oracle problem gives a broader account of this issue. Barr et al. describe the challenge of distinguishing intended behaviour from incorrect behaviour and review sources of oracle information, including specifications and contracts [12]. More checks do not automatically resolve an ambiguous requirement. A check can miss a fault, but it can also encode a restriction that the public specification did not impose. For the present thesis, this implies that evaluator review must consider both the coverage of required behaviour and the justification for rejecting an alternative implementation.

This leads to a useful distinction between behavioural requirements and incidental features of a reference repair. A required output or persistence rule describes what the application must do. A variable name or the layer in which a default is assigned may instead be an implementation choice unless the task contract makes it necessary. TSB's uniform review policy uses the public brief, agent-visible repository requirements, and authorised response card to make that distinction.

Architecture introduces an additional dimension of acceptability. A response can look correct while bypassing a prescribed service or replacing stored data with a constant. In the selected TSB tasks, architectural invariants rule out such contract-violating workarounds. Functional and architectural checks are reported separately so that a reader can distinguish failure to produce the requested behaviour from failure to preserve the required structure. Their separation does not establish two fully independent constructs, because an architectural violation can also affect functionality; it makes the assessment criteria visible.

Success criteria also depend on the temporal unit being scored. SWE-Bench Pro checks both resolution of the target issue and preservation of existing functionality, whereas SWE-CI aggregates progress across an evolving sequence of implementations [52, 53]. A binary result on one submitted repair and a trajectory-based maintenance score answer different questions. Comparing their numerical values would require aligning the task population, available information, execution budget, and scoring unit first. In this thesis, prior benchmark results therefore establish relevant evaluation designs rather than numerical baselines for the four configured systems.

ToolSandbox evaluates intermediate and final milestones in stateful tool-use conversations [71]. This makes progress through dependent operations inspectable, while using a task population and interaction protocol different from application repair. TSB retains action and final-state evidence as separate diagnostics rather than adopting ToolSandbox's milestone score.

The stopping rule is another part of the outcome definition. A system that reaches a working state but does not complete the required submission procedure has a different end-to-end outcome from one that submits that state successfully. TSB therefore retains the stopping reason alongside the test results. This allows the results to describe passing checks after a limit stop without silently reclassifying that run as successful. Comparisons with other studies must take account of this completion requirement rather than assume that every reported success metric uses the same terminal condition.

Repeated-trial metrics distinguish consistent completion from the availability of a successful candidate. τ-bench evaluates agents that use domain tools while conversing with a simulated user, checking the goal database state and required response content [49]. Its pass^k metric concerns success on all k trials, whereas pass@k concerns at least one success among k attempts. SWE-Gym makes a further distinction through Best@k: success of the candidate selected by a learned verifier, rather than merely the existence of a correct candidate in the generated set [41]. A

system can discover an acceptable repair yet fail to select it for delivery. TSB scores the final submitted artefact under its completion rule, not the best intermediate candidate that an evaluator could retrospectively identify. The repetition unit, candidate-selection procedure and submission condition must therefore be specified before comparing success metrics.

Repeated success is also only one dimension of agent reliability. Rabanser et al. distinguish consistency across repetitions, robustness to perturbations, predictability of failure, and the severity or safety implications of errors [72]. Repeating a fixed task does not directly test the response to changed instructions or faulty tool returns. Practical evaluation additionally needs to consider resource use: AI Agents That Matter argues for evaluating cost alongside accuracy and distinguishing the needs of model researchers from those of application developers [73]. Success rate, elapsed time and monetary cost describe different properties. TSB retains repeated outcomes and timing, but its data do not support a complete cost comparison or a comprehensive reliability assessment. These additional dimensions guide interpretation and future evaluation rather than change its existing success endpoint.

Evaluation adequacy can be examined along two directions. One asks whether an incorrect implementation can pass because important behaviour is untested; the other asks whether a valid implementation can fail because the tests impose an unstated restriction. EvalPlus provides evidence for the first problem through stronger test suites [9], while the oracle literature motivates relating assessment to the intended specification [12]. The second question requires inspecting the requirement behind each rejection, rather than assuming that agreement with a reference implementation is sufficient. The reviewed TSB scores apply this distinction to a common public contract; the scoring amendments and sensitivity analysis are reported with the experimental evidence.

CheckList approaches evaluation coverage through a matrix of capabilities and test types, demonstrating behavioural testing beyond aggregate accuracy in NLP [74]. This provides a useful design principle for interpreting software-agent checks: a larger suite is informative when the added cases exercise meaningful obligations or conditions. The principle does not imply that an NLP capability taxonomy transfers unchanged to application repair. For TSB, persistence, accepted-input behaviour and preservation of service boundaries are candidate obligations to describe explicitly. Counting tests without their relationship to those obligations would conceal whether many checks cover the same narrow behaviour.

The Agentic Benchmark Checklist adds a distinction between task validity and outcome validity [75]. The former concerns whether task success measures the intended capability; the latter concerns whether a positive evaluation actually corresponds to task success. Its audits identify problems in task construction and reward design across agentic benchmarks. This framework makes two reviews necessary: assessing what the selected repair tasks represent, and checking whether the evaluator accepts the right final states. A reproducible task can still admit a shortcut unrelated to the intended capability, and a stable evaluator can still encode an inappropriate acceptance rule. TSB's controlled fault design, public contracts and scoring review address specific parts of this chain. Their coverage and remaining limitations require separate discussion rather than a single assertion that the benchmark is valid.

2.5 Cross-Service Debugging and Integration Faults

Microservice research provides evidence that debugging dependencies between services is a substantive software-engineering problem. Zhou et al. combine an industrial survey, replicated fault cases in a benchmark system, and an empirical investigation of debugging practices; their findings motivate tracing and visualisation support for fault analysis [63]. This work studies the evidence developers need to investigate distributed behaviour. It motivates examining the diagnostic evidence available in integration tasks.

Operational diagnosis provides a more specific comparison for the evidence used in cross-service debugging. MicroRCA relates application performance symptoms to resource utilisation through an attributed graph of services and machines [76]. MicroHECL follows possible anomaly-propagation chains in service-call graphs and ranks candidate causes using correlations [77]. MicroRank instead combines normal and abnormal request traces with weighted spectrum analysis to localise latency issues [78]. Their common output is evidence about where a performance or operational problem originates. A ranked service is not a submitted code repair, and localisation accuracy cannot be substituted for TSB's end-to-end success criterion. The comparison nevertheless makes an important dependency explicit: useful diagnostic feedback must connect an observed symptom to the components that could produce it.

Gregor et al. develop their integration-fault taxonomy from a literature review and ten practitioner interviews, followed by validation with sixteen additional practitioners [65]. The taxonomy distinguishes faults associated with service descriptions, deployment, discovery, composition, connection, and execution. This organises a broader fault space than a single application example can represent. Its categories are useful for explaining a task's failure mechanism, but their presence in practitioner accounts does not give a probability distribution from which TSB's twelve tasks were sampled.

The source of an executable workload is a separate design choice from the source of its faults. DeathStarBench supplies modular, end-to-end microservice applications for studying implications across the hardware and software stack [79]. Such workloads expose interactions among services, but an application suite alone does not specify a repair obligation, an allowed patch, or a stopping rule for an agent. Filibuster addresses another part of this chain: service-level fault injection extends functional tests to explore failures in service interactions using static analysis and execution-guided exploration [80]. Its target is resilience under injected service failures. TSB's constructed code and configuration faults instead persist until the relevant implementation is changed. Both support controlled failure conditions, but they exercise different obligations: handling a disrupted dependency and repairing a defective application should not be treated as interchangeable tasks.

Stateful API testing further shows why a boundary check may require a sequence of operations. RESTler infers producer–consumer dependencies from an API specification and uses response feedback to explore request sequences [81]. A later request can depend on a resource created by an earlier one, so isolated endpoint calls may not reach the relevant state. This mechanism is pertinent to TSB's import, read-back and persistence obligations, although TSB does not implement RESTler's fuzzing procedure. It motivates distinguishing the ability to reach a state from the oracle that judges that state. Resetting a task and executing its assessment sequence must preserve the intended dependency order; a successful individual request is insufficient evidence for the entire contract. RESTTESTGEN complements this sequencing perspective by generating nominal and error scenarios from Swagger interface definitions [82]. Together, these studies

motivate specifying both the state needed to exercise an operation and the expected treatment of its inputs. TSB uses task-specific checks rather than either automated test generator.

Real-defect benchmarks provide another relevant comparison. Defects4REST mines and organises faults from REST API projects and supplies access to buggy and patched versions for controlled testing and debugging research [64]. This offers evidence grounded in developer-fixed defects. TSB instead prioritises a small, explicitly constructed set of faults with controlled resets and common agent-run procedures. These approaches make different trade-offs between the provenance and diversity of defects and the precision with which a particular experimental condition is specified.

These studies also separate the origin of a fault from the method used to reproduce it. Zhou et al. replicated industrial fault cases in a benchmark system, while Defects4REST provides developer-fixed defects from existing projects [63, 64]. Reproduction can therefore combine a controlled execution setting with evidence from real faults. For TSB, the corresponding methodological choice is to prioritise transparent, constructed fault conditions. Its rationale rests on control over the selected comparison, while external validity depends on how those conditions relate to other applications and fault types.

Cross-file scope, cross-service behaviour, and architectural compliance are three distinct properties of a repair. A change spanning several files can remain entirely within one component. A service-boundary failure can arise from a small mismatch between a producer and consumer, while a behaviourally plausible workaround can bypass a required component. Integration-fault research motivates examining boundary behaviour [63, 65]; it does not automatically determine which implementation structures a particular task must preserve. Those restrictions need to follow from the task contract.

In TSB's upper tiers, an API may return data that a frontend interprets incorrectly, or application code may retain references to a removed database field. These examples require evidence across component boundaries rather than only inspection of the edited file. HTTP, browser, persistence, and schema checks make the selected contracts observable. Their coverage remains limited to the specified applications and dependencies: the tiers do not establish a universal ordering of software difficulty or test deployment-scale concurrency and operational traffic.

2.6 Clarification Behaviour and Human–AI Interaction

Technical correctness remains central to code benchmarks, but interactive tools also introduce communication costs and opportunities. Conversational assistants support interactive software-development dialogue [83]. Copilot user research also reports difficulties understanding, editing, and debugging generated code [22]. These findings motivate recording clarification behaviour rather than treating every human input as equivalent assistance.

Clarification research addresses specifications whose intended behaviour cannot be recovered from the initial description alone. Li et al. introduce CodeClarQA, pairing Python-generation descriptions with synthetic clarification questions and answers [7]. Its relevance lies in distinguishing detection of missing intent from implementation of the supplied intent. ClarifyGPT examines a more explicit sequence: it uses consistency between sampled programs' behaviours to detect possible ambiguity and then generates targeted clarification questions [84]. Its real-feedback study provides participants with expected input–output examples when eliciting answers. Thus, the information available to the respondent is part of the experimental condition, not an incidental

detail of asking a question. These studies motivate separating ambiguity detection, question selection, answer provision and implementation. TSB narrows that sequence to one designated missing requirement and a predefined authorised response; it does not evaluate unrestricted question generation against respondents with varying access to the intended solution.

Deciding when to ask also requires distinguishing uncertainty about an answer from uncertainty about the user's intent. Testoni and Fernández examine uncertainty-guided clarification in collaborative dialogue and find differences between human questioning and model uncertainty [85]. Zhang and Choi explicitly model ambiguity over interpretations while accounting for the cost of interaction in deciding whether clarification is useful [86]. These studies caution against interpreting either low confidence or frequent questioning as sufficient evidence of appropriate clarification. Their dialogue and language-task settings differ from repository repair, but they help specify what a clarification opportunity represents: information that can select among plausible task interpretations, rather than any situation in which execution is difficult. CLAMBER organises ambiguous information needs through a taxonomy and evaluates both ambiguity identification and clarification [87]. Its broader coverage highlights the restricted scope of TSB's single omitted business rule.

Question quality can additionally be assessed through what the answer enables. Rao and Daumé III rank clarification questions by modelling the expected usefulness of their answers in StackExchange discussions [88]. Zhang, Knox and Choi train clarification behaviour using preferences derived from possible future conversation outcomes [89]. Both connect question selection to the value of subsequent information, although they use different settings and objectives. For TSB, a qualifying request can obtain the permitted business rule without producing a correct implementation. Recording these events separately describes that distinction without estimating the causal benefit of an answer.

SWE-Bench Pro takes a different approach by augmenting the initial problem statement with requirements and relevant interface details [52]. This reduces ambiguity before execution rather than making acquisition of the missing information the target behaviour. Comparing these designs explains why a task can be demanding without containing an intentional information gap. Architectural complexity and requirement completeness are separate dimensions; success or failure on one does not, by itself, measure the other.

Programming studies show why human contribution and assisted performance need several measures. Grounded Copilot distinguishes acceleration, where programmers have a next step in mind, from exploration, where suggestions help them investigate possible approaches [90]. Nguyen et al. examine beginning programmers' difficulties in expressing intent, evaluating generated code and revising prompts, even with correctness feedback [91]. Why Johnny Can't Prompt documents informal experimentation and assumptions drawn from human communication in non-AI experts' prompt design [92]. In a 32-participant study, Nam et al.'s GILT assistant improved task progress relative to web search, while completion time and measured code understanding did not show statistically significant gains [93]. The outcomes can therefore diverge: progress on an assisted programming task does not by itself establish faster completion or improved understanding. The study's unfamiliar-library tasks and human users also differ from autonomous repository repair. For TSB, this reinforces the need to identify the outcome actually measured instead of interpreting a repair pass rate as evidence about all aspects of collaboration.

For a controlled clarification comparison, the source of an answer matters alongside its content.

A fixed response constrains how much assistance the evaluator can supply, whereas an open dialogue can adapt to earlier questions and introduce additional information. TSB adopts the former design to make the permitted information transfer inspectable. Its repetitions test whether systems recognise and use the same designated gap; they do not provide multiple independently sampled types of ambiguity. This limits generalisation even if a system behaves consistently across all repetitions.

For the present comparison, asking a question is not intrinsically good or bad. A request can target information deliberately withheld by the task, seek information already supplied, or ask the evaluator to perform part of the debugging. These behaviours have different implications for assistance. Counting all requests together would conceal whether the system identified a genuine specification gap. Equally, counting every routine permission approval as clarification would mix product operation with the transfer of task information.

TSB consequently distinguishes the opportunity to ask, the relevance of the request, and receipt of an authorised response. Opportunity capture concerns whether the system asks when the designated gap is present; request precision concerns the proportion of its requests that qualify; unnecessary-request rate concerns requests on complete briefs. These definitions make the denominators explicit. A system that asks nothing can have no unnecessary requests without demonstrating that it recognises missing requirements. A system that asks a qualifying question can still fail to implement the answer.

Observed professional use further distinguishes the purpose of interaction from the amount of interaction. In a one-week study of 24 software engineers, Khojah et al. describe ChatGPT use for artefact manipulation, expert consultation and training [94]. These purposes include seeking conceptual guidance as well as obtaining immediately usable code; the observational study does not establish a causal productivity gain. TSB's evaluator cannot iteratively redesign a request, coach the agent or co-develop a repair. Its fixed response consequently represents one constrained information transfer within a broader space of professional assistance, rather than an evaluation of user satisfaction, learning or the quality of an extended collaborative session.

A useful interpretation therefore keeps three outcomes distinct: recognising a specification gap, obtaining the permitted answer, and implementing the resulting requirement. They can diverge within the same run. A relevant question followed by an incorrect repair indicates a different limitation from silent guessing, even if both end in failure. This separation motivates reporting clarification behaviour alongside task correctness instead of combining them into one score. The eligibility and coding rules are specified in Section 4.3.

2.7 The Research Gap

The reviewed studies provide complementary evidence about isolated synthesis, repository repair, interactive execution, evaluation adequacy, integration faults, and clarification. Recent long-horizon benchmarks and the SWE-CI preprint further show that repository complexity and maintenance over time are already active evaluation targets [52, 53]. The research opportunity addressed here is to bring selected aspects of these concerns into one controlled comparison of configured systems. TSB combines a tiered debugging focus with deterministic architectural faults and repeatable multi-container resets through 12 debugging tasks spanning four architectural tiers.

These strands motivate three linked questions. For RQ1, repository-level repair benchmarks establish executable assessment, while integration-fault studies identify dependencies across

components [10, 63, 65]. An overall repair score alone does not show how success varies across the architectural tiers considered here. A controlled tier comparison therefore helps locate where the evaluated systems cease to satisfy the full repair contract.

For RQ2, clarification research shows how missing intent can be elicited through questions [7, 84]. Repair success alone cannot distinguish appropriate clarification from a successful guess, or reveal unnecessary requests on complete briefs. Measuring both behaviours makes the assistance boundary explicit. TSB examines this behaviour through one predefined information gap and 11 complete briefs.

For RQ3, agent-interface design and evaluation adequacy affect what reported performance means [5, 9]. Comparisons across different tasks, assistance rules, and scoring conditions cannot by themselves establish the relative performance of the four configurations studied here. A common task set and evaluation protocol therefore support a bounded comparison of success, time, and clarification behaviour. The comparison concerns configured systems, rather than attributing differences solely to their underlying models.

The study also isolates a reproducible subset of interaction behaviour. One predefined Level 2 task deliberately withholds one non-technical business requirement that is essential to a correct implementation. This task has a single pre-written answer, while the other 11 task briefs contain the information required to proceed. The clarification-task identifier, omitted field, eligibility rule, and response card were validated in the pilot and frozen before the main experiment. Across four systems and three repetitions, this design produces 12 clarification opportunities without adding extra benchmark tasks. Unexpected omissions outside this design remain task-specification defects. The study asks:

- **RQ1 (Tiered Task Correctness):** How does task success change as debugging tasks progress from static websites to multi-container full-stack applications?
- **RQ2 (Clarification-Seeking Behaviour):** When a predefined information gap is present, how often do evaluated systems request the required clarification, and how often do they make unnecessary requests on complete task briefs?
- **RQ3 (Comparative System Performance):** How do Cursor, Devin Desktop, GPT-5.4, and Qwen2.5-Coder-7B-Instruct differ in run-level success, execution time, clarification behaviour, and success without evaluator clarification responses on L3/L4 tasks?

In RQ3, success without evaluator clarification responses requires normal submission and passage of all applicable checks on L3/L4 tasks. This outcome retains the prespecified robustness definition, but does not independently measure recovery: when no applicable run receives a response, it coincides with success on those tiers.

3 Technical Background

This chapter introduces tool-mediated actions, dependencies between application components, and evaluation against task requirements. These concepts connect an agent's actions to the application state assessed at the end of a run. The emphasis is on observable interfaces and behaviour. The exact task settings, limits and scoring procedures are specified in Methodology.

3.1 Architecture of Agentic IDEs

Earlier AI coding assistants were commonly evaluated as code-completion systems [1, 2]. Research prototypes of software-engineering agents extend this interaction model by combining repository inspection, editing, command execution, and iterative feedback [4, 5, 35].

Retrieval-Augmented Generation conditions generation on retrieved external information [95]. Software-engineering agents use repository search and structure-aware navigation to supply code context, then use execution feedback and, when available, tests to guide subsequent actions [5, 35]. The study characterises Cursor and Devin Desktop through the interfaces and behaviour visible during each run: file operations, command execution, approval prompts, and diagnostics. Proprietary indexing, retrieval, model orchestration, and private reasoning are outside the observable evidence.

Context acquisition and context use are separate stages. A repository retriever selects candidate evidence, such as a definition or related call site. The generation stage receives a bounded presentation of that evidence. RepoCoder's iterative retrieval and Repoformer's selective retrieval illustrate choices about updating this evidence and deciding whether another retrieval is useful [27, 28]. Access to a repository therefore does not imply that every relevant dependency is present in the current input. Conversely, including more text need not improve the evidence available for the particular decision.

Lost in the Middle demonstrates sensitivity to the position of relevant information in long inputs on multi-document question answering and key–value retrieval tasks [96]. This finding distinguishes context capacity from effective information use, but does not establish a coding-agent failure mechanism. In this thesis, a context limit describes an operational boundary of the runner. It does not measure retrieval quality, prove that the relevant file was considered, or explain a failed repair on its own. Observable searches, file reads and tool results support a more limited account of the evidence exposed during a run.

An agentic execution loop connects the requested change, application state, actions and observations. The task brief describes the desired change, while the repository and running application supply the current state. An action specifies an operation, and the tool result returns an observation. ReAct provides a method-level example of interleaving generated language steps with actions that obtain observations from an external environment [97]. Subsequent actions can use an observation to narrow a search, revise a patch or run another check. Proposing a command in generated text does not establish that it executed. The evaluation record needs to establish what the environment actually accepted and what state followed.

Repository context and execution feedback also serve different purposes. Reading a source file can reveal a field name or a dependency, while executing the application can reveal whether that dependency behaves as expected in the current environment. Neither observation substitutes completely for the other. A plausible source-level explanation still needs to be connected to an observable repair, and a failed command may report a setup problem rather than a failure of the changed application.

3.2 Evaluation Baselines: API-Based Agents

The study evaluates four end-to-end systems: Cursor and Devin Desktop in their native IDE environments, plus GPT-5.4 and Qwen2.5-Coder-7B-Instruct as API-based agents. Cursor and

Devin Desktop use their default agent model or mode and default settings at the configuration-freeze date; no manual model override is applied. The two API agents use the same custom wrapper, prompt format, logical action space, execution limits, and logging schema. GPT-5.4 is called through the OpenAI Responses API using the dated snapshot `gpt-5.4-2026-03-05`[3], and Qwen2.5-Coder-7B-Instruct is loaded from a pinned revision of `Qwen/Qwen2.5-Coder-7B-Instruct`[4] on a managed Hugging Face Inference Endpoint with vLLM[5]. The pilot established the initial endpoint, version and inference records; later amendments and replacement freezes identify the configuration applicable to each run.

For this study, reliable agent–tool interaction is a core baseline requirement. Toolformer demonstrates learning when and how to call APIs and incorporate their results [98]. In the TSB, text generation alone is insufficient: the agent must inspect a repository, invoke bounded tools, use execution feedback, and produce a verifiable final system state. This action–feedback design is informed by interactive evaluation work [11]. Provider-specific transport adapters map model responses to the same validated logical operations for file reading, patching, command execution, and clarification requests. Transport differences are logged and do not change the available action space.

Qwen2.5-Coder-7B-Instruct provides the open-weight API configuration, and GPT-5.4 provides the second API configuration under the same wrapper. The comparison records success, time and observable tool use; token usage and auditable cost are supplementary where the execution route exposes them.

The shared Python Finite State Machine (FSM), detailed in Section 4.3, standardises interaction with the sandbox and records observable tool calls, outputs, timestamps, and final outcomes. This supports a controlled comparison between the two API agents and a broader end-to-end comparison with Cursor and Devin Desktop under the common task protocol.

Figure 2 distinguishes the two execution routes and the boundary between agent-visible feedback and evaluator-only assessment.

3.3 Tool Actions, Execution Context and Observable Progress

The API wrapper separates a model response from an accepted operation. A transport adapter interprets the provider's response, and the runner validates the resulting logical action before performing it. At this boundary, the request must identify an available operation and supply its required arguments. Prose describing an intended edit does not meet those requirements. The resulting record distinguishes proposed activity from accepted file access, command execution or modification.

API-Bank's separation of planning, retrieval and invocation helps describe these stages without treating tool use as a single event [25]. For a fixed wrapper, the corresponding observable questions are whether the requested action is available, whether its arguments are accepted, and whether its execution produces the intended effect. These conditions can diverge. A valid file-read operation can return irrelevant code, while an accepted command can exit with an error. The action log identifies the boundary crossed; application checks establish selected properties of the resulting state.

[3] https://developers.openai.com/api/docs/models/gpt-5.4
[4] https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct
[5] https://huggingface.co/docs/inference-endpoints/about

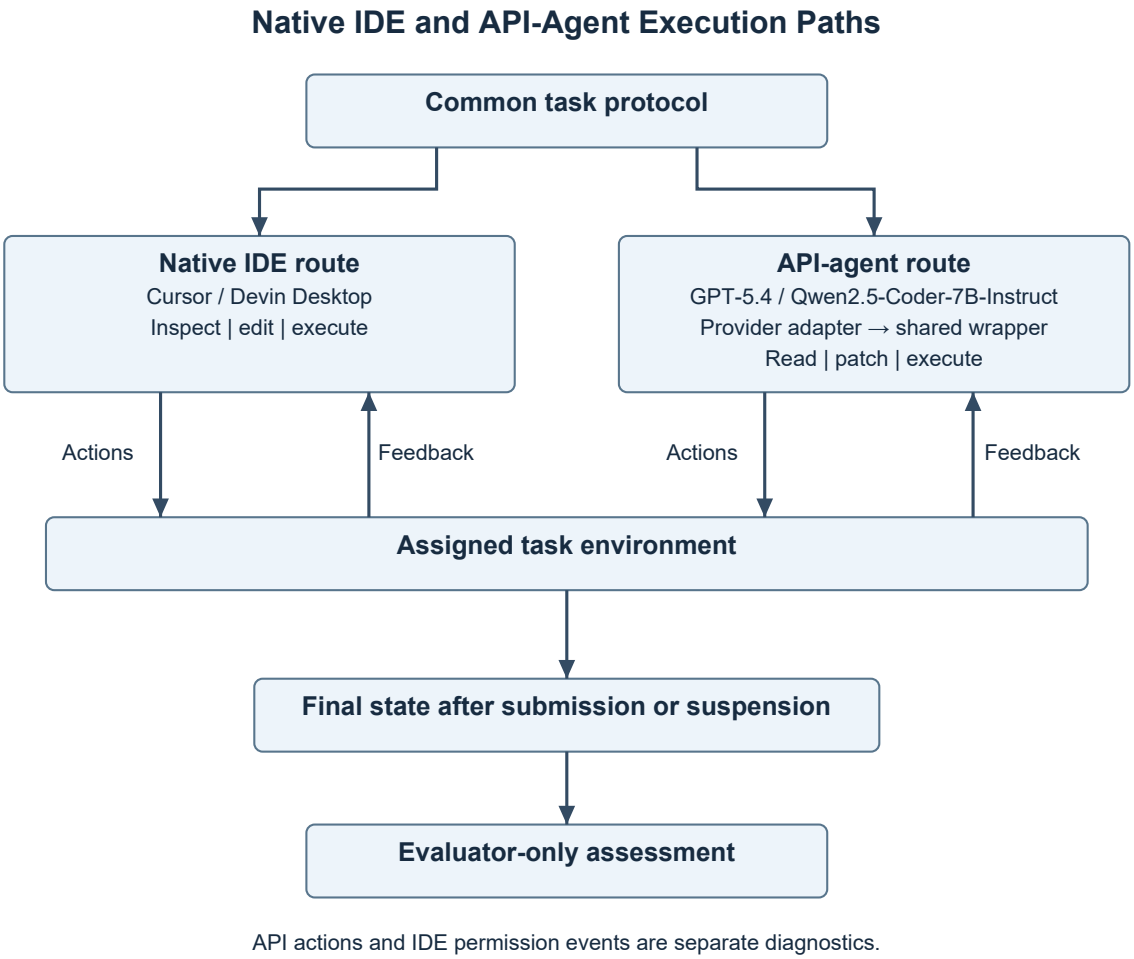


Figure 2: Native IDE and API-agent execution paths under the common TSB task protocol. Cursor and Devin Desktop operate through their native interfaces, while GPT-5.4 and Qwen2.5-Coder-7B-Instruct use provider-specific adapters and a shared API wrapper. Both routes exchange actions and observable feedback with the task environment before final-state assessment. Clarification requests follow the separate authorised response protocol in Figure 5. API actions and IDE permission events remain separate diagnostics; proprietary internal reasoning is not represented.

Execution context determines what an accepted operation refers to. A relative file path is interpreted from a working directory, while a command runs in a particular environment with its available dependencies. For the TSB, these concepts matter because the repository can contain several components and the target application can execute inside containers. Inspecting a similarly named path outside the intended project does not establish that the task code has been read. Likewise, a validation command that does not start cannot establish whether the application passes its intended test. The record of the operation and its result is therefore needed alongside the model's description of what it attempted.

An edit and a completed repair are separate milestones. A final diff records changes to tracked task code, but does not establish that the application starts or meets its requirements. Conversely, a service can satisfy some checks in its initial faulty state because those checks concern unaffected behaviour. The experiment consequently needs both evidence of the submitted state and evidence of its assessed behaviour. This distinction helps explain why observable progress categories and diagnostic checks accompany success without replacing it.

The FSM distinguishes acting, clarification, submission and suspension before evaluator-only assessment. These are execution states rather than judgements of repair quality. Section 4.3 specifies the transitions and completion rules; the quantitative metrics in Methodology define how the terminal state and assessed behaviour determine success.

3.4 Application Layers and Cross-Service Contracts

The application layers in the benchmark describe different responsibilities. The frontend presents information and issues requests; the backend handles application operations and constructs responses; persistent storage retains data beyond an individual request or process lifetime. An API contract connects these responsibilities through expected inputs, outputs and behaviour. For example, a frontend binding and a backend response need to agree on which field carries a displayed status. A disagreement can leave the user-facing feature broken even when the response is otherwise well formed.

Integration-relevant faults are recognised as a distinct concern in microservice testing; Gregor et al. organise such faults in a practitioner-informed taxonomy [65]. The TSB uses a narrower set of executable cases to illustrate related boundary problems. Its frontend–backend tasks include origin-policy, base-path and response-field mismatches. These cases require an assessment at the boundary where a component consumes another component's output. They should not be read as reproducing the organisational scale, traffic patterns or fault prevalence of industrial microservices.

A symptom and its origin can occupy different points in a service dependency. MicroHECL's use of anomaly-propagation chains illustrates why an abnormal caller need not be the originating component [77]; MicroRank similarly uses request traces rather than treating each service observation in isolation [78]. For a repair task, this creates two evidential steps. The agent first needs observations that identify a plausible cause, then needs validation that the changed application satisfies the required interaction. A plausible diagnosis supports where to investigate; it does not itself establish that the final producer and consumer agree. TSB's boundary checks concern the second step and should not be interpreted as a separate measurement of root-cause ranking accuracy.

The database layer introduces a further contract between stored structure and application expectations. An application may refer to a column through a query, an object–relational mapping,

a response schema and a frontend display. In the benchmark's schema-cascade case, a prescribed migration has already removed a field while downstream references remain. The repair obligation is to bring those references into agreement with the required schema. Restoring the old field could remove a local error yet violate the stated migration requirement. This example illustrates why the allowed final architecture matters in addition to whether an immediate exception disappears.

Persistence also differs from having the right value in memory during a single execution. A storage task can require data to remain available after a service restart, and import semantics can specify replacement rather than accumulation. These are different obligations from returning a successful HTTP response to an import request. The TSB's persistence cases make the distinction concrete: the relevant behaviour spans writing, later reading and the required restart boundary. The task mapping in Methodology identifies which of these obligations applies to each case.

An interaction contract can also constrain the order and lifetime of operations. RESTler makes request dependencies explicit by using values produced by earlier API calls when constructing later calls [81]. Applied conceptually to this benchmark, an import followed by a read examines a different obligation from either request alone, while a restart adds a persistence boundary. The evaluator must specify which initial state and operation sequence make the obligation observable.

The lifetime of a failure must likewise be distinguished from the lifetime of the test environment. Filibuster explores application resilience by injecting service-level failures during test execution [80]. A TSB code or configuration defect instead belongs to the initial task state and remains until repaired. Restarting a healthy dependency after an injected interruption and resetting a repository to its defective starting version therefore serve different purposes. Keeping these concepts separate explains why repeatable containers support experimental control without turning every task into a test of production fault tolerance.

Containers provide an explicit execution boundary for these compact applications. In the TSB, the versioned configuration connects service processes with the files, settings and stored state required by the task. Resetting only an edited file would not necessarily restore the complete experimental starting condition if the run had also changed stored data. The reset procedure therefore treats the repository and execution state as parts of the task package. This makes repeated attempts comparable at their starting point; it does not make the agents' subsequent choices deterministic.

3.5 Requirements, Test Oracles and Architectural Checks

A test oracle supplies the criterion for deciding whether an observed result is acceptable [12]. In this benchmark, the public brief states the repair obligations, while evaluator-only checks assess selected consequences of those obligations. Keeping the checks outside the agent-accessible repository separates performing the task from editing the machinery that judges it. The hidden location of a check does not create a new requirement: its expected behaviour still needs to correspond to the disclosed contract or an authorised clarification.

Functional checks and architectural invariants examine related aspects of the final system. A functional check may ask whether an operation returns or stores the required value. An architectural invariant may ask whether the required service, schema or cross-layer relationship has been preserved. The two families can overlap in their practical consequences, so they are not assumed to be independent dimensions. Their separation makes it easier to interpret a repair that produces a plausible response while bypassing a required dependency or retaining an incompatible

data structure.

The distinction is behavioural rather than a preference for one code style. Where a requirement specifies an externally visible outcome without specifying an implementation layer, different implementations can satisfy it. A static source check can be informative when the contract requires a particular structural property, but matching a string alone is not the definition of application correctness. The evaluator-review procedure in Methodology explains how the study handles identified mismatches between public obligations and scoring conditions.

3.6 Clarification and the Boundary of Evaluator Assistance

Clarification concerns information needed to determine the intended task, rather than any obstacle encountered while executing it. Li et al.'s work on underspecified Python generation provides an example of making missing intent an explicit part of the interaction [7]. The TSB applies a constrained version of that idea: a designated business-rule gap can receive a predefined answer, whereas the included technical faults remain problems for the agent to diagnose. The distinction depends on what the task discloses and permits, not simply on whether the agent uses a question mark.

The distinction depends on what additional evidence would resolve the uncertainty. Execution feedback may establish whether an endpoint fails or a stored value survives a restart, but it cannot select an omitted business preference unless that preference is encoded elsewhere in the disclosed task. Clarify When Necessary treats interaction as a way to resolve competing interpretations while considering its cost [86]. In TSB, the predefined response supplies that missing choice. It does not supply a technical diagnosis, a patch, or a new exception to the architectural requirements. The content and authority of the answer are therefore part of the experimental condition.

An answer has instrumental value only if the subsequent work uses it appropriately. Future-turn modelling in clarification research makes this downstream perspective explicit [89]. For this thesis, it motivates preserving the trace from an eligible request to the authorised answer and then to the assessed submission. These are distinct observations, not interchangeable measures of understanding. The protocol records information delivery and assesses final behaviour against selected checks. It cannot directly observe how a proprietary system represented or reasoned about that information.

Permission handling has a different role. A request to authorise a tool operation concerns whether an action may proceed; a clarification response supplies task information. Counting routine IDE approval prompts as clarification would therefore combine two different events. The common protocol records them separately and keeps approval time within the end-to-end timing boundary. The exact eligibility rule and response policy appear in Methodology; their conceptual purpose is to make the evaluator's contribution identifiable across execution routes.

Finally, the absence of an evaluator response describes an interaction condition, not the quality of every decision made within it. A system can work without a response because the brief is complete, because it does not ask, or because its request is outside the authorised condition. Success without a response consequently needs to be interpreted together with the task's opportunity to ask.

4 Methodology

4.1 Overview of the Multi-tier Sandbox Architecture

This study uses a controlled multi-tier sandbox to evaluate system-level debugging. The workflow has four elements:

- **Frozen Task Package:** Each task provides a public brief, a versioned starting repository, and a reset procedure. Evaluator-only hidden tests and architectural invariants are stored separately from the agent-accessible repository.
- **Evaluated Systems and Resettable Execution Environment:** Cursor, Devin Desktop, GPT-5.4, and Qwen2.5-Coder-7B-Instruct receive the same fixed task brief and operate within a resettable environment. The environment ranges from Level 1 static websites to Level 4 multi-container applications, and is restored to the same initial state before every independent run.
- **Clarification Exception:** L2-02 is the only task with a predefined essential information gap, providing the opportunities for measuring clarification capture. The other 11 tasks, including L2-01 and L2-03, have complete briefs and provide the observations for measuring unnecessary requests. Only an authorised request on L2-02 receives the pre-written response; all repairable technical faults must be resolved autonomously.
- **Evaluation Layer:** After submission or suspension, the evaluator applies frozen hidden tests and architectural invariants to the final system state. It records binary run-level success as the primary outcome, together with functional and architectural diagnostic results, elapsed time, stopping reason, and clarification events.

Figure 3 shows how a frozen task package supplies the public brief and each system operates on a fresh resettable environment. The evaluator assesses the final state after submission or suspension. The single predefined clarification pathway supplies a frozen response to an eligible request; all other evaluation occurs post-run. Section 4.3 and Figure 5 specify execution, clarification, submission, suspension, and execution limits. This design produces comparable runs and traceable evidence across the four systems.

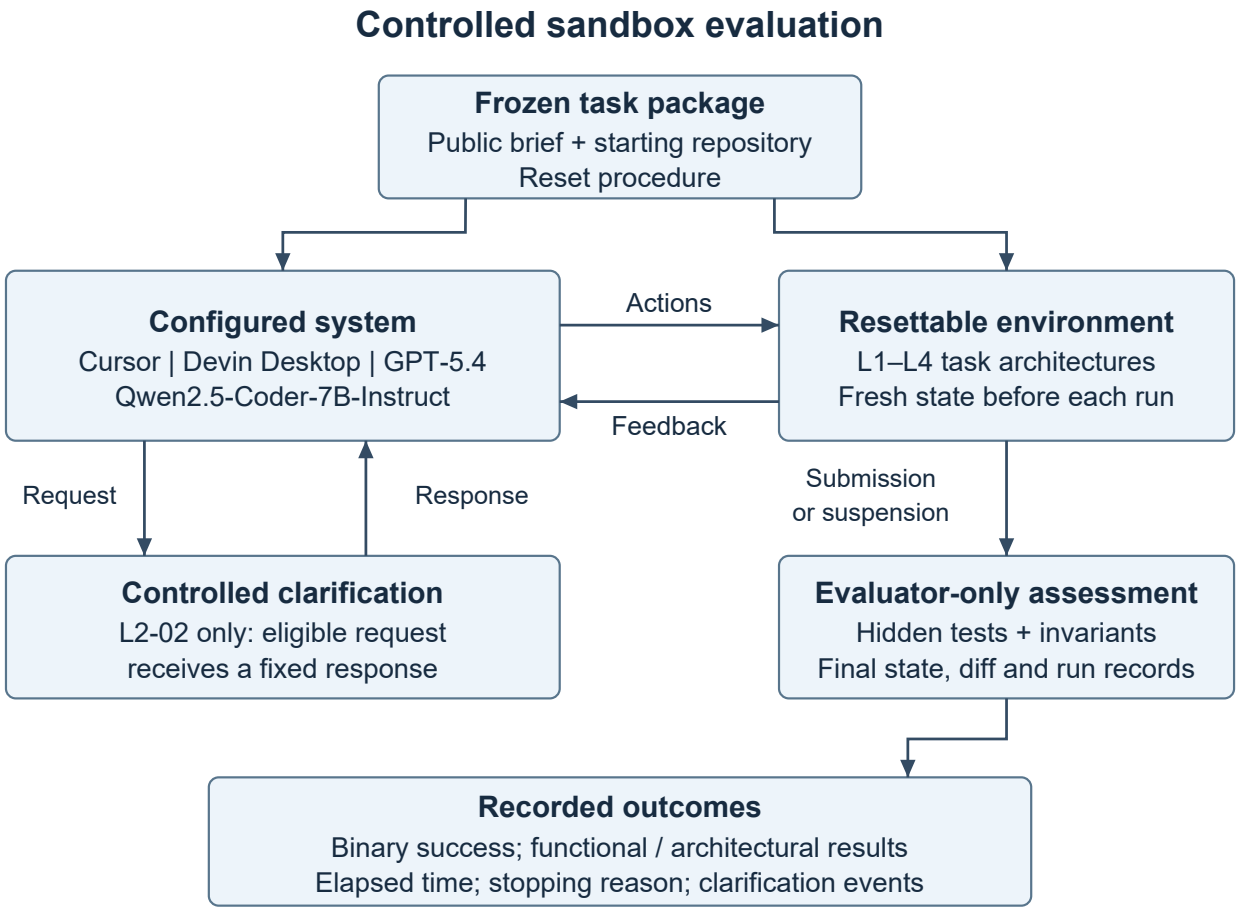


Figure 3: Frozen-task, resettable-environment workflow. Each configured system acts on a fresh task environment and receives execution feedback. Only an eligible request on L2-02 receives the predefined clarification response. After submission or suspension, evaluator-only tests and architectural checks assess the final state; run records supply complementary diagnostic and interaction evidence.

Object of Evaluation: The object of evaluation is the complete repair process available through a configured system. A run begins with a faulty repository and a public description of the required behaviour, and ends with a submitted or suspended repository state. Producing a plausible explanation, identifying a likely faulty file, or proposing a patch in conversational text is insufficient if the required change is not applied to the task environment. This distinction makes repository state and observed application behaviour central to the evaluation. The protocol measures whether the system can turn the supplied evidence into an applied repair that passes the declared assessment within its operating constraints.

The benchmark separates three questions that would otherwise be obscured by a single final score. First, did the run reach an eligible completion state? Second, did the resulting application meet the functional requirements? Third, did the repair preserve the required architecture? The stopping record answers the first question, whereas the evaluator checks answer the second and third. Keeping these records separate makes it possible to distinguish an incomplete interaction from a submitted but incorrect repair, and a functional workaround from an acceptable architectural solution. The binary outcome combines these requirements only at the final success decision; the

underlying observations remain available for diagnosis.

4.2 System-level Execution Environment and Task Progression

Docker Compose provides the execution and reset mechanism for 12 deterministic debugging tasks, with three tasks at each of four architectural tiers. Each task injects a specified fault into a versioned starting repository, giving the evaluator control over the initial state and enabling exact resets. The synthetic design prioritises experimental control; real-world transfer is addressed in the external-validity discussion.

Each task package contains an immutable start commit, a deterministic fault-injection procedure, a public task brief, evaluator-only hidden tests stored outside the agent-accessible repository, predefined architectural invariants, a reset procedure, and a known-good oracle repair. Before main collection, all 12 task packages demonstrated the cycle `reset -> intended failure -> oracle repair -> all checks pass -> reset -> same failure`. Docker Compose is therefore designed to restore services, volumes, and database state, while the validation cycle provides evidence that the reset is deterministic. The four tiers are:

- **Level 1: Static Websites.** Three HTML/CSS tasks cover asset paths, responsive layout, and client-side validation without backend services.
- **Level 2: Single-Service FastAPI Applications.** Three tasks cover routing, request or response validation, backend logic, and local persistence within one application service.
- **Level 3: Vue–FastAPI Multi-Service Applications.** Three tasks cover separated frontend and backend services, including CORS configuration, API base paths, and frontend–backend contract mismatches. These are repairable faults and do not justify evaluator assistance.
- **Level 4: Vue–FastAPI–PostgreSQL Multi-Container Applications.** Three tasks cover schema migrations, foreign-key behaviour, default values, database-host configuration, and cross-layer consistency after a database change.

Level 4 tasks require resolving cross-layer dependencies. The schema-cascade case begins from an injected initial fault: a database migration has already removed `users.age`, while legacy ORM models or queries, REST API request or response schemas, and frontend bindings still refer to it. The permitted repair scope is limited to updating these downstream references; the migration must not be rolled back and `users.age` must not be reintroduced. Figure 4 depicts the transition from this faulty starting state to an acceptable repaired state. A run succeeds only after normal submission and passage of all applicable functional checks and architectural invariants, including the continued absence of `users.age`.

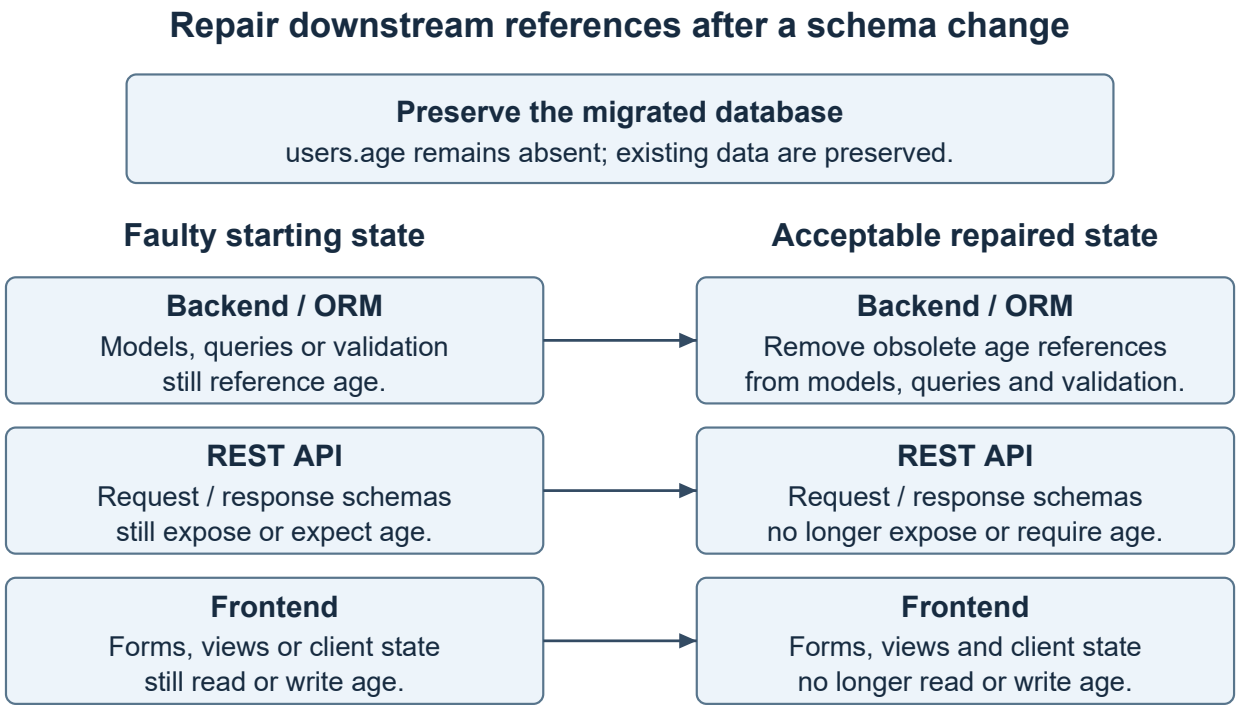


Figure 4: Injected Level 4 schema-cascade task. The migration has removed `users.age`; the repair updates dependent Backend/ORM, REST API and frontend references while preserving the migrated schema and existing data. Horizontal arrows compare faulty and repaired states, not a required edit sequence. Success requires normal submission and passage of all applicable functional checks and architectural invariants, including the continued absence of `users.age`.

Task Coverage and Architectural Boundaries: The tiers organise the location and extent of the dependencies that a repair may need to preserve. At Level 1, the relevant relationship can be between a document and an existing asset or script. At Level 2, request handling and stored application state enter the repair scope. At Level 3, a functioning backend is not sufficient if the frontend uses an incompatible endpoint, response field, or access policy. At Level 4, the application must additionally remain consistent with the prescribed database state and service configuration. The progression therefore describes distinct architectural settings rather than a calibrated scale on which each successive task is necessarily harder.

The selected tasks also distinguish visible symptoms from the location of the injected fault. For example, an empty frontend field can coexist with a correct backend response, while a database connection failure can surface as an unavailable dashboard. These cases require the final state to be assessed at the affected application boundary rather than solely by checking whether a changed file looks plausible. The task specifications and validation reports identify the intended initial failure, the preserved requirements, and the observations used to establish recovery.

Mapping Tasks to Repair Evidence: Table 3 makes the selected coverage explicit. It summarises the researcher-known fault and the required repaired behaviour; it does not imply that the agent received the fault location or the hidden checks. Each row is one task specification, evaluated through three fresh runs per system. The mapping also separates the application boundary at which a failure becomes visible from the component in which a change might be

implemented.

Table 3: Task coverage and required repair evidence. These summaries complement, rather than replace, the full public contracts and versioned evaluator records.

Task	Fault boundary	Required behaviour and preserved constraints
L1-01	Document references to image assets	Existing local images load through corrected asset references.
L1-02	Responsive CSS and navigation markup	Navigation remains usable at the specified 390-pixel mobile viewport.
L1-03	Script loading and form behaviour	The contact form loads its script and handles invalid input, valid submission and reset.
L2-01	Route parameter and handler binding	The existing item-detail endpoint returns the required response rather than a validation error.
L2-02	Request validation and stored due date	An omitted or empty due date is accepted and stored as null under the authorised business rule.
L2-03	Application working directory and storage	Import/export and subsequent reads use the intended persistent store, including after restart.
L3-01	Browser origin and backend access policy	The supplied frontend can access the backend without adding unrelated origins or a wildcard.
L3-02	API response field and frontend binding	The order status supplied by the API is rendered correctly by the frontend.
L3-03	Frontend request path and backend route	Password reset reaches the required backend endpoint directly and returns the expected response.
L4-01	Removed database field and dependent layers	Downstream references to the removed age field are repaired while its absence and existing data are preserved.
L4-02	Database status constraint and application contract	Customer creation supplies the required default status, restricts allowed values and preserves existing records and unique email behaviour.
L4-03	Database hostname and service discovery	Backend startup, health and dashboard access recover using the configured database service.

The task-specific preserved constraints prevent success from being achieved by removing the behaviour being evaluated. Examples include retaining the migrated database state, preserving stored records, and keeping the intended frontend–backend connection. These constraints must still follow the public contract. In particular, a default-value requirement specifies the accepted application behaviour unless the contract explicitly requires a particular implementation layer. A researcher-authored oracle and its file-edit sequence cannot supply an additional, undisclosed restriction. The uniform review procedure below applies this distinction to every task.

The mapping supports RQ1 through task and tier coverage, RQ2 through the single deliberate information gap in L2-02, and RQ3 through the repeated comparison of all systems on the same rows. It does not constitute a factorial manipulation of individual fault characteristics. For example, service count, persistence technology and fault family vary together across parts of the benchmark. A tier-level difference can therefore locate an observed performance pattern but cannot isolate which of these characteristics caused it.

Validation of the Starting and Repaired States: Task validation checks two distinct properties: a reproducible faulty starting state and the existence of a repair that satisfies the

evaluator. Repeating the reset after applying the oracle establishes that the repaired state has not silently become the starting point for a later run. Depending on the task, the recorded evidence includes source checks, HTTP responses, browser behaviour, persistent storage, and database contents. These observations serve different purposes. A source check can detect a prohibited dependency or retained reference, whereas a browser or persistence probe can establish whether the application exhibits the required behaviour across its execution boundary.

An oracle repair provides a positive control for the task package, not a unique answer that every agent must reproduce. The test-oracle problem concerns determining whether observed behaviour agrees with the intended behaviour [12]. In this benchmark, the researcher-authored repair demonstrates one accepted route from the faulty state to a working state. It does not establish that every alternative implementation is invalid or that the checks exhaust all possible defects. Consequently, task-package validation and the later public-contract review address different risks: the former checks that the task can be executed and solved, while the latter checks whether the scoring criteria impose restrictions beyond the stated repair contract.

Agents interact with the tiered environment through an Agent-Computer Interface (ACI) [5]. Cursor and Devin Desktop are evaluated through their native IDE environments using the default agent model or mode and default settings available at the configuration-freeze date, with no manual model override. GPT-5.4 is evaluated through the OpenAI Responses API using the dated snapshot `gpt-5.4-2026-03-05`. Qwen2.5-Coder-7B-Instruct is evaluated from a pinned revision of the official `Qwen/Qwen2.5-Coder-7B-Instruct` repository on a managed Hugging Face Inference Endpoint using vLLM. The two API configurations use the same custom wrapper, logical tool interface, execution protocol, and logging schema.

The study therefore has two frozen execution routes. Cursor and Devin Desktop run through their native IDE interfaces, including the product-controlled orchestration available in those interfaces. GPT-5.4 and Qwen2.5-Coder-7B-Instruct run through the shared API wrapper. Route-specific observables are kept separate, but the public brief, initial repository, reset, task-level time limit, clarification rule, and evaluator are common across all four systems.

The study compares end-to-end systems under a common task protocol. For each system, the record binds the product or model version, displayed default model or mode, provider endpoint, pinned model or repository revision, controllable system prompt, visible inference settings, available tools, evaluation date, and pricing information. Provider-specific transport adapters expose the same validated logical actions and preserve raw responses in the audit log.

Qwen's deployment used vLLM 0.26.0 with the Hermes tool parser and a recency chat template that requested actions inside `<tool_call>` tags. The template prohibited Markdown fences and bare JSON. When a response contained no native structured tool call, the wrapper's frozen `qwen-fenced-tool-json-v3` fallback could convert a restricted subset of textual requests into logical actions. This fallback required a single-line JSON object with the specified tool name and argument schema, in an accepted fenced or unfenced form. A multiline object could therefore be rejected even when its tool name and arguments were otherwise acceptable.

Each system contributed three selected valid terminal observations for every task. The main experiment therefore comprises 12 tasks $\times$ 4 systems $\times$ 3 repetitions $=$ 144 selected valid observations. Pilot runs are separate and are not included in this total.

Before each main run, the runner restores the immutable start commit and resets the applicable containers, volumes, and database. It records the system and model identifiers, provider or product

version, task and repetition, prompt and configuration hashes, starting commit, timestamps, and applicable container versions. After submission or suspension, evaluator-only tests and invariants are run against the final state, and the final diff, logs, test results, container state, database checks, clarification events, and stopping reason are retained.

The 144-run schedule is blocked by task and repetition. Within each task–repetition block, a seeded random permutation orders the four systems. The frozen seed and schedule distribute temporal ordering across systems.

Repetitions and the Scope of Comparison: The balanced layout gives every system the same number of observations on every task. With three repetitions per task, each task contributes equally to a system's overall success rate, irrespective of how many hidden checks that task contains. The repetitions expose whether the configured system reaches the same terminal outcome across fresh attempts at the same problem. They do not expand the number of distinct problem specifications: the study still contains twelve tasks, including only three tasks within each tier. For this reason, the analysis retains task identity when comparing systems and reports individual task outcomes alongside pooled summaries.

There are also two different meanings of control in this design. The task contract and final evaluator are shared controls, while the interfaces remain part of the systems being compared. A native IDE may organise searches, context, permissions, and tool execution differently from the API wrapper. Applying a common brief therefore does not make these internal processes identical. The comparison estimates performance under the recorded system configurations; it does not isolate the effect of model weights, orchestration, or interface design. A model-only comparison would require a different experiment in which those surrounding components were held constant.

The seeded within-block ordering reduces reliance on a fixed sequence in which one system always runs first. It is not a guarantee that every temporal factor is balanced, and it does not remove the need to record configuration changes. Fresh resets address carryover in the task repository and service state, whereas version records and the amendment chain document changes in the execution setting. These controls solve different problems and are interpreted separately when assessing reproducibility.

Control Conditions and Residual Variation: Table 4 distinguishes what the protocol standardises from what remains part of the evaluated system. This distinction is necessary when interpreting both success and elapsed time: equal task exposure does not imply identical orchestration, inference latency or observable action granularity.

Alignment of Research Questions and Experiments:

RQ1 (tiered task correctness): Compare binary success across the four architectural tiers, using three tasks and three fresh repetitions per task for each system (nine scheduled outcomes per system and tier). Task-level results and separate functional and architectural pass proportions show where repairs fail. Because fault families change with tier, this is a descriptive progression across the selected tasks, not a causal estimate of architectural complexity alone.

RQ2 (clarification seeking): Compare the 12 scheduled runs of the single Level 2 information-gap task with the 132 runs of the 11 complete-brief tasks. Opportunity capture, request

Table 4: Scope of experimental controls and the variation they leave unresolved.

Dimension	Recorded control	Remaining boundary
Task exposure	Same brief, starting repository and reset procedure for each task	Different tasks deliberately contain different faults and dependencies.
Final assessment	Common submission rule and task evaluator; review applied across systems	Passing the finite check set is not proof of freedom from all defects.
Information access	Frozen clarification eligibility and response; hidden evaluator kept separate	Native products and the API wrapper manage context differently.
Execution budget	Common 5,400-second task limit; shared API-wrapper limits	Equivalent internal action and command limits are unavailable for native IDEs.
Run ordering	Seeded system order within task–repetition blocks	Temporal position is not guaranteed to be perfectly balanced.
Configuration	Version, endpoint, settings and amendments recorded per run	Product-controlled orchestration and remote service behaviour are not fully reproducible.

precision, and unnecessary-request rate measure distinct behaviours. The opportunity-capture result describes this one deliberately incomplete task; it cannot establish a general clarification rate across other kinds of missing requirements.

RQ3 (system performance): Compare the four end-to-end systems across the 36 task–repetition blocks, retaining the 12 tasks as the units for paired inference. The primary contrast is the difference in run-level success rates; elapsed time, clarification behaviour, and success without evaluator clarification responses on the 18 Level 3–4 runs per system provide separate outcomes. The comparison concerns the frozen system and interface configurations rather than isolated foundation models.

Run Validity and Replacement Attempts: A run is invalid only when an evaluator mistake, runner failure, billing or provider-access failure, or other infrastructure failure outside the assigned task prevents delivery of the promised brief or environment. The invalid attempt and all available evidence remain immutable, its lifecycle is recorded as `invalidated`, and it is excluded from success analysis. A replacement increments only the attempt suffix, returns to the original task–repetition and seeded order position, uses a fresh reset, and preserves the same frozen system configuration. Agent-caused service failures, tool errors, execution-limit stops, and unsuccessful repairs remain valid unsuccessful outcomes and are not repeated. If the cause cannot be removed without changing a frozen component, collection pauses until an append-only amendment and a replacement freeze define the affected runs and their disposition.

4.3 Human-Agent Interaction Protocol and State Machine

The interaction layer uses a deterministic Finite State Machine (FSM). The protocol distinguishes appropriate clarification of a deliberately omitted business requirement from unnecessary requests on otherwise complete task briefs. To support reproducibility, the implemented protocol applies

fixed logic-gating rules and records every interaction between the agent and evaluator. Execution feedback remains available within the assigned environment [11].

For the API-based agents, as illustrated in Figure 5, the wrapper models the run through an Execution State, a limited Clarification State, an explicit Submit transition, a Suspension State, an evaluator-only Evaluation State, and terminal Success or Failure outcomes. During the Execution State, the agent may inspect files, modify code, execute commands, and run tests. Observable actions, command outputs, tool results, and timestamps are recorded by the API-agent wrapper.

Normal completion follows the path Execution → Submit → Evaluation when the agent emits the `<SUBMIT_FIX>` marker. The wrapper then stops further agent actions and freezes the final system state. Reaching the common task-level wall-clock limit or an applicable API-wrapper safety limit instead moves the run from Execution to Suspension; the stopping reason is recorded, further agent actions are prohibited, and the same final-state freeze is applied. Both Submit and Suspension therefore lead to the evaluator-only Evaluation State. A valid run enters Success only when it ends through normal submission and every applicable frozen hidden test and architectural invariant passes. A valid run stopped at an execution limit enters Failure even if all checks pass; the check results remain diagnostic evidence. Evaluation is post-run and does not provide intermediate debugging feedback.

A clarification request can be authorised only when it targets the single essential information gap deliberately embedded in one predefined Level 2 clarification-opportunity task. The gap concerns a non-technical business requirement rather than a repairable fault, and the task has one pre-written response. The task identifier, omitted field, accepted-request rule, and response card are validated during the pilot and frozen before the main experiment. The other 11 briefs contain the information required to proceed. An unforeseen essential omission outside the frozen design is handled under the invalid-run rule rather than scored as agent performance.

An authorised request receives only the task-specific pre-written response; debugging hints, patches, additional hidden-test results, and unsolicited guidance are prohibited. After the response, the run continues from its current state and remains eligible for ordinary success scoring. Every request, decision, response identifier where applicable, and timestamp is recorded before the agent returns to the Execution State without resetting the environment.

If a clarification request does not identify the frozen information gap, or if it occurs on a complete brief, it is recorded as not authorised. No additional information is provided, and the agent remains in or returns to the Execution State. This procedure distinguishes appropriate uncertainty management from unnecessary information seeking and prevents clarification from becoming debugging assistance.

Technical faults are not treated as clarification triggers. In particular, CORS errors, API-base-path errors, database-host configuration errors, schema-migration failures, and API-contract mismatches are intentionally included as repairable benchmark faults. An agent encountering these faults must diagnose and repair them autonomously while remaining in the Execution State.

To ensure a comparable stopping rule across all evaluated systems, every run is governed by a common task-level wall-clock limit of $T_{task} = 5{,}400$ seconds. The pilot did not justify changing this limit, so it is frozen for the main experiment. If the limit is reached, no further agent actions are permitted and the run enters the Suspension State. The runner records the applicable stopping reason, freezes the final system state, and transitions the run to the Evaluation State, where the same frozen hidden tests and architectural invariants used for submitted runs are applied.

For API-based agents, the frozen main-experiment wrapper enforces a hard maximum of 30 observable actions and a hard 180-second timeout for each terminal command. One observable action is one accepted wrapper tool invocation: `execute_bash`, `read_file`, `patch_file`, or `request_human_clarification`. If a model response contains multiple tool calls, each call counts separately. The `<SUBMIT_FIX>` marker does not count as a tool action, and provider retries that never reach the tool executor are logged but not counted. The action threshold is the single documented pilot-to-main adjustment: the provisional 15-action limit was increased to 30 after pilot evidence showed repeated termination of progressing GPT-5.4 runs at the provisional boundary. The final 30-action threshold and 180-second command timeout are hard limits applied identically to both API agents.

The API wrapper also suspends a run after three consecutive model responses that produce no accepted tool action, authorised clarification request, or valid submission marker. It retains at most 100,000 characters of interaction context; older retained interaction material is truncated according to the frozen context policy. These controls bound unproductive response cycles and context growth without treating either quantity as a performance score. API-agent commands execute in the assigned repository sandbox with external network access and host-Docker control disabled. Evaluator-only validation remains outside the agent-accessible environment.

Clarification as a Controlled Information Intervention: The clarification opportunity in L2-02 concerns the business rule for a missing or empty `due_date`. An eligible question asks whether the field is required or how an omitted value should be represented or persisted. The authorised response specifies that the field is optional, missing or empty values must be stored as null, and supplied dates must follow the required date format. Requests for a file location, a diagnosis, or an expected patch do not qualify. The response supplies the missing requirement without selecting an implementation strategy for the agent.

Work on clarification in Python code generation motivates distinguishing an incomplete specification from a failure to implement a known requirement [7]. Here, that distinction is operationalised through one frozen response card and an explicit request-acceptance rule. The protocol does not permit an open-ended conversation in which the evaluator progressively repairs the agent's understanding. A run can ask the appropriate question and still produce an incorrect final state; conversely, an agent can produce a correct repair without issuing an authorised request. Clarification capture and final success are therefore separate observations rather than alternative definitions of the same outcome.

Budget and Paid-Execution Controls: Provider budgets, account limits, and per-run reservations control paid execution. Usage is reserved before each provider action and settled afterwards; token use and monetary cost are recorded when exposed. Authorised headroom allows a valid run to reach a frozen stopping rule and remains fixed during the run. An account or billing interruption triggers the invalid-run policy. Any prospective increase is recorded in the paid-execution ledger and, when it changes a frozen component, in an append-only amendment and replacement freeze.

The API-wrapper limits bound execution and define deterministic suspension points. Reaching the action, command-timeout, no-progress, or task-time limit records the reason, stops further actions, and sends the frozen final state to the same Evaluation State used after normal submission. API tool-action counts and native-IDE permission events remain separate interface diagnostics.

The environment is reset before each independent run, never during one.

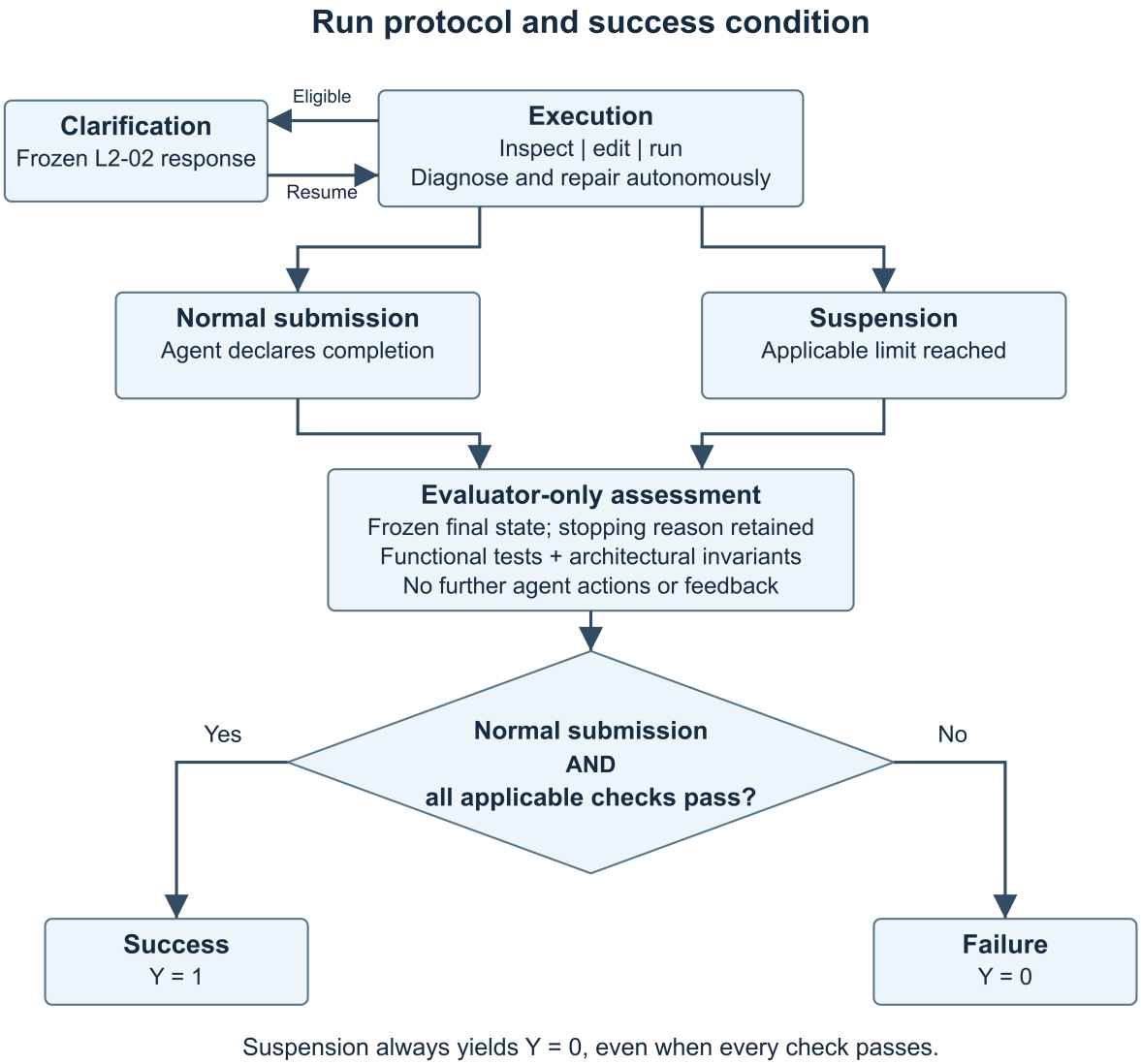


Figure 5: Run protocol and success condition for valid runs. An authorised request for the frozen L2-02 information gap receives the predefined response before execution resumes. Normal submission and suspension both lead to evaluator-only assessment of the frozen final state. Success requires normal submission and passage of every applicable functional test and architectural invariant; suspension yields failure even when all checks pass. Check results and stopping reasons remain available as diagnostic evidence.

Figure 5 summarises the control flow. The terminal decision combines two necessary conditions: normal submission and passage of all applicable checks. A normally submitted run with any failed check receives $Y = 0$, as does every suspended valid run, regardless of its check results. Retaining the stopping reason alongside diagnostic check results prevents a passing final state from being mistaken for successful completion under the run protocol.

Cursor and Devin Desktop are evaluated through their native IDE workflows; GPT-5.4 and Qwen2.5-Coder-7B-Instruct use the shared API-agent wrapper. All four systems receive the same frozen brief, starting repository, reset procedure, task-level wall-clock limit, network policy, clarification rule, and post-run evaluator. The clarification protocol and task-level stopping rule apply to every system. API action-count and command-timeout limits apply to the API agents

because equivalent native-IDE controls are unavailable. Permission events and API tool calls remain separate interface-specific measures.

For all systems, intervention burden is measured only through evaluator-mediated assistance under the predefined clarification protocol. The evaluator records each clarification request, whether it satisfies the clarification rule, and whether a pre-written response is issued. Normal interactions required to operate a commercial IDE, such as submitting the initial task prompt or responding to a standard tool-permission prompt, are handled mechanically under a frozen policy: the evaluator approves or rejects only the displayed action and does not add code, diagnosis, hints, or task information. Each permission event and decision is logged with a timestamp. These events are not counted as clarification or human assistance, while time spent awaiting a decision remains inside the native-IDE wall-clock measurement. IDE permission-event counts and accepted API tool-action counts are retained as separate diagnostic measures and are not directly compared.

4.4 Pilot Study and Main-Experiment Configuration Freeze

Qwen's pre-main checks covered different parts of the interaction path. A three-request smoke test verified a text sentinel, conversion of a `read_file` request through the earlier v1 fallback, and a continuation after a linked tool result. That result was supplied by the smoke program; the test did not read an actual task file. The v3 validation subsequently replayed 630 preserved provider responses offline, without new model requests or tool execution. These checks established limited protocol round trips and adapter behaviour. Successful file access and repair still had to be assessed from the task runs. The later configuration audit found that the template and normalizer matched their frozen references and that the two recorded deployment revisions retained the same effective settings; the later amendment rebound the same endpoint URL.

The pilot covered one selected run in each system–tier cell: four evaluated systems across four architectural tiers, for 16 selected pilot runs. Pilot runs are excluded from the 144-run main experiment and are used only to validate task execution, evidence capture, reset behaviour, and configuration limits. The audit trail retains 27 pilot attempts because superseded or replacement attempts are preserved as provenance; exactly one attempt is selected for each of the 16 cells. All 16 selected runs passed the common evidence validators, and the eight Cursor and Devin Desktop runs also passed the native-IDE evidence validator. The selected pilot outcomes comprise seven successful and nine unsuccessful runs. With one selected run per system–tier cell, these outcomes are descriptive process evidence and are not used for system ranking, confidence intervals, or inferential comparison.

The provisional 15-action API limit was reached in all four selected GPT-5.4 pilot runs. Three were suspended without a valid submission marker, while the Level 4 run reached a final state that passed the evaluator checks but still lacked the required valid submission marker. This pattern showed that the provisional threshold could terminate an otherwise progressing run, so the amendment log records the single pre-freeze increase from 15 to 30 actions. No selected pilot run reached the 5,400-second task limit or the 180-second per-command timeout; the longest selected run lasted 1,603.297 seconds. These observations supported retaining both time limits.

The three-response no-progress rule is retained because it provides a deterministic termination condition for repeated non-acting responses. The maximum retained context observed in the selected Qwen pilot evidence was 23,025 characters, well below the frozen 100,000-character boundary. Accordingly, the context threshold is retained as a conservative operational bound

rather than described as empirically stress-tested. External network access and host-Docker control remain disabled for API agents so that agent actions stay within the assigned task repository and cannot affect evaluator infrastructure.

The pilot-to-main decisions and technical prerequisites were completed before main collection. All 12 task packages and start commits, the 105-file evaluation bundle, the seeded 144-run schedule, and the analysis plan were bound by the strict main freeze, with subsequent changes retained in its amendment chain. Main data collection was completed on 26 September 2026: all 144 scheduled task–system–repetition slots have a selected valid terminal observation. Each tier contributes 36 observations and each system contributes 36, comprising three repetitions on each of the 12 tasks. Invalidated attempts remain in the audit trail and are excluded from performance denominators; valid unsuccessful outcomes remain in the dataset. A selection check confirmed 144 unique slots after applying the recorded validity adjudications. The completed comparative analysis is reported in Section 5.

Figure 6 summarises how pilot validation and run-validity decisions lead to the selected main-analysis dataset.

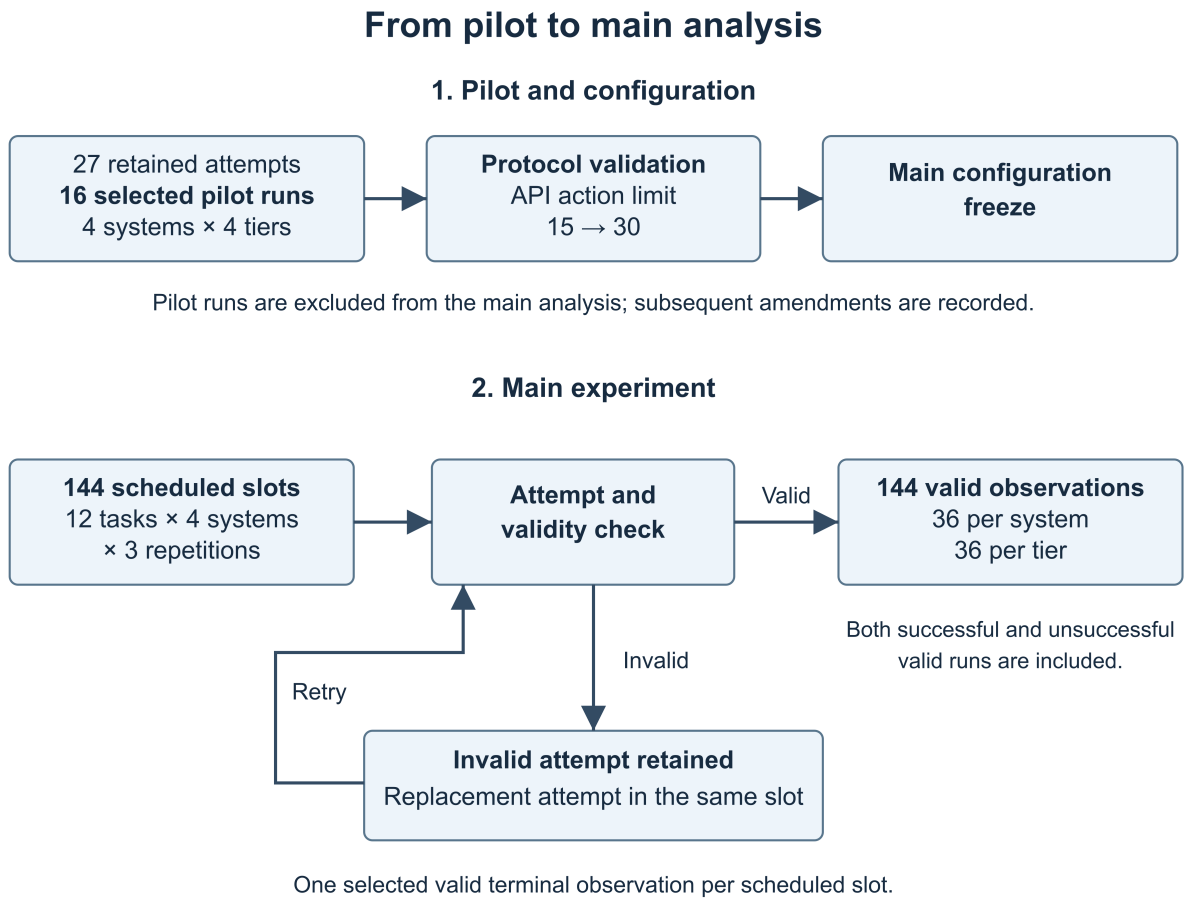


Figure 6: Provenance of pilot and main-analysis observations. The pilot retains 27 attempts and selects 16 runs, one per system–tier cell, for protocol validation; pilot observations are excluded from the main analysis. The main design comprises 12 tasks, four systems and three repetitions, yielding 144 selected valid terminal observations. Invalid attempts remain in the audit trail and are replaced within the same slot; valid unsuccessful outcomes remain in the analysis. Recorded amendments preserve configuration provenance.

Evidence Provenance and Amendment Chain: The researcher task package, agent-visible workspace, and evaluator-owned run directory are separate trust zones. Each run identifier binds the exact task prompt, starting commit, system and configuration identifiers, timestamps, transcript or raw model responses where available, adapter decisions, clarification and permission events, final diff, stopping reason, and hidden-evaluator outputs. Native timing, permission, and clarification journals are append ordered and hash-chained; run records bind their chain heads. Frozen configuration manifests bind the exact bytes of the task, prompt, limits, evaluator, schedule, and analysis components. A post-freeze change is recorded in an append-only amendment whose entry links to the preceding entry and to a new replacement freeze. Superseded manifests and invalid attempts are retained rather than overwritten, allowing every reported result to be traced to the configuration and evidence that produced it.

Uniform Evaluator Review: After collection, the evaluator was checked across all tasks against the public briefs, agent-visible repository contracts, and authorised clarification responses. Behaviourally equivalent repairs are not rejected solely for source-code syntax, variable names, or an unspecified implementation layer. Any evidence-supported correction applies uniformly to every system and repetition for the affected task. The analysis records the reviewed scoring version separately from the original execution freeze and retains the original scores for sensitivity analysis; agent submissions, timing, stopping reasons, and validity remain unchanged. Post-collection decisions and affected runs are documented in Appendix D.8. Both score versions require normal submission and passage of every applicable check.

Figure 7 separates changes in scoring conditions from changes in the agent runs themselves.

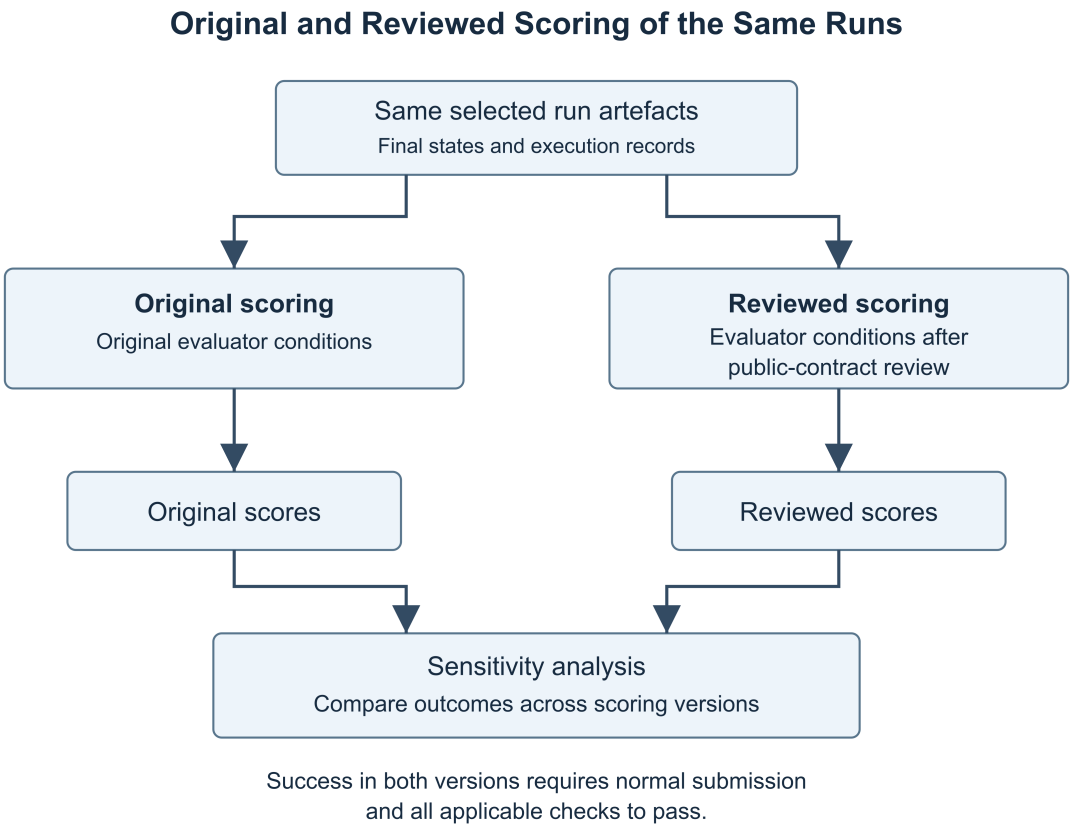


Figure 7: Original and reviewed scoring of the same selected run artefacts. Evaluator conditions are reviewed against public briefs, agent-visible repository contracts and authorised clarification responses. The same submissions, timing, stopping reasons and validity records are retained; no new agent runs are performed. Corrections apply uniformly to every system and repetition of each affected task. Both versions require normal submission and passage of every applicable check. Comparing the scores assesses sensitivity to scoring conditions.

Separating Execution Validity from Scoring Validity: Execution validity concerns whether the scheduled attempt received the promised task and operating conditions. Scoring validity concerns whether the observed final state is judged against the requirements that were actually available to the agent. An evaluation correction can therefore change the interpretation of an existing submission without creating a new run or changing its elapsed time. Conversely, a formally invalid attempt is not made valid merely because some checks happen to pass. Preserving these distinctions prevents a scoring review from becoming an opportunity to select a more favourable attempt.

The review treats implementation choices as requirements only when the public contract justifies that restriction. If the required behaviour can be obtained at more than one implementation layer, the assessment must examine the externally required behaviour and preserved invariants rather than require the layer used by the oracle. Equally, accepting an alternative implementation does not waive a stated constraint, such as permitted input values or preservation of existing records. This rule applies across tasks, systems, and repetitions. Task-specific evidence and score transitions are retained in the appendix, while the main analysis identifies the scoring version and reports sensitivity to the original scores.

The amendment history provides provenance rather than a claim that every condition remained

unchanged throughout collection. Each observation must be interpreted with the configuration bound to that run, and later manifests retain links to their predecessors. This is particularly relevant when product-controlled settings or infrastructure conditions require a recorded amendment. A final configuration identifier summarises the latest recorded state; it should not be read as evidence that every earlier attempt used that exact state. The same distinction applies to analysis: the frozen plan describes prespecified choices, while the implemented scripts and post-collection coding decisions identify how the completed data were processed.

4.5 Supplementary Qwen Interface Control

The response audit identified requests blocked by the fallback's single-line requirement. A supplementary online experiment tested whether removing this restriction enabled the configured Qwen system to complete repairs. Arm A retained the frozen v3 fallback; Arm B allowed physical line breaks within the JSON object. Tool names, argument schemas, single-call handling, residual-text checks and the other acceptance rules were unchanged. Both arms used the same model revision, vLLM deployment, chat template, task briefs, system prompt, tool definitions and sandbox. Each run began from a fresh task state and retained the original action, time and stopping limits.

The schedule was fixed before new model calls and covered all 12 tasks, with three repetitions per task in each arm: 36 runs per arm and 72 in total. These observations form a separate interface comparison; the 144-run main experiment retains its original trajectories and scores. Before the scheduled runs, guided tool checks verified actual file reading, patch application and command execution through both arms. The step-by-step guidance applied only to those checks. The benchmark prompts and interaction policy remained unchanged.

The primary endpoint was normal submission with every applicable reviewed evaluator check passed. Secondary measures recorded accepted tool calls, successful tool operations, non-empty final diffs and stopping reasons. A successful tool operation returned without a tool error; an accepted call had only passed parsing and dispatch. Results are reported descriptively by arm and task, with the three repetitions treated as repeated observations of the same task. Replaying each saved response through both parsers separately identifies acceptance differences for the same generated text. Appendix E records configuration details and attempt accounting.

4.6 Quantitative Evaluation Metrics

Multi-metric and scenario-based evaluation provide broader context than a single score [99, 8]. In this study, four complementary outcome families are reported separately. Binary run-level success is the primary outcome; no weighted aggregate or penalty score is used.

Run-Level Success and Diagnostic Correctness: For system s, task i, and repetition r, a valid run has $Y_{s,i,r} = 1$ only when the agent submits normally and the final system state passes every applicable hidden test and architectural invariant under the declared scoring version. A valid run stopped at an execution limit has $Y_{s,i,r} = 0$, even if its final state passes the checks; an invalidated attempt is excluded and replaced under the frozen rule. Each system has 36 scheduled main-experiment outcomes, so its final overall success rate is $\sum_{i=1}^{12} \sum_{r=1}^{3} Y_{s,i,r}/36$.

Two diagnostic proportions explain unsuccessful outcomes: C_{func} is the proportion of applicable frozen functional tests passed, and C_{arch} is the proportion of applicable architectural invariants satisfied. They are reported separately and are not combined. The original hidden tests are researcher-authored, frozen before the main experiment, and stored outside the agent-accessible repository. Analysis-layer amendments are versioned separately under the uniform review rule above.

The diagnostic proportions measure the state of the applicable checks, not the amount of repair work completed. Some checks already pass in a faulty starting repository, so a non-zero diagnostic proportion can occur without any successful modification. In addition, tasks can have different numbers and kinds of checks. A diagnostic proportion is therefore interpreted within the task contract and alongside the baseline failure description, final diff, and binary outcome. It is not used to assign a common distance-to-success scale across heterogeneous tasks.

Execution Performance: End-to-end elapsed time is recorded as $\Delta T = T_{end} - T_{start}$ for every run, together with the stopping reason and timeout indicators. Observable action counts, token usage, and API cost are supplementary where the product or provider exposes them. Unavailable commercial-IDE data are reported as unavailable rather than estimated.

The time summaries answer different operational questions depending on their denominator. All-valid-run time describes how long the configured system occupied the run before submission or a stopping condition, including early unsuccessful termination. Successful-run time describes completion only among runs that met the success criterion. That conditional sample may contain different tasks for different systems, so a shorter successful-run median alone does not establish greater efficiency on the same workload. The figures retain individual observations and identify the population being summarised to keep these differences visible.

Clarification-Seeking Behaviour: For each run, the evaluator records the number of clarification requests (N_{req}), authorised requests (N_{acc}), and pre-written responses issued (N_{resp}), together with whether the run contained a frozen clarification opportunity. The single opportunity task produces 12 opportunity runs overall and three per system; the other 11 tasks produce 132 complete-brief runs overall and 33 per system. Three descriptive measures are reported: clarification opportunity capture, defined as the proportion of opportunity runs with at least one authorised request; request precision, N_{acc}/N_{req} when $N_{req} > 0$; and the unnecessary-request rate, defined as the proportion of complete-brief runs containing any clarification request. The initial task prompt and routine IDE permission approvals are not counted.

Success Without Evaluator Clarification Responses (Prespecified Robustness): The prespecified robustness outcome is operationalised as success without an evaluator clarification response on the six Level 3 and Level 4 tasks. For system s,

$$R_s = \frac{S_{auto,s}}{18},$$

where $S_{auto,s}$ is the number of the 6 tasks $\times$ 3 runs = 18 applicable runs that submit normally and pass all tests and invariants without an evaluator clarification response. A valid run that reaches a task-level or API-wrapper limit is unsuccessful and does not count towards this outcome. This

prespecified use of robustness is narrower than a multidimensional reliability assessment [72]: it does not test resilience to input perturbations or tool faults, failure predictability, or error severity.

Qualitative Failure Analysis: Observable transcripts, tool calls, diffs, command outputs, and test results are reviewed to identify recurring failure trajectories, including premature completion, repeated ineffective commands, incorrect API or database assumptions, inability to trace cascading dependencies, unnecessary clarification requests, and failure to validate the final state. The analysis covers all 42 reviewed unsuccessful observations. Operational codes may overlap and require observable evidence; a failed final check alone does not establish that the agent skipped validation. Where native transcripts are unavailable, coding is restricted to the final diff and evaluator behaviour. Script-assisted extraction and evidence inspection were used without an independent second-coder assessment. The refined coding definitions are post-collection decisions, documented in Appendix D.9. The analysis does not infer private reasoning or assign subjective reasoning points.

Statistical Estimands and Descriptive Intervals: The analysis uses a tidy run-level table with one selected observation per scheduled task–system–repetition slot. The task is the unit retained for comparative resampling; the three repetitions are repeated attempts at that task, not three additional problem specifications. Define the task success proportion as $\bar{Y}_{s,i} = \sum_{r=1}^{3} Y_{s,i,r}/3$. The overall estimate is then $\hat{p}_s = \sum_{i=1}^{12} \bar{Y}_{s,i}/12$, which equals the run-level proportion because the design is balanced. For systems a and b, the reported effect is

$$\widehat{\Delta}_{ab} = \frac{1}{12} \sum_{i=1}^{12} \left(\bar{Y}_{a,i} - \bar{Y}_{b,i} \right).$$

Positive values favour system a; multiplying by 100 expresses the difference in percentage points. This effect compares absolute success probabilities on the selected workload and remains defined when a system has no successful runs. It is not a ratio of success rates or an estimate of model capability independent of the execution interface.

Overall and tier-specific plots show Wilson 95% intervals, with 36 and nine run outcomes per system respectively. These are descriptive binomial intervals, calculated using the standard normal 0.975 quantile. Their binomial reference model treats outcomes as independent trials with a common probability; the benchmark's task heterogeneity and repeated attempts do not establish that model. Consequently, the intervals are not presented as cluster-adjusted population uncertainty, and overlap of two plotted intervals is not the pairwise significance test. The task–system heat map retains the three-run denominators so that pooling does not hide which tasks contribute to an overall rate.

Task Bootstrap and Its Scope: For each pair, the workflow draws 10,000 samples of 12 task identifiers with replacement, retaining all systems and repetitions for every sampled task. It recalculates $\widehat{\Delta}_{ab}$ on each sample and uses the 2.5th and 97.5th percentiles, with linear quantile interpolation, as the unadjusted 95% interval. The same task draws are used across contrasts and scoring versions. Resampling whole clusters preserves their internal observations, a principle supported by the clustered-data bootstrap literature [100]. It avoids treating the 36 outcomes per system as 36 separately sampled tasks.

This implementation samples from the pooled set of 12 tasks, without constraining every bootstrap sample to contain three tasks from each tier. The resulting interval therefore reflects changes in the mix of observed tasks as well as their paired system differences. Its sampling interpretation requires task vectors to act as independent, exchangeable draws from a relevant task distribution. Keller et al. distinguish performance on fixed benchmark items from generalised accuracy over a population of potential items, which introduces additional modelling and sampling assumptions [101]. Here the tasks are deliberately constructed, share technologies and number only twelve; representativeness of a wider task population is not established. The intervals are therefore exploratory summaries of sensitivity to the observed task mix, not guaranteed population coverage. Repeating one task more often would not resolve the limited diversity of task specifications.

Exact Paired Comparisons and Multiplicity: The test uses integer task success-count differences $D_i = \sum_{r=1}^{3}(Y_{a,i,r} - Y_{b,i,r})$. It enumerates all $2^{12} = 4{,}096$ sign vectors $g \in \{-1, 1\}^{12}$ and calculates the two-sided probability

$$p_{ab} = \frac{1}{2^{12}} \sum_{g \in \{-1,1\}^{12}} \mathbf{1}\left\{ \left| \sum_i g_i D_i \right| \geq \left| \sum_i D_i \right| \right\}.$$

A sign change exchanges the two systems' complete task-level results. Ties and the observed assignment are included, and integer counts avoid numerical ambiguity at equality. Full enumeration removes Monte Carlo error from this calculation; it does not remove the assumptions needed to interpret the probability.

The relevant null requires invariance under these within-task system-label swaps, rather than merely equality of two overall means. Permutation-test validity depends on the transformation group and its null invariance [102]. Randomising execution order does not establish that invariance: all systems perform all tasks, and the study does not randomly assign model identities as interchangeable treatments. The paired test is accordingly interpreted as an exploratory comparison under the stated exchangeability assumption. A non-rejection is not evidence of equivalence, especially with few tasks and near-ceiling outcomes.

The six pairwise comparisons form one family for each scoring version. Holm's sequential adjustment controls family-wise error when the individual null probabilities are valid [103]. For ordered raw probabilities $p_{(1)} \leq \cdots \leq p_{(6)}$, the implementation reports $\widetilde{p}_{(j)} = \min\{1, \max_{k \leq j}[(7-k)p_{(k)}]\}$ and flags adjusted values below 0.05. Original and reviewed scores are adjusted separately; sensitivity results are not an independent replication because they reuse the same submissions. The bootstrap intervals remain unadjusted 95% intervals and are not simultaneous intervals corresponding to the Holm procedure. Effect sizes, intervals and exact adjusted probabilities are therefore reported with their respective interpretations.

Implementation and Reproducibility Boundary: Elapsed time is summarised with medians and interquartile ranges. Successful-run completion times and all-valid-run stopping times are shown separately; runs reaching T_{task} are flagged at the limit and are not treated as ordinary successful completions. Clarification opportunities are reported in a compact table with counts and explicit denominators, and functional and architectural diagnostic proportions are shown separately. The reproducible workflow produces the tables and figures from the selected run table while preserving both scoring versions. The analysis plan, data dictionary, exclusion rules and

random seed were frozen before collection; the implemented scripts and refined evidence-coding rules were versioned after collection and are not described as preregistered. The implementation used Python 3.12.14 rather than the planned 3.12.13, with NumPy 2.3.5 and the added plotting dependency Matplotlib 3.10.8. The frozen seed string is converted to an unsigned integer using the first eight bytes of its SHA-256 digest in big-endian order, then supplied to NumPy PCG64; this previously unspecified conversion is recorded with the scripts.

Appendix D.7 maps the selected dataset, score files, analysis scripts and failure-evidence index to their release-relative paths. Reproducing the numerical analysis and repeating agent execution are distinct goals. The former requires the selected dataset, scoring version, analysis scripts, dependencies and declared random-number procedure; it does not require another model call. The latter additionally depends on the run-bound prompts, starting states, execution configuration and provider availability. Frozen local files can be restored, but a commercial product's internal model routing, service latency or subsequently changed orchestration may not be recoverable. A repeat study must therefore identify its own execution dates and configurations rather than assume that a matching product name recreates the original system.

The record also bounds what can be reconstructed from missing observables. Unavailable native transcripts restrict failure interpretation to the evidence actually retained, and absent comparable usage or cost data cannot support a complete cross-system cost comparison. No missing token counts are inferred from elapsed time. Reproducibility here concerns the recorded inputs, selected outcomes and processing of those outcomes.

5 Results

The main analysis contains 144 selected valid observations: 12 tasks, four systems and three fresh repetitions per task–system combination. The provenance index retains 155 attempts, of which 11 were invalidated and excluded from performance denominators. All 144 valid observations, including unsuccessful runs, remain in the analysis; pilot runs are excluded. There were 109 normal submissions and 35 no-progress-limit stops. Unless stated otherwise, results use the uniform reviewed scoring version described in Appendix D.8.

The results address each research question at its relevant unit of analysis. RQ1 concerns where correctness varies across tiers and individual tasks, so the aggregate success estimates are followed by the task decomposition and diagnostic checks. RQ2 concerns clarification behaviour, for which request eligibility and opportunity capture are assessed separately from task completion. RQ3 concerns differences between the configured systems, using paired success contrasts and elapsed-time distributions. The failure evidence then identifies which observable stages and contract violations accompany unsuccessful outcomes, and the scoring comparison establishes which findings depend on the evaluator version.

5.1 End-to-End Success

Cursor succeeded in 35/36 observations (97.2%; descriptive 95% Wilson interval 85.8–99.5%), GPT-5.4 in 34/36 (94.4%; 81.9–98.5%), Devin Desktop in 33/36 (91.7%; 78.2–97.1%), and Qwen2.5-Coder-7B-Instruct in 0/36 (0%; 0–9.6%). Figure 8a reports these end-to-end outcomes. A submission marker alone does not establish success: seven normally submitted runs remained unsuccessful after final-state assessment.

The tier pattern is not a monotonic decline with architectural depth (Figure 9). Cursor, Devin Desktop and GPT-5.4 each passed all nine L1 and all nine L3 observations. Their respective L2 counts were 9/9, 7/9 and 8/9; each achieved 8/9 on L4. Qwen had no successful observation in any tier. The task decomposition (Figure 8b) locates the six failures among the three higher-success systems in L2-03, L4-02 and L4-03. With only three distinct tasks per tier, these observations describe this task sample rather than a causal effect of architectural complexity.

Task Differences Within and Across Tiers: The task heat map identifies a more specific pattern than the tier averages. At L2, the observed shortfall is confined to L2-03, which requires correct import/export semantics and persistence across a service restart. Devin Desktop completed one of its three repetitions and GPT-5.4 completed two, while Cursor completed all three. The other two L2 tasks did not differentiate these systems under binary success. Thus the lower L2 result for Devin Desktop and GPT-5.4 reflects a particular storage contract rather than a general failure across single-service backend tasks.

The identical L4 totals also conceal different task outcomes. Cursor and Devin Desktop each failed one repetition of L4-02, while GPT-5.4 passed all three repetitions of that task but failed one repetition of L4-03. All three systems passed every repetition of L4-01, the removed-column cascade. Equal tier-level rates therefore do not imply interchangeable repair profiles: the relevant distinction is between handling accepted customer-status inputs and preserving dashboard connectivity after a database-host repair. The final-state evidence for these cases is examined in Section 5.5.

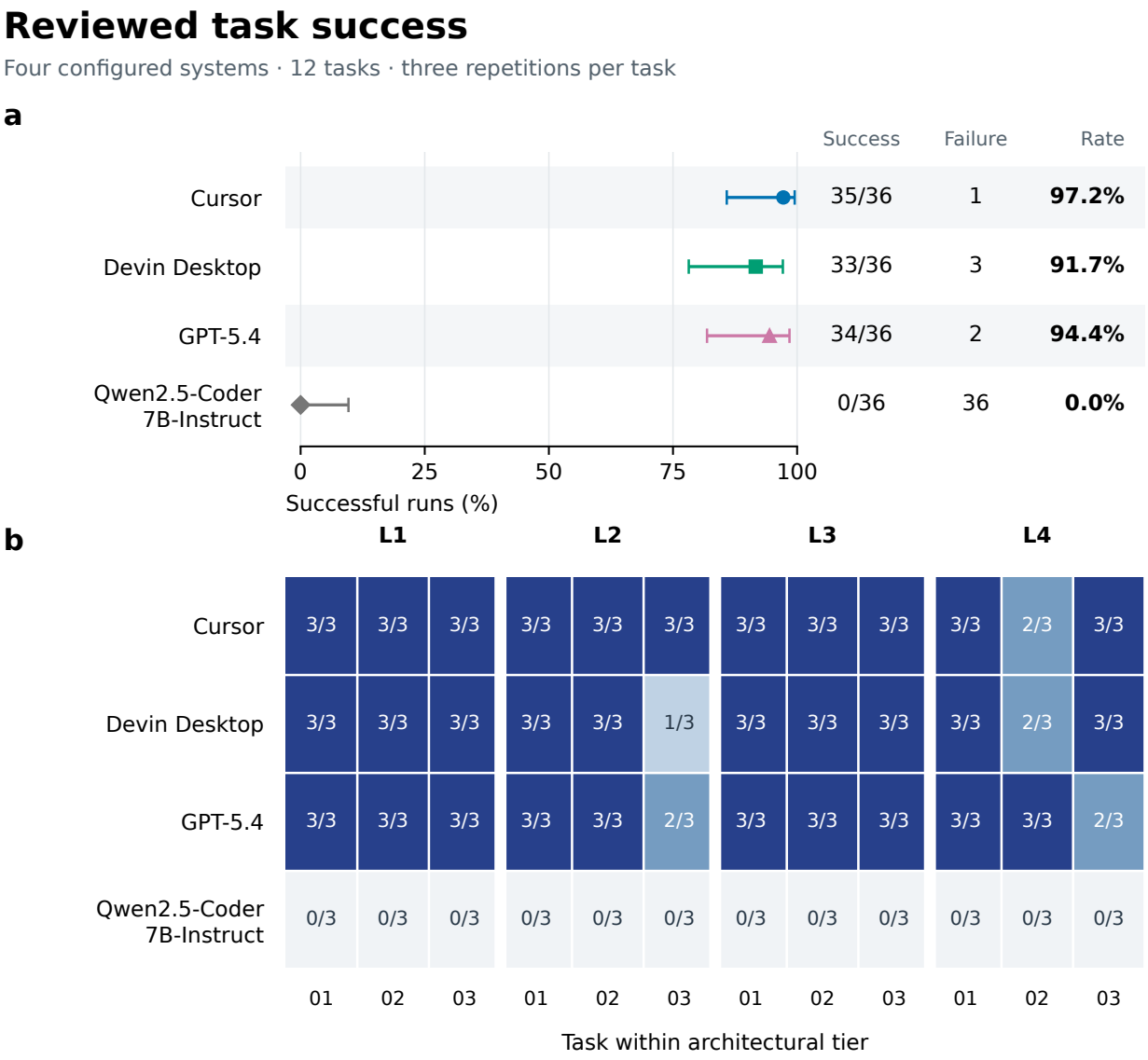


Figure 8: Reviewed success across four configured systems. (a) Points show successful observations divided by 36 runs per system (12 tasks with three repetitions); horizontal bars are descriptive 95% Wilson intervals. Columns report successful runs out of 36, unsuccessful-run counts and success rates. All valid failures remain in the denominator. These run-level intervals do not adjust for within-task dependence; comparative inference uses 12 paired tasks. (b) Cells show successful repetitions out of three for every task–system combination; darker blue denotes more successes on the fixed count scale 0–3. Tier headings and task suffixes together identify tasks (for example, L2 and 03 denote L2-03). Tasks follow tier order without clustering or normalisation. No cell-wise hypothesis tests are performed.

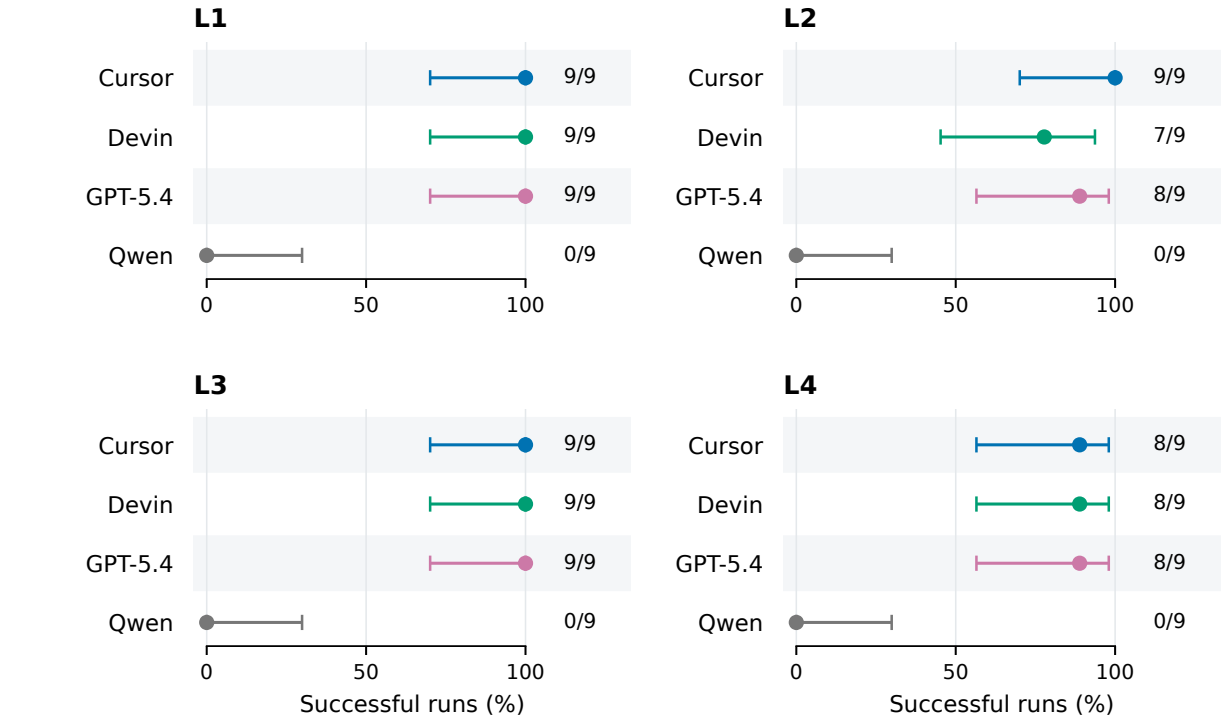


Figure 9: Reviewed success by tier. Each point uses nine observations from three tasks and three repetitions per system, with descriptive 95% Wilson intervals. Separate panels show L1–L4; rows identify systems and labels give successful counts out of nine. Each tier contains only three distinct tasks.

Mixed cells such as one or two successful repetitions out of three show that the selected task was solvable by that system but was not completed consistently across its fresh attempts. Qwen's uniformly unsuccessful cells contain no successful repair from which to assess differences in repair quality across tiers; its recorded progress before editing supplies the more informative decomposition below.

For RQ1, the evidence therefore supports a task-specific account of the observed variation. The three higher-success systems completed the selected frontend–backend integration tasks at L3 even though some lower-tier persistence attempts failed. Architecture, fault family and task contract vary together, and the observed failures are concentrated in a small subset of that combined design.

5.2 Paired System Comparisons

The six paired comparisons report effect sizes and their uncertainty (Figure 10). Differences among Cursor, Devin Desktop and GPT-5.4 ranged from -2.8 to 5.6 percentage points in the specified contrast directions. None was statistically significant under the exact task-label-swap tests after Holm adjustment (all adjusted $p = 1.000$). This does not establish equivalence. Each system exceeded the configured Qwen system by 91.7–97.2 percentage points, with adjusted $p = 0.0029296875$ for each comparison. These estimates compare the complete system configurations, including their interfaces.

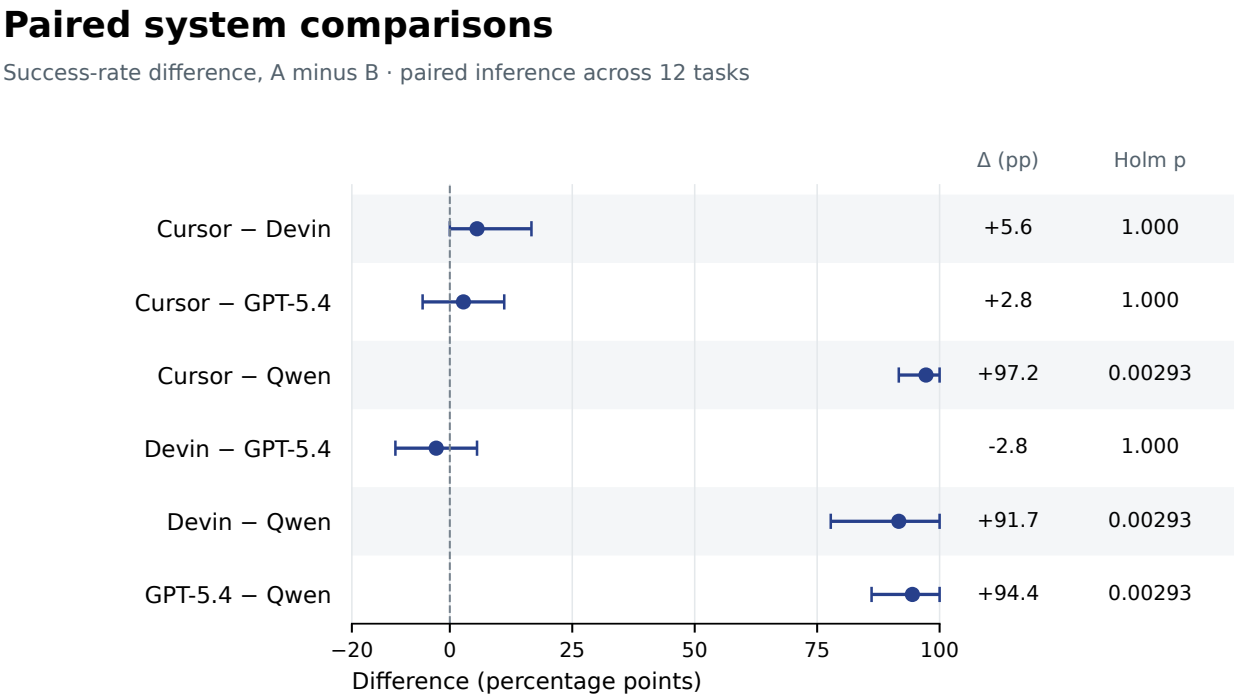


Figure 10: Reviewed success-rate differences, system A minus B, in percentage points. Bars are unadjusted two-sided 95% percentile intervals from 10,000 task-bootstrap resamples of 12 paired tasks, retaining all systems and repetitions. The dashed line marks zero; the right-hand columns report the difference in percentage points and the Holm-adjusted p-value. Exact two-sided tests enumerate 4,096 task-label assignments; Holm correction covers six contrasts. Bootstrap intervals are not simultaneous intervals or inversions of the adjusted tests.

The task pairing explains why the aggregate ordering should not be read as a general ranking. Cursor and Devin Desktop have equal successful-repetition counts on every task except L2-03; their overall difference is supplied entirely by that task. Cursor and GPT-5.4 differ on three tasks: Cursor has more successes on L2-03 and L4-03, whereas GPT-5.4 has more on L4-02. These offsets are visible in the task heat map and are retained when complete tasks are resampled. The small pooled differences therefore arise from sparse, task-dependent disagreements rather than a consistent separation on every task.

The comparison with Qwen has a different structure. Each of the other systems achieved at least one success on every task, while Qwen achieved none. Its separation is consequently present across the task set rather than being attributable to one particularly difficult case. For RQ3, this supports a broad within-benchmark difference in completed repairs under the recorded configurations. The failure records below show how far runs progressed.

5.3 Elapsed Time and Diagnostic Correctness

Successful-run completion time and time to termination across all valid runs answer different questions. For reviewed successful runs, median time was 102.27 seconds for Cursor (Q1–Q3: 61.61–191.34; $n = 35$), 209.00 for Devin Desktop (108.36–251.38; $n = 33$), and 27.01 for GPT-5.4 (20.66–34.75; $n = 34$). Qwen has no successful-run duration estimate. Across all 36 valid observations per system, medians were 96.70 seconds for Cursor (65.46–190.95), 218.70 for Devin Desktop (108.96–260.54), 27.72 for GPT-5.4 (21.03–34.98), and 19.82 for Qwen (15.19–36.96).

Qwen's short stopping times do not indicate faster successful repair. Figure 11 retains unsuccessful outcomes and distinguishes their stopping times from completion times. Successful-only populations also differ across systems, so their medians are conditional descriptions rather than matched causal speed comparisons. Recorded time includes the configured interface and execution process.

GPT-5.4 had the shortest median completion time among the three systems with successful runs. The success rates and conditional completion times jointly describe their observed repair performance for RQ3.

Functional and architectural checks provide complementary diagnostic information (Figure 12). For Cursor, Devin Desktop and GPT-5.4, both overall run-level diagnostic proportions have median 100% and Q1–Q3 of 100–100%. These ceiling summaries do not mean that every run passed: a median across repetitions can conceal an individual failure. Qwen's functional median was 12.5% (0–37.5%), while its architectural median was 66.7% (45.8–80.0%). Some checks already pass in a faulty starting repository, so partial diagnostic scores do not by themselves demonstrate agent repair. Each run contributes equally; check counts are not pooled across tasks or combined into a weighted score.

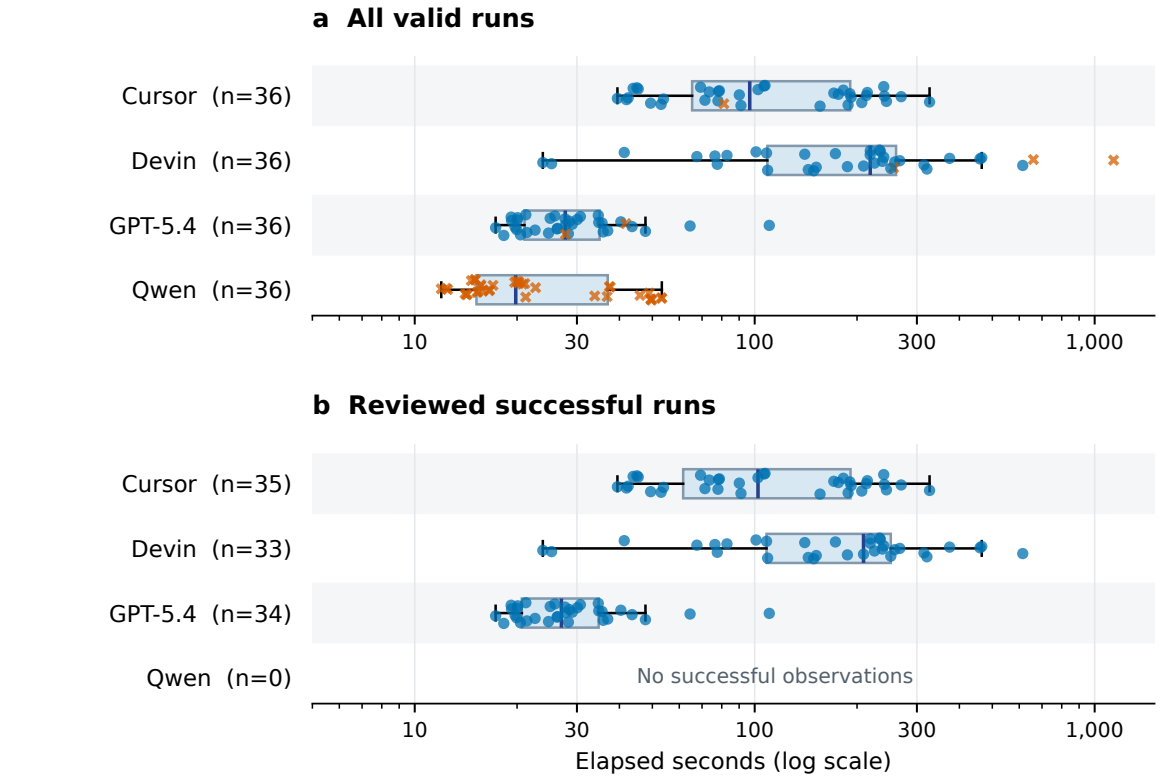


Figure 11: Elapsed seconds for all valid runs (a) and reviewed successful runs (b). Boxes span Q1–Q3 with a median line; whiskers reach observations within 1.5 IQR of the quartiles, calculated in seconds before logarithmic display. Every observation, including outliers, is plotted with deterministic vertical offsets. Blue circles show success and orange crosses unsuccessful outcomes. Sample sizes appear beside each system; Qwen has no successful observations.

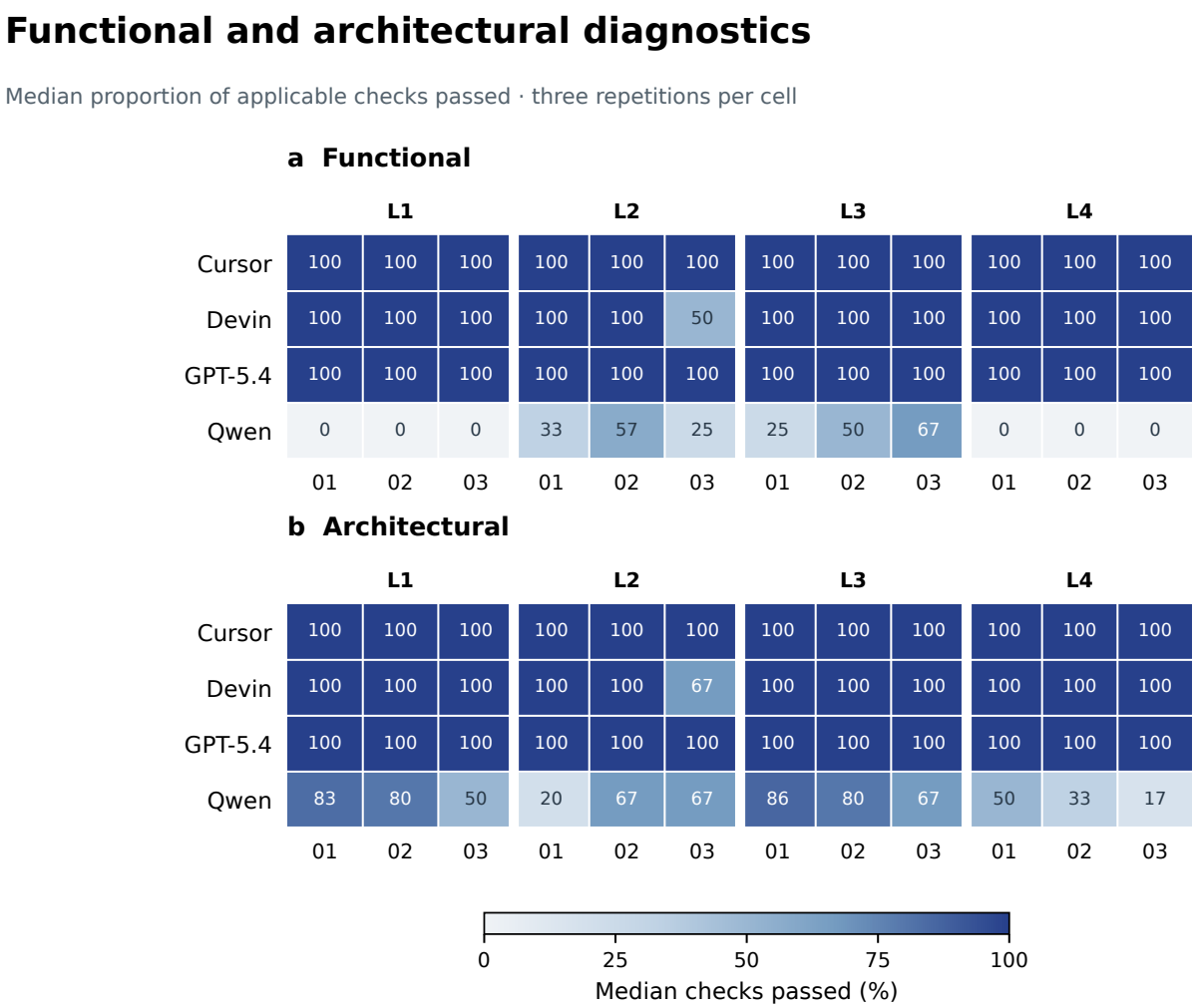


Figure 12: Reviewed median proportion of applicable checks passed across three repetitions per task–system combination, separately for functionality (a) and architecture (b). Cells show whole-number percentages; colour uses the unrounded median on a shared 0–100% scale. Rows identify systems; tier headings and task suffixes identify tasks (for example, L2 and 03 denote L2-03). Tasks are not clustered and denominators are not pooled. Read this figure alongside task-level binary success because medians can conceal failed repetitions.

What the Check Counts Reveal: The run-level counts clarify why diagnostic correctness is reported separately from binary success. In the two unsuccessful L4-02 submissions, both functional checks passed, but only five of six architectural checks passed. The remaining accepted-input violation was sufficient to make each submitted run unsuccessful. For the unsuccessful GPT-5.4 L4-03 run, the corresponding counts were two of three functional checks and five of six architectural checks. A largely passing final state therefore still contained a failure relevant to the required application contract.

The L2-03 failures affected both diagnostic families: the three unsuccessful submissions passed either one or two of four functional checks and four of six architectural checks. This contrasts with the L4-02 cases, where the functional set alone would have missed the remaining violation. These examples answer different parts of RQ1: the task success map locates inconsistent completion, while the separate check counts identify which requirements remain unsatisfied in those attempts.

Aggregation accounts for an apparent discrepancy between the success and diagnostic heat maps. For GPT-5.4 on L2-03, two fully passing repetitions accompany one failure, so the median functional and architectural proportions are both 100%, although success is two out of three. For Devin Desktop on the same task, two repetitions fail, and the displayed medians fall to 50% functional and approximately 67% architectural. A diagnostic median is thus useful for the typical recorded final state but must be read with the successful-repetition count. The underlying run-level numerators and denominators remain necessary for inspecting exceptional failures.

5.4 Clarification and Success Without Evaluator Responses

GPT-5.4 made three authorised requests and received three responses, capturing all three deliberate clarification opportunities. Cursor and Devin Desktop made no requests. Qwen made eight recorded requests, none authorised, in two complete-brief runs. Table 5 retains the counts and denominators: precision is undefined when no requests are made, rather than zero. All deliberate opportunities come from the same L2-02 task, so three captured repetitions are not evidence of broad clarification competence across different requirements.

Table 5: Recorded clarification behaviour. Capture is opportunity runs with an authorised request divided by three; precision is authorised requests divided by requests; unnecessary-request rate is complete-brief runs with any request divided by 33. Permission prompts are excluded. NA denotes no requests.

System	Requests/authorised/responses	Capture	Precision	Unnecessary
Cursor	0/0/0	0/3	NA	0/33
Devin Desktop	0/0/0	0/3	NA	0/33
GPT-5.4	3/3/3	3/3	3/3	0/33
Qwen	8/0/0	0/3	0/8	2/33

Completion and clarification capture separate most clearly on L2-02. Cursor, Devin Desktop and GPT-5.4 all succeeded in its three repetitions, but only GPT-5.4 issued authorised requests. Successful completion alone therefore does not establish that a system recognised and explicitly resolved the deliberately omitted requirement. Conversely, the recorded difference in asking behaviour did not produce a difference in binary success on this particular task. There is no within-system comparison that would identify the causal benefit of the response.

Qwen's eight requests also illustrate why request-level precision and run-level unnecessary-request rate use different denominators. They arose in two complete-brief runs, not eight separate failed opportunities. The observed issue was repeated requesting in contexts without an authorised information gap; the opportunity task itself received no request. For RQ2, the dataset therefore distinguishes targeted capture, absence of requests and requests outside the permitted condition.

On L3/L4, Cursor, Devin Desktop and GPT-5.4 each succeeded without an evaluator clarification response in 17/18 observations (94.4%); Qwen did so in 0/18 (Figure 13). All 72 eligible observations received zero clarification responses. Consequently, this prespecified measure coincides with L3/L4 success here and supplies no independent estimate of recovery ability or the causal effect of withholding help.

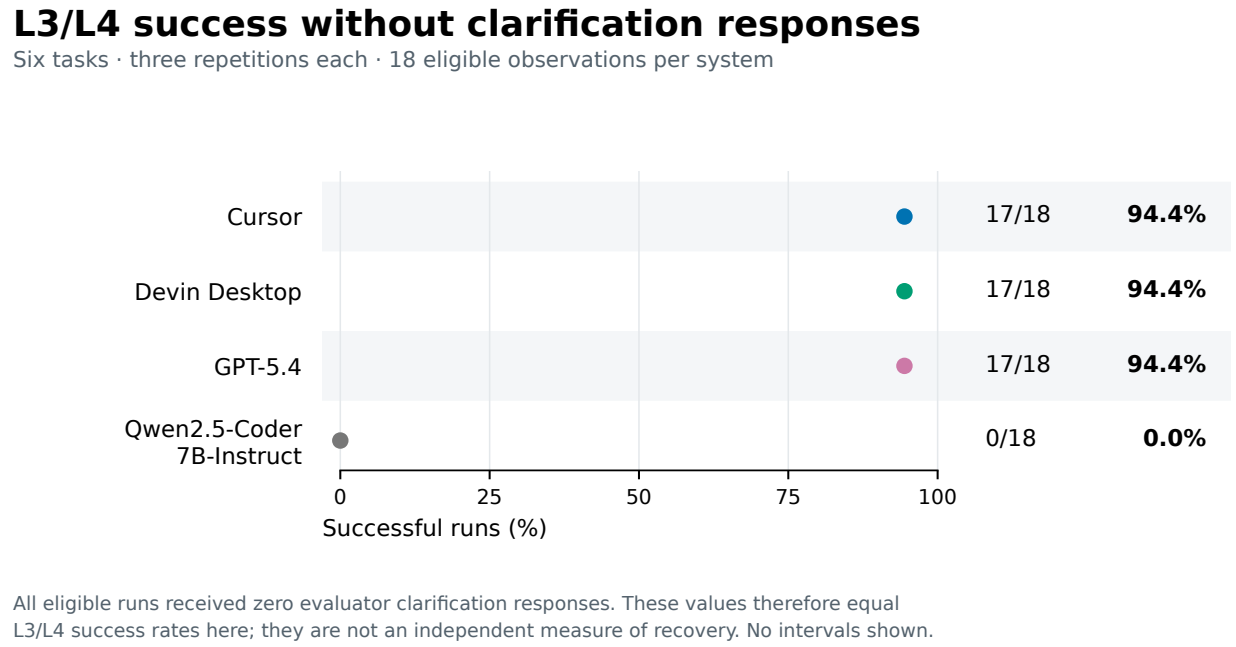


Figure 13: Reviewed normal-submission success with all applicable checks passed and zero evaluator clarification responses, among 18 observations per system (six tasks with three repetitions). Points are descriptive rates; labels give counts. No inferential intervals are shown. Because every eligible run received zero responses, these values equal L3/L4 success rates in this dataset.

5.5 Observable Failure Trajectories

Evidence review covered all 42 reviewed unsuccessful observations: one Cursor, three Devin Desktop, two GPT-5.4 and 36 Qwen runs. Eighteen Qwen runs had no accepted tool action; another 18 recorded inspection or clarification actions but no task-code diff. Fifteen of its runs contain an actual missing-path error, whereas other traces contain proposed commands or file requests without accepted execution. Thus all 36 Qwen final diffs are empty, but not every failure is an observed path-resolution failure. Thirty-five stopped at the no-progress limit; L3-02 repetition 3 emitted the submission marker without an accepted action and left the status display unfixed.

A post-hoc audit of all 36 Qwen runs examined 139 raw model responses. Each recorded a successful transport attempt, with consistent model identifiers and vLLM fingerprints, but none

contained a native structured tool call. The v3 fallback produced all 28 accepted requests: 17 file reads, three shell commands and eight clarification requests. Each had a corresponding tool-result record. All file reads requested paths absent from both the preserved fixture and run workspace; the three shell searches returned exit code 1; and the clarification requests received no answer because they were unauthorised. No `patch_file` request was accepted. These records show that the request-processing path operated, while also showing why acceptance alone did not establish useful task progress.

Replaying the frozen normalizer reproduced the saved acceptance counts and rejection reasons for all 139 responses, with no mismatches. A separate formatting control removed only whitespace outside JSON strings in the first lowercase `json` fence, preserving tool names, arguments and string contents. Nine previously rejected responses, distributed across seven runs, then yielded an accepted call: all three L1-02 repetitions, L3-01 repetition 3 and all three L3-03 repetitions. This identifies a direct effect of formatting on acceptance in those responses. This response-level control motivated the online comparison in Section 5.6. The main-experiment failure categories describe the original executed trajectories.

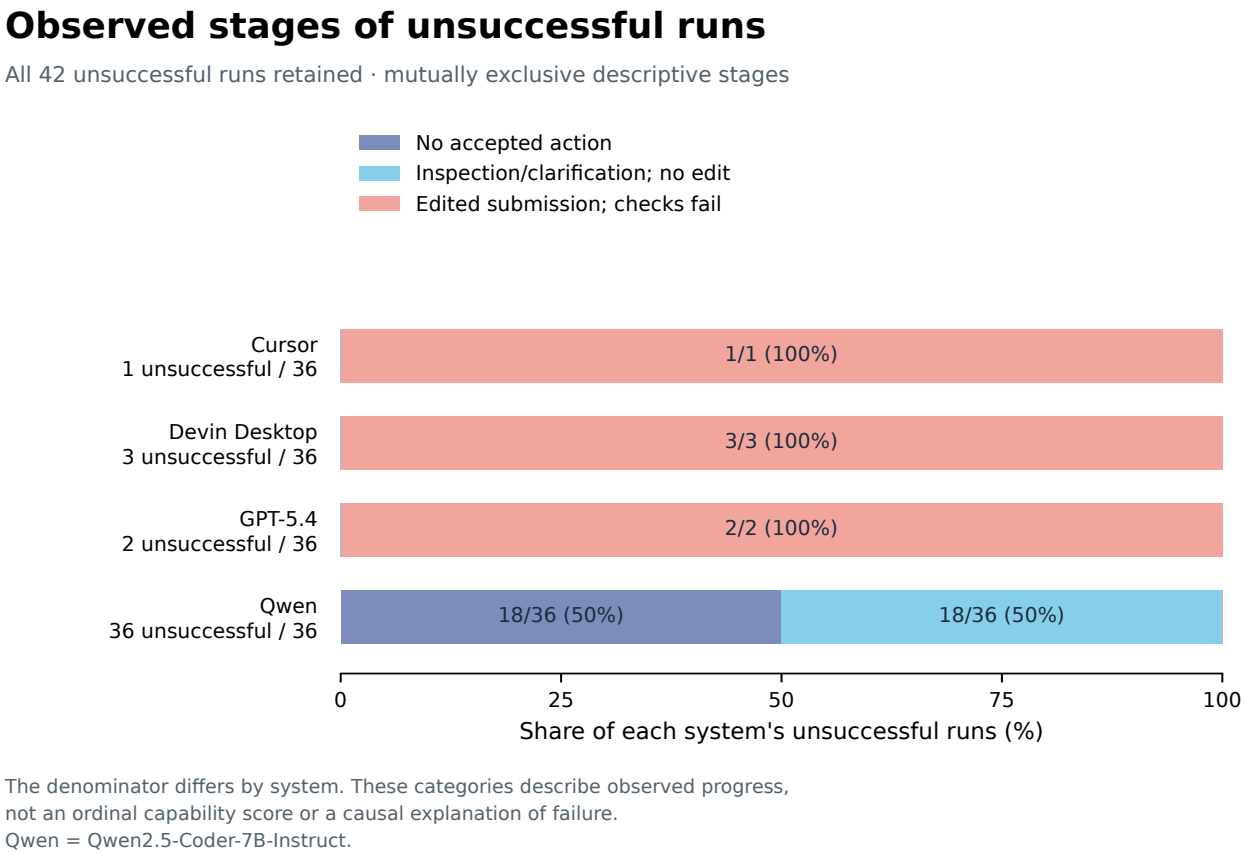


Figure 14: Mutually exclusive observed stages among all 42 reviewed unsuccessful runs. Each bar uses that system's unsuccessful runs as its denominator; labels report counts and percentages, and the total failed out of 36 runs is shown beside each system. Accepted inspection or clarification actions without a recorded code change form the middle category. Native final diffs and stopping records establish edited submission, even where the interaction trajectory is unavailable. These are descriptive terminal-progress categories, not an ordinal capability score; no inferential intervals or tests are applied.

Figure 14 separates failure before an effective edit from failure of an edited submission. The equal-length bars compare the composition of each system's failures, not their overall failure rates. In particular, a full bar for Cursor represents one unsuccessful observation, whereas the Qwen bar represents 36. The stage distinction therefore complements the overall success comparison rather than supplying another system ranking.

The six remaining failures involved edited submissions. Three concern L2-03 storage behaviour: a working-directory-dependent path, a Dockerfile-only environment change that did not pass application-level persistence checks, and a change from replacement import to merging. Two L4-02 submissions violate the accepted-input contract, respectively accepting explicit null and ignoring supplied status values. In the GPT-5.4 L4-03 failure, the database hostname was repaired, but additional frontend/proxy changes produced a browser HTTP 404. The recorded validation commands reported missing pytest and npm executables before submission. These records show that the attempted validation did not establish end-to-end correctness.

Figure 15 summarises the evidence-supported categories. Categories can overlap. Four unsuccessful native-IDE runs lack a usable interaction transcript; their codes describe final diffs and observed test behaviour, not an inferred sequence of actions. Coding was assisted by scripted extraction and inspection of retained evidence, without an independent second-coder assessment. The operational definitions and illustrative evidence are retained in Appendix D.9.

The category map adds detail to the stage distinction. The three documented API/database-contract violations concern edited native submissions, whereas Qwen's largest category records no-progress termination. The broad other-supported category is heterogeneous and should not be interpreted as a single common cause. Likewise, the linked-service failure and unsuccessful validation labels overlap on one GPT-5.4 run. Summing category counts would therefore overstate the number of distinct failed observations. Differences in failure denominators and trace availability also preclude treating colour intensity as a comparable propensity to exhibit each behaviour.

Taken together, the two failure figures distinguish an operational boundary from a final-state correctness boundary. The configured Qwen runs did not provide edited solutions whose programming correctness could be compared with the edited failures of the other systems. For the latter, a submitted change was observable, but the retained contract or linked-service behaviour remained unsatisfied. These are different routes to the same binary outcome and explain why failure counts alone provide an incomplete account of RQ3.

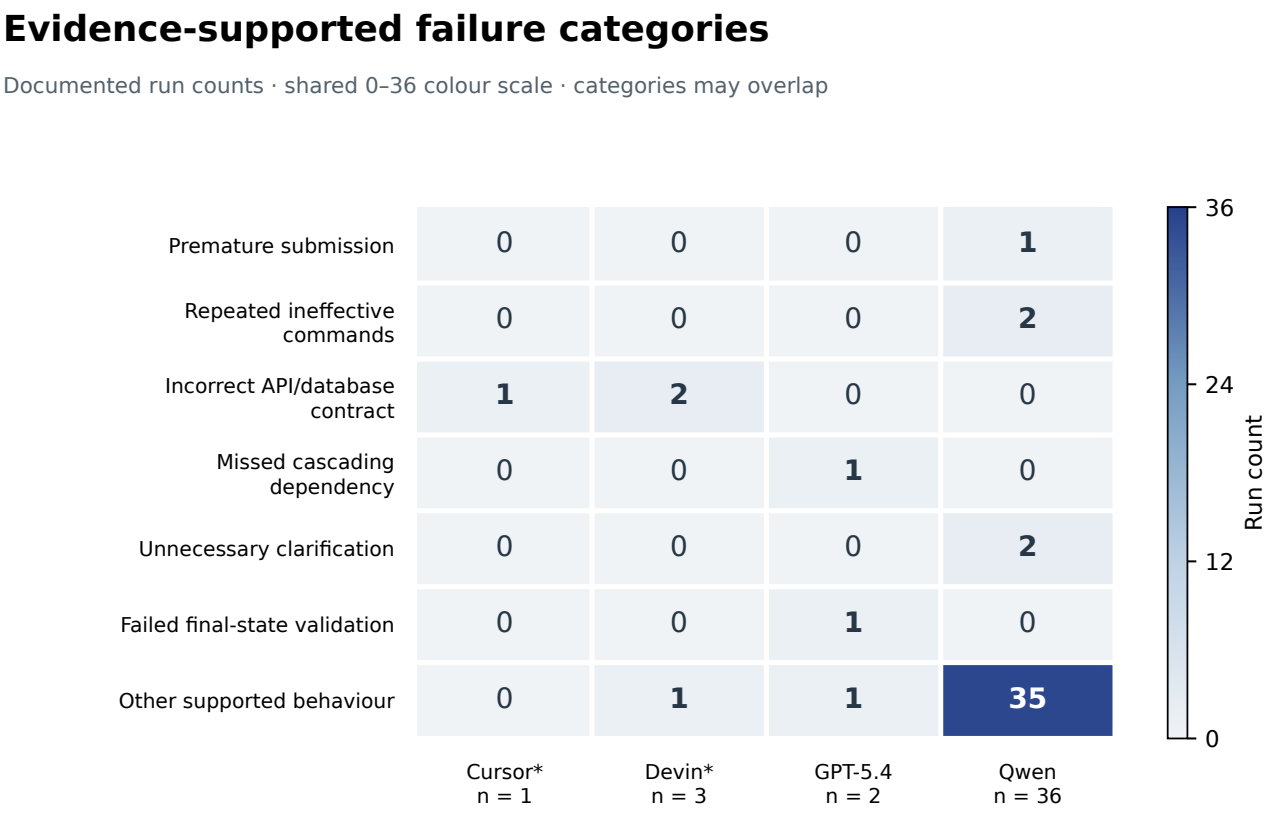


Figure 15: Documented category counts for the 42 reviewed unsuccessful runs, with per-system failure denominators shown beneath the columns. Colour uses a common linear count scale; categories may overlap, and no clustering or rate normalisation is applied. Zero means no assigned evidence-supported code, not demonstrated absence of a behaviour. Asterisks identify the four native-IDE failures whose interaction trajectories are unavailable. Other supported behaviour comprises 35 Qwen no-progress/protocol terminations and two storage/configuration failures. Counts are descriptive; no inferential comparisons are made.

5.6 Supplementary Qwen Interface Control

Neither interface arm produced a successful repair across its 36 runs (Table 6). All 72 final diffs were empty, and every run stopped after reaching the limit for consecutive responses without an accepted action. Arm A accepted 29 tool calls and Arm B accepted 52. Three operations succeeded in Arm B, all during repetition 3 of L2-02: listing the workspace, listing its `app` directory and reading `app/main.py`. No patch was accepted in either arm.

Table 6: Supplementary Qwen interface results. Each arm contains three fresh runs on each of 12 tasks. Counts of calls and operations are totals across runs; task success uses the reviewed end-to-end criterion.

Measure	A: frozen fallback	B: multiline JSON
Valid runs	36	36
Successful repairs	0/36	0/36
Accepted tool calls	29	52
File reads	18	25
Shell commands	11	27
Successful tool operations	0	3
Non-empty final diffs	0	0
Model responses	155	197

Replay of all 352 saved responses found 13 for which A rejected the request and B accepted one file-read call. Seven came from actual A trajectories and six from actual B trajectories. The six B requests occurred once per repetition on L1-02 and L3-03; all named nonexistent paths and failed when executed. The seven A responses describe additional requests that the alternative parser would accept, without a corresponding executed continuation. Across the remaining responses, both parsers produced the same normalisation result. The aggregate difference of 29 versus 52 accepted calls therefore also reflects differences in generated trajectories, including repeated failed commands, and cannot be assigned wholly to newline handling.

Of the 81 accepted calls, 78 returned errors: 42 file-read errors, 30 commands with exit code 127 and six with exit code 1. Command failures included attempts to use `docker-compose` and `pytest`, which were unavailable in the fixed sandbox. Together with the empty diffs, these records show that accepting a request was often followed by another obstacle before an effective edit. Removing the newline restriction alone yielded no successful repair under the tested configuration.

5.7 Sensitivity to Scoring Review

Uniform review changed ten observations from unsuccessful to successful, with no transition in the opposite direction. Cursor changed from 30/36 to 35/36, Devin Desktop from 31/36 to 33/36, GPT-5.4 from 31/36 to 34/36, and Qwen remained at 0/36 (Figure 16). Seventeen observations have changed diagnostic check counts or denominators; a diagnostic change need not change binary success. All comparisons use the same 144 selected observations.

The descriptive ordering among the three higher-success systems changes, but their pairwise adjusted p-values remain 1.000 under both versions. Each comparison with Qwen remains detected:

original Holm-adjusted p values were 0.0078125 for Cursor, 0.0048828125 for Devin Desktop and 0.0029296875 for GPT-5.4, compared with 0.0029296875 for each reviewed comparison. The broad separation from Qwen is therefore insensitive to these corrections, while the point-estimate ordering among the other systems is sensitive.

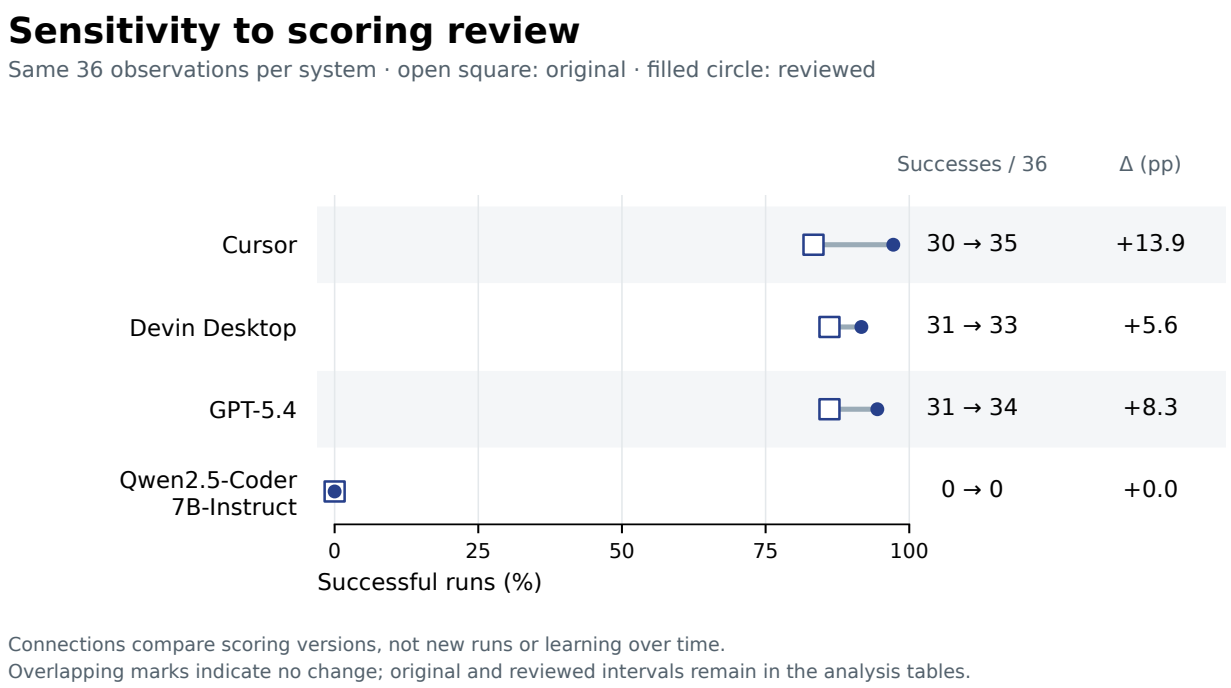


Figure 16: Original (open square) and reviewed (blue circle) success proportions for the same 36 observations per system. Lines connect scoring versions; labels give original → reviewed successful counts and the change in percentage points. Coincident marks denote no change. Original and reviewed Wilson intervals and paired effect intervals are retained in the accompanying analysis tables.

The reviewed results are used for the substantive account of task completion; original scores retain the effect of the initial evaluator on credited completions and their descriptive ordering.

The difference between seventeen diagnostic changes and ten binary transitions is also informative. A change to one check or its applicability need not resolve every remaining reason for failure, and the normal-submission condition still applies. Sensitivity at the check level and sensitivity at the success level therefore measure different consequences of the same review. This connects the diagnostic analysis back to the primary endpoint without treating partially improved scores as newly successful runs.

All-valid elapsed times and recorded clarification behaviour are unchanged. Successful-only time summaries change through membership rather than altered execution speed. Qwen's architectural median changes from 61.9% to 66.7%, while its functional median remains 12.5%. Original L3/L4 no-response success counts were 12/18 for Cursor and 15/18 for both Devin Desktop and GPT-5.4, compared with 17/18 for each after review; Qwen remains at 0/18. Original scores and task-specific amendment evidence are preserved in Appendix D.8 and the versioned analysis tables.

6 Discussion

The TSB results distinguish whether a configured system completes a repair from how far an unsuccessful run progresses and which requirements its final state satisfies. Read together, these measures support a more specific account of agent performance than an aggregate success ordering alone. This discussion interprets that account in relation to prior work and examines study-specific validity threats [104].

6.1 Interpretation of the Main Findings

The reviewed outcomes show high end-to-end success for Cursor, Devin Desktop and GPT-5.4 on this compact benchmark, while the configured Qwen system completed no successful repair. The small differences among the three higher-success systems were not distinguished by the paired task-level tests; their descriptive ordering also changes under original scoring. The tier comparison covers three distinct tasks per tier, with fault families varying alongside architecture.

Qwen's main-experiment runs ended with 36 empty final diffs. The audit in Section 5.5 identified formatting-sensitive rejection alongside requests to nonexistent paths and unauthorised clarification requests. The supplementary online comparison then removed the newline restriction in one arm, while retaining the other interface and execution conditions. Both arms again produced no successful repair, although the relaxed parser admitted some requests that the frozen parser rejected. The execution records locate the remaining obstacles before an effective edit.

Answers to the Research Questions. RQ1 is answered by the tier and task results in Section 5.1: the observed success pattern does not decline monotonically with architectural depth, within three tasks per tier. RQ2 is answered by Table 5: GPT-5.4 captured all three opportunities on the single incomplete task, while Qwen made unauthorised requests on two complete-brief runs. RQ3 is answered by the paired comparisons, timing and no-response results: the three higher-success systems are separated from configured Qwen but not from one another by the adjusted tests; successful-run timing is conditional, and the no-response measure duplicates L3/L4 success here.

Figure 17 brings together three complementary perspectives used in the following discussion: interface execution, task obligations and scoring conditions.

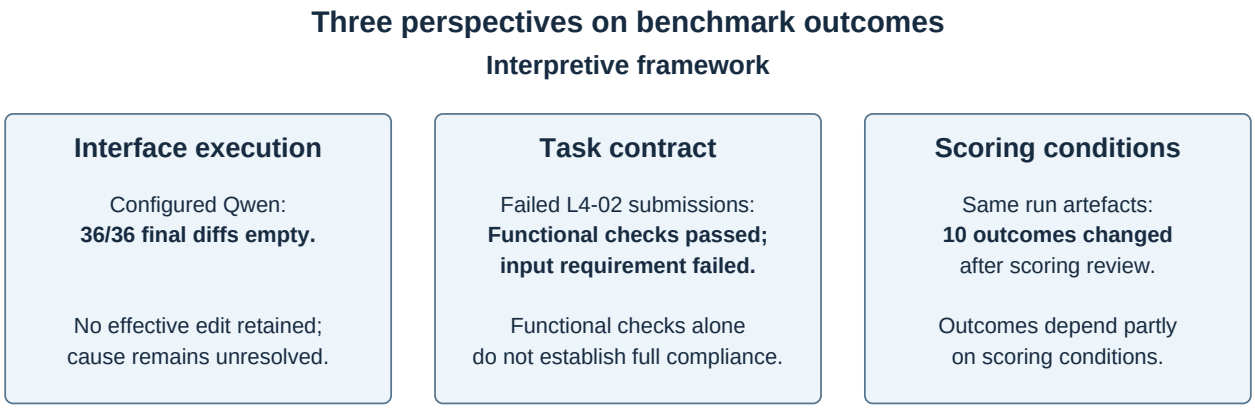


Figure 17: Three complementary perspectives on TSB outcomes. Success is interpreted alongside execution evidence, task obligations and the scoring version. The examples summarise empty final diffs in all 36 configured Qwen runs, functional-check-passing but contract-violating L4-02 submissions, and ten binary outcome changes after review of the same run artefacts. These perspectives organise the recorded evidence rather than establish a causal chain. Run-level success requires normal submission and passage of all applicable assessed checks.

6.2 End-to-End Performance and the Role of the Interface

The contrast between pre-edit failures and edited submissions changes what can be inferred from a common unsuccessful outcome. An edited submission supplies a candidate repair whose remaining defects can be examined. An empty final diff supplies no completed modification on which to make the same judgement. Qwen's results locate its observed failure before a lasting code change, directing attention to how a generated request becomes an operation in the task workspace.

Interface design is a relevant comparison point in the literature. SWE-agent investigates agent–computer interfaces for repository navigation, file editing and program execution [5]. TSB's supplementary control tests one part of that interaction: accepting JSON objects across multiple lines. Response replay isolates a direct acceptance effect in 13 of 352 responses. In the actual relaxed-arm trajectories, the six affected requests all targeted nonexistent files. The intervention therefore removed a formatting obstacle for those requests without resolving file selection. The unchanged repair outcome and the changed acceptance behaviour describe different stages of the same workflow.

The guided preflight adds evidence about the execution path. Both arms could read a real file, apply a patch and execute a command when the model received explicit step-by-step guidance. This verifies that the configured route could perform those operations. During the benchmark, the model had to choose and format its own actions from the original task brief. The gap between guided operation and autonomous repair makes action selection and use of returned feedback relevant to interpreting the result. CodeAct similarly examines how action representation and execution shape agent interaction [36].

The benchmark traces expose concrete obstacles after parsing. File-read errors followed requests to absent paths; command errors included unavailable `docker-compose` and `pytest` executables. These failures concern the model's chosen operation in the supplied environment. A more permissive parser alone does not supply the missing executable or choose a valid repository

path. The 29 versus 52 accepted-call totals also include different generated trajectories and repeated failed commands. Comparing these totals as if each extra call were caused by newline handling would obscure the narrower effect established by replay. With no successful repairs in either arm, the experiment shows that this single interface change was insufficient on the tested tasks. Native tool calling and broader changes to action handling remain separate comparisons.

The comparison with Agentless sharpens the distinction between choosing an action and completing a repair. Agentless uses a predefined localisation, repair and validation workflow [4]; API-Bank evaluates progressively demanding combinations of API calling, retrieval and planning [25]. TSB's supplementary records show why these stages need separate observations: the relaxed arm reached successful directory listings and a file read in one run, yet produced no edit. OSWorld also evaluates completed outcomes within configured computer environments [48]. Across these settings, execution evidence helps explain how far a system progressed before its final assessment.

Feedback availability further qualifies what action counts can explain. InterCode treats execution observations as part of the interaction loop [11], whereas TSB separates agent-visible feedback from evaluator-only assessment. The visible-test and held-out-test distinction in Self-Debugging provides a concrete comparison [56]: an executed check can inform a repair without establishing that it will satisfy final assessment. In TSB, observing that an agent ran a check therefore matters most when related to what the check exposed and what the agent did next. One interface may also combine operations that another exposes separately. Accepted API actions and native permission events describe progress within their respective routes, so dividing success by these counts would not yield a common measure of reasoning efficiency.

6.3 Task Contracts and Architectural Boundaries

The task-level results qualify the expectation that architectural depth alone determines repair difficulty. A shallow application can still impose a demanding behavioural contract, such as preserving imported data across a restart, while a multi-service fault can have a correction that the evaluated system applies consistently. In this dataset, selected persistence failures at L2 coexist with complete success by the three higher-success systems on the L3 tasks. Tier membership alone is an incomplete description of the tested demand: fault location, required behaviour and the scope of validation also vary across tasks.

The edited failures further distinguish correcting the visible symptom from satisfying the whole task contract. The L4-03 example contains a repaired database hostname alongside an unresolved browser-facing failure after additional changes. The unsuccessful L4-02 submissions satisfy the functional check set but violate an accepted-input requirement captured by the architectural checks. These cases support examining the final application across its affected boundaries rather than crediting a locally plausible edit in isolation. They do not establish that the agents followed a particular reasoning strategy, especially where the native interaction transcript is unavailable.

Integration-fault research helps locate these obligations more precisely than architectural depth alone. Zhou et al. examine faults and debugging evidence in microservice systems, while Gregor et al. organise integration-relevant faults at service boundaries [63, 65]. Read against that literature, the TSB examples concern relationships that must remain valid across a repair: accepted inputs must retain their specified meaning, stored data must survive the required lifecycle, and one service's output must remain usable by another. These relationships provide a basis for interpreting why a local correction may be insufficient. RESTTESTGEN's distinction between nominal and

error scenarios provides a testing counterpart [82]: the accepted-input obligation concerns which inputs should remain valid as well as which should be rejected. For the L4-02 interpretation, this makes the input contract relevant even when the ordinary functional checks pass.

This distinction also explains the value and limits of partial check scores. A result such as five of six architectural checks passed identifies a remaining obligation, but it is not a calibrated measure that the repair is five-sixths complete. Checks differ in what they cover, and one failed requirement can prevent the intended workflow even when other services remain healthy. Conversely, a passing invariant may simply describe a component that was never broken. The diagnostic proportions are therefore most useful alongside the task's public contract and final diff. Keeping them separate from binary success preserves information about the final state without treating counts as interchangeable units of engineering value.

The contract also determines which implementation differences should affect the score. Two repairs can use different internal mechanisms while satisfying the same required behaviour; an implementation choice becomes disqualifying only when it violates an explicit behavioural or architectural obligation. This is why the reviewed evaluator can accept alternative repairs while continuing to reject contract-breaking shortcuts. Each rejection needs a traceable connection to an obligation available to the system. This connection links task specification to scoring validity and explains why evaluator review belongs in the interpretation of the findings.

6.4 Clarification as a Distinct Interaction Outcome

The clarification results separate explicit information seeking from successful task completion. On the deliberately incomplete task, the three higher-success systems reached passing outcomes, although only GPT-5.4 captured the authorised clarification opportunities. The observation supports reporting both endpoints: successful code does not itself reveal whether a system identified an information gap, and asking a permitted question does not itself establish a better repair.

Li et al. study clarification for underspecified Python generation and distinguish predicting whether clarification is needed, ranking candidate questions, and generating code [7]. The TSB's opportunity-capture and request-precision measures address related distinctions within a repository task and a fixed response policy. The present evidence covers one omitted business requirement, repeated three times per system.

The unnecessary-request records add a complementary boundary. Requests in a complete brief consume interaction opportunities without obtaining authorised information, but their occurrence does not by itself show that asking caused the failed outcome. Similarly, the absence of requests cannot be treated as evidence of ideal interaction when the same system misses the deliberately available opportunity. A broader evaluation would vary the type of missing requirement and separately control response availability. It could then distinguish identifying uncertainty, formulating a relevant question and benefiting from the answer, instead of inferring all three from one completion score. CLAMBER supplies a broader taxonomy of ambiguity for designing such variation [87]. STYLE further studies the transfer of clarification strategies between conversational-search domains [105]. These studies suggest varying both the missing-information type and its domain in a repository-debugging evaluation.

Task documentation and clarification access determine different information conditions for a repair. SWE-Bench Pro supplies augmented requirements with its tasks [52]; TSB withholds

one requirement in one task and makes its fixed answer conditional on an eligible request. By comparison, τ-bench evaluates tool agents in dynamic conversations with simulated users under domain policies [49]. TSB's fixed response removes variation in the answer supplied after an eligible question, making opportunity capture interpretable under a common answer condition. It does not assess whether an agent can negotiate changing intent, recover from an ambiguous answer or maintain policy compliance through an extended dialogue. The passing non-asking runs further show that final correctness cannot substitute for observing whether the authorised channel was used. Comparisons should therefore record both how missing information becomes available and what interaction demands precede the final outcome.

A follow-up could hold the repository fault and intended behaviour constant while comparing three information conditions: the complete requirement supplied in the initial brief; the requirement omitted but available through an authorised clarification request; and the requirement omitted with no answer supplied during the run. Each condition would use fresh resets, the same execution budget and the same final-state checks. Assigning fresh runs to conditions in random order would reduce systematic ordering effects. The response policy would need to be stated in each brief so that agents could decide whether asking was a useful action.

The complete-brief condition would establish performance when the required information is available from the outset. Comparing it with the clarification condition would test the additional demand of recognising and obtaining missing information. Comparing the two incomplete-brief conditions would estimate the effect of making the answer available through the stated channel. Within the clarification condition, request records and checks targeting the omitted requirement would distinguish failure to obtain the answer from failure to implement it. Repeating this design across requirements with different valid answers would also test whether successful completion without asking persists beyond one business rule. This would extend RQ2 from describing channel use to examining when access to clarification changes repair outcomes, while retaining separate measures of information seeking and implementation.

6.5 Construct Validity

Construct validity concerns whether the metrics represent the properties they are intended to measure [106]. Here, clarification seeking is observed under the eligibility and response rules in Section 4.3. Opportunity capture, request precision and unnecessary requests describe different aspects of that behaviour. Their joint interpretation is limited by the single designated information-gap task: repeated attempts improve description of that condition but do not sample the diversity of missing requirements. Likewise, CRUXEval separately assesses predicting program inputs and outputs [46]. TSB's repair endpoint combines code interpretation with navigation, editing and execution, so it cannot identify an isolated code-reasoning capability.

The same distinction between a measured condition and a broader capability applies to robustness. TSB's prespecified metric concerns completion without an evaluator response on the higher-tier tasks. The reliability dimensions identified by Rabanser et al. make clear why that outcome does not establish resilience to changed inputs or faulty tools [72]. A follow-up could vary semantically equivalent instructions, tool-return failures and application-state perturbations separately, while recording recovery and error severity.

The comparison is interface-aware. Interactive performance reflects the model, product interface, and interaction process [107]. Native IDEs and the shared API wrapper expose different file

operations, command paths, and approval mechanisms. The protocol holds task briefs, starting repositories, deterministic resets, wall-clock limits, and evaluator-only final-state checks constant. Routine IDE permission prompts follow a mechanical policy and do not count as assistance, although approval time remains within elapsed time. Permission events and accepted API actions are reported as separate interface-specific diagnostics. Final-state success under the common protocol is the primary comparison; broader IDE usability is outside this construct.

Insufficient functional test suites can overestimate correctness [9]. The separate functional and architectural diagnostics make the assessed obligations visible, but their coverage must still be judged against the task contract. Requiring all applicable checks to pass strengthens the declared completion criterion without demonstrating that untested behaviour is correct.

The oracle problem concerns whether an observed behaviour is judged correctly against its intended meaning [12]. The Agentic Benchmark Checklist further separates task validity, linking task success to the target capability, from outcome validity, linking a positive score to actual task success [75]. This distinction clarifies what TSB's validation steps establish. Seeded-failure and oracle-repair checks provide evidence that the configured tasks expose an intended defect and admit a repair; they do not show that every implementation accepted by the evaluator satisfies the public contract. Conversely, a deterministic check can consistently reject an acceptable alternative if its condition encodes an unrequired implementation form. The uniform public-contract review addresses identified mismatches between those acceptance conditions and the stated requirements. Retaining the original scores makes its effect inspectable, while the same submitted artefacts keep scoring changes separate from new repair attempts.

The submission rule adds a procedural condition to the behavioural assessment. Reaching a state that passes some checks and completing the required workflow are distinct endpoints, so changing the stopping or submission rule would change the meaning of success even with the same tests. In TSB, retaining both the terminal state and stopping reason makes that distinction inspectable. It also prevents a misleading interpretation of early termination as speed: elapsed time to an unsuccessful stop cannot be exchanged directly for time to a completed repair. Together, the task contract, available feedback and completion rule define the outcome being compared; none can be inferred from a pass percentage alone.

Consequently, reviewed success means that a normal submission meets the applicable assessed requirements under the recorded evaluator version. Security, maintainability and behaviour outside the specified cases require separate assessment. This interpretation gives the success endpoint a clear use: comparing complete task outcomes under a shared assessment, supplemented by diagnostic evidence where that assessment is incomplete or where submissions fail.

6.6 Internal Validity

Internal validity concerns whether observed differences arise from the evaluated systems rather than uncontrolled variation [108]. Model updates, inference settings, prompts, and stochastic generation are the principal threats. HELM motivates transparent evaluation under standardised conditions [99]. In this study, the corresponding control is to freeze and record each evaluated configuration. Cursor and Devin Desktop use the default agent model or mode and settings observed at configuration freeze. GPT-5.4 uses the dated OpenAI snapshot `gpt-5.4-2026-03-05`; Qwen2.5-Coder-7B-Instruct uses a pinned official Hugging Face revision on a managed vLLM endpoint.

Controllable settings are specified in versioned configuration records, with changes documented through the amendment process. For commercial IDEs, these records capture the product version, displayed default model or mode, visible configuration options, and evaluation date. Detected configuration changes are documented alongside the disposition of affected runs and the applicable replacement freeze. This procedure preserves traceability; it does not imply that all selected observations used identical configurations.

Environment stability and model-service stability are separate controls. StableToolBench addresses changing online API behaviour through caching and simulation [60]. TSB instead restores executable task repositories and application state, so its resets control the software being repaired. They cannot freeze an externally operated product's hidden implementation or guarantee identical model responses. The configuration records and amendment process therefore complement reset validation: they document which execution condition produced an observation, while repeated runs describe variation within the recorded condition. This distinction also explains the invalid-run policy. An infrastructure interruption and a valid agent-caused failure may both prevent completion, but treating them identically would change the intended system comparison. Their recorded adjudication and replacement provenance are part of interpreting the selected dataset.

Each system contributed three selected valid terminal observations on every deterministic task. Docker Compose restores the versioned repository, containers, volumes, and database state before each run. All 12 task packages passed reset, intended-failure, oracle-repair, full-check, and second-reset validation. Their start commits, briefs, evaluator-only tests, architectural invariants, the seeded 144-run schedule, evaluation bundle, and common wall-clock limit are bound by the strict main freeze. Main collection was completed on 26 September 2026, yielding all 144 scheduled observations, with 36 per system and 36 per tier. The selected dataset retains valid unsuccessful outcomes and excludes invalidated attempts under the recorded adjudications. Section 5 reports system-level estimates from this completed dataset.

Post-collection review introduces outcome-informed judgement. Corrections therefore use a common public-contract interpretation for all systems and repetitions of each affected task, retain original scores, and enter a separate analysis layer. Appendix D.8 records the decisions and score transitions. Section 5.7 reports their effect on comparative conclusions. Ten binary outcomes change, while the paired tests still do not distinguish the three higher-success systems.

The sensitivity comparison also separates evaluator dependence from execution variability. The original and reviewed scores describe the same submitted artefacts, so agreement between their broad conclusions is not an independent replication. Their differing point-estimate ordering nevertheless matters: it shows that selecting a winner among the three higher-success systems would depend on details of how a small number of borderline outcomes were assessed. Applying the same public-contract principle across affected systems makes the amendments traceable, but does not remove the outcome-informed nature of the review. The complementary but incomplete patch-assessment methods studied by Wang et al. reinforce the need to examine evaluator evidence rather than regard a reviewed score as a guarantee of semantic correctness [70].

Statistical uncertainty is constrained by the task sample as well as the number of runs. The three repetitions describe variation within each task–system condition; they do not turn the twelve constructed tasks into 144 independently sampled software problems. The distinction between fixed-item performance and generalised accuracy in LLM evaluation is relevant here [101]:

task-level pairing and resampling preserve grouping, but cannot supply diversity absent from the benchmark. Likewise, an adjusted test that does not distinguish two systems is not evidence that their practical performance is identical [109]. An equivalence claim would require prespecified bounds for a practically negligible difference and an analysis designed to assess them; this study does not perform that analysis.

The pilot identified one material protocol defect: the provisional 15-action API limit repeatedly stopped progressing GPT-5.4 runs. The amendment log records the increase to 30 actions before replacement attempts began from fresh resets. The 180-second command timeout, 5,400-second task limit, three-response no-progress rule, 100,000-character context boundary, and API sandbox restrictions remain fixed for the main experiment. Seeded random order distributes temporal ordering across systems within each task–repetition block. The invalid-run policy separates infrastructure failures from valid agent-caused failures and preserves both invalid attempts and their replacements in the provenance record.

Comparable per-run cost is unavailable for all 144 selected observations, and token usage is unavailable for the 72 native-IDE observations. Consequently, no cross-system cost or token-efficiency ranking is reported. This missing dimension matters because accuracy and cost jointly inform practical agent evaluation [73]. Account-level managed-endpoint charges also include configuration and infrastructure activity outside the selected-run set and cannot be allocated retrospectively as comparable run costs.

Failure coding is limited by unequal trace availability. The four unsuccessful native-IDE runs lack usable interaction transcripts, so their final diffs and evaluator outputs support descriptions of the resulting fault but not claims about the sequence of debugging decisions. The qualitative coding was not independently double-coded. Counts therefore describe documented evidence rather than unbiased frequencies of every possible agent behaviour.

6.7 External Validity

External validity concerns transfer beyond the evaluated setting [108]. The TSB contains 12 deterministic tasks spanning static websites, single-service APIs, multi-service applications, and Vue–FastAPI–PostgreSQL containers. This compact synthetic task set does not represent the full distribution of industrial software engineering, including the architectural and technical debt and continuous-integration/deployment challenges described in prior research [110, 111]. Results apply to the evaluated product versions, model snapshots, tasks, and protocol.

The pilot includes one selected run per system–tier cell. It tests protocol operation, evidence completeness, reset behaviour and execution boundaries; its descriptive outcomes are excluded from the 144-run main analysis. It also did not approach the 100,000-character context boundary, leaving that threshold an operational bound rather than an empirically optimised value for larger repositories.

Repository scale introduces an information-access demand that architectural tiers alone do not measure. CrossCodeEval constructs completion problems requiring cross-file context [44], whereas Lost in the Middle shows sensitivity to the position of relevant information in question-answering and retrieval tasks [96]. Together, they motivate separating the amount of available context from whether the needed evidence is found and used. TSB did not manipulate context placement or measure retrieval quality, so these studies cannot diagnose its specific failed runs or establish how its configured systems would handle much larger repositories. A transfer study would need to

control repository size and dependency distribution alongside the repair obligation, and record the evidence actually exposed to the agent.

Within this scope, the TSB is designed to provide controlled evidence on system-level debugging. Its faults include CORS mismatches, API-contract errors, database connection failures, schema migrations, and cascading frontend–backend inconsistencies that are less visible in function-level benchmarks such as HumanEval and MBPP [8, 9]. Deterministic resets and recorded evaluation versions make these comparisons traceable.

The literature comparison identifies specific conditions for transferring the findings. SWE-bench and SWE-Bench Pro evaluate repository issues, while Defects4REST supplies real REST API defects [10, 52, 64]. TSB's constructed tasks instead make selected requirements and initial states controllable. Similarity in programming language or use of containers is therefore insufficient to justify transferring a success rate: task provenance, information availability, permitted operations and scoring obligations would also need to align. The contribution that transfers most directly is the evaluation approach—retaining configuration records, final-state evidence and scoring provenance—rather than the observed ordering of systems. Testing the same configurations on a broader real-defect sample would examine this transfer.

6.8 Practical Implications and Priorities for Further Evaluation

For users choosing a system for a similar bounded repair workflow, the results support a sequence of questions rather than a single leaderboard decision. First, can the configured system reach and modify the relevant repository through its actual interface? Second, which task obligations does the final-state assessment cover, and does the submission meet them? Third, how long do successful completions take under that execution route? The failure-stage, diagnostic and elapsed-time evidence answer different parts of this sequence. A short termination time is useful operational information, but it supplies no successful repair when the task remains unresolved.

The high observed success of three systems also limits how decisively this compact benchmark can differentiate them. Their task profiles are more informative than small differences in their aggregate rates, particularly when the original and reviewed scoring versions alter the descriptive ordering. A practical selection would therefore need evidence on the user's own workload and execution constraints before transferring these rates to a deployment decision.

Monetary cost and developer effort would add different information to that decision. Cost-aware agent evaluation motivates recording usage and expenditure alongside successful outcomes [73]. A future comparison should capture the pricing version, attributable model and tool usage, retries and total cost prospectively, rather than infer them from elapsed time. Human effort requires separate observation. Mozannar et al.'s suggestion-display model accounts for verification, editing, writing and latency, and shows why suggestion acceptance alone is an incomplete utility signal [112]. Its retrospective evaluation is not a direct estimate of productivity in TSB. An assisted repair study would need to record review, correction and waiting effort alongside task success. These measurements would extend the decision evidence; their absence prevents an overall utility ranking from the present experiment.

For benchmark development, the next priority is to separate architectural depth from task semantics. Repeating a matched requirement or fault family across architectures would test whether the same obligation becomes harder when its implementation spans more services. CheckList's capability-by-test-type approach suggests a complementary way to organise coverage [74]: a future

TSB matrix could cross obligations such as persistence and accepted-input semantics with nominal, boundary and lifecycle cases. The matrix would identify untested obligations explicitly, instead of using a larger check count as evidence of complete coverage.

Evaluator development also needs examples that challenge the acceptance rule. Guided by the distinction between task and outcome validity [75], a future audit could pair known contract-breaking submissions with alternative contract-satisfying repairs and document which each check accepts. Such an audit would examine both missed defects and unnecessary implementation restrictions before collecting a new comparison dataset.

BetterBench identifies continued usability, maintenance contacts, feedback channels and an explicit end to maintenance as aspects of benchmark quality [113]. Future TSB releases should preserve the version behind each result and document subsequent changes to tasks, dependencies and scoring.

LiveBench uses frequently refreshed questions with objective ground-truth scoring [114]. Reporting TSB results separately on retained tasks and new faults would distinguish stable-set performance from performance on refreshed content. Training-data overlap would require separate investigation.

7 Conclusion

This thesis examined how complete agentic coding systems perform bounded repairs across application components. The Tiered System Benchmark (TSB) makes the comparison inspectable through twelve deterministic tasks, resettable starting states, explicit task contracts and evaluator-only checks. Its central contribution connects the submitted application state with the execution route and assessment conditions behind each reported outcome. Success, diagnostic checks, stopping reasons and clarification records describe complementary aspects of a repair.

For RQ1, success did not decline monotonically across the selected tiers, from static websites to multi-container applications. The three higher-success systems completed all selected Level 3 runs, while some lower-tier persistence repairs remained unsuccessful. Each tier contains three tasks with differing fault semantics and requirements, so the observed pattern does not isolate the effect of architectural depth.

For RQ2, clarification seeking and successful implementation emerge as distinct outcomes. GPT-5.4 captured all three authorised opportunities on the deliberately incomplete task, whereas the other two higher-success systems also completed that task without using the response channel. Qwen made unnecessary requests on two complete-brief runs. These observations show why opportunity capture and request precision should accompany completion scores: neither passing code nor a low request count establishes that a system recognised missing intent appropriately. This finding concerns the single withheld business requirement.

For RQ3, reviewed success was 35/36 for Cursor, 34/36 for GPT-5.4, 33/36 for Devin Desktop and 0/36 for the configured Qwen2.5-Coder-7B-Instruct system. The exploratory task-paired tests distinguished each of the first three systems from Qwen after multiplicity adjustment, but did not distinguish those three from one another. Qwen's empty final diffs show that no effective task-code change remained at the end of the evaluated workflow. A separate 72-run control retained the model and task conditions while relaxing only JSON newline handling in one arm. Both arms produced zero successful repairs. Replay identified requests affected by the restriction, but the newly accepted requests in the relaxed arm failed on nonexistent paths. The tested formatting change was insufficient to enable repair.

Scoring sensitivity limits how strongly the systems can be compared. Reviewing the same submissions against the public contracts changed credited completions and the descriptive ordering of the three higher-success systems. Their broad separation from Qwen remained, while the tests still did not distinguish them from one another. Retaining both score versions makes the effect of assessment changes inspectable.

The practical contribution is a way to ask more precise questions of a coding-system comparison. Can the system act effectively through its interface, and does its repair meet the assessed task requirements? Do the scoring and completion rules justify the resulting success claim? The task packages and versioned evidence support inspection of those questions across the evaluated configurations. Their transfer beyond this setting remains limited by the constructed task sample, configuration differences, incomplete native trajectories and unavailable comparable cost data.

Further interface evaluation should examine native tool calling and broader action-handling changes while holding the model and tasks fixed. It should also compare matched fault families across architectures and broaden the range of missing requirements. A larger real-defect sample and more complete trace capture would test how far the observed patterns extend.

References

[1] M. Bruch, M. Monperrus, and M. Mezini, "Learning from examples to improve code completion systems," in *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering*, ESEC/FSE '09, (New York, NY, USA), pp. 213–222, ACM, 2009.

[2] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, and E. Aftandilian, "Productivity assessment of neural code completion," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming (MAPS '22)*, pp. 21–29, Association for Computing Machinery, 2022.

[3] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[4] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying llm-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025.

[5] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, pp. 50528–50652, Curran Associates, Inc., 2024.

[6] W. Wang, H. Ning, G. Zhang, L. Liu, and Y. Wang, "Rocks Coding, Not Development: A Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 699–721, 2024.

[7] H.-S. X. Li, M. Mesgar, A. Martins, and I. Gurevych, "Python code generation by asking clarification questions," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14287–14306, Association for Computational Linguistics, 2023.

[8] D. G. Paul, H. Zhu, and I. Bayley, " Benchmarks and Metrics for Evaluations of Code Generation: A Critical Review ," in *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*, (Los Alamitos, CA, USA), pp. 87–94, IEEE Computer Society, July 2024.

[9] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 21558–21572, 2023.

[10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?," in *The Twelfth International Conference on Learning Representations*, 2024.

[11] J. Yang, A. Prabhakar, K. Narasimhan, and S. Yao, "InterCode: Standardizing and benchmarking interactive coding with execution feedback," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 23826–23854, 2023.

[12] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[13] M. White, C. Vendome, M. Linares-Vasquez, and D. Poshyvanyk, "Toward deep learning software repositories," in *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*, pp. 334–345, 2015.

[14] X. Gu, H. Zhang, D. Zhang, and S. Kim, "Deep api learning," in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE '16, (New York, NY, USA), pp. 631–642, Association for Computing Machinery, 2016.

[15] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, "Summarizing source code using a neural attention model," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (K. Erk and N. A. Smith, eds.), (Berlin, Germany), pp. 2073–2083, Association for Computational Linguistics, Aug. 2016.

[16] V. Raychev, M. T. Vechev, and E. Yahav, "Code completion with statistical language models," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '14, (New York, NY, USA), pp. 419–428, ACM, 2014.

[17] V. J. Hellendoorn and P. Devanbu, "Are deep neural networks the best choice for modeling source code?," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, pp. 763–773, 2017.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, Curran Associates, Inc., 2017.

[19] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, Curran Associates, Inc., 2020.

[20] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. J. Jiang, "Github copilot ai pair programmer: Asset or liability?," *Journal of Systems and Software*, vol. 203, p. 111734, 2023.

[21] N. Nguyen and S. Nadi, "An empirical evaluation of github copilot's code suggestions," in *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*, pp. 1–5, 2022.

[22] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, CHI EA '22, (New York, NY, USA), pp. 1–7, Association for Computing Machinery, 2022.

[23] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *The Twelfth International Conference on Learning Representations*, 2024.

[24] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *International Conference on Learning Representations*, pp. 9695–9717, 2024.

[25] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, "API-Bank: A comprehensive benchmark for tool-augmented LLMs," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3102–3116, Association for Computational Linguistics, 2023.

[26] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large Language Model Connected with Massive APIs," in *Advances in Neural Information Processing Systems*, vol. 37, pp. 126544–126565, Curran Associates, Inc., 2024.

[27] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2471–2484, Association for Computational Linguistics, 2023.

[28] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, and X. Ma, "Repoformer: Selective retrieval for repository-level code completion," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 53270–53290, PMLR, 2024.

[29] A. Blinn, X. Li, J. H. Kim, and C. Omar, "Statically contextualizing large language models with typed holes," *Proc. ACM Program. Lang.*, vol. 8, pp. 468–498, Oct. 2024.

[30] Z. Chen, X. Tang, G. Deng, F. Wu, J. Wu, Z. Jiang, V. Prasanna, A. Cohan, and X. Wang, "LocAgent: Graph-Guided LLM Agents for Code Localization," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8697–8727, Association for Computational Linguistics, 2025.

[31] K. Liu, A. Koyuncu, D. Kim, and T. F. Bissyandé, "TBar: revisiting template-based automated program repair," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 31–42, ACM, 2019.

[32] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, "CoCoNuT: combining context-aware neural translation models using ensemble for program repair," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 101–114, 2020.

[33] N. Jiang, T. Lutellier, and L. Tan, "CURE: Code-Aware Neural Machine Translation for Automatic Program Repair," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 1161–1173, 2021.

[34] B. Berabi, J. He, V. Raychev, and M. Vechev, "TFix: Learning to Fix Coding Errors with a Text-to-Text Transformer," in *Proceedings of the 38th International Conference on Machine Learning*, vol. 139, pp. 780–791, PMLR, 2021.

[35] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*, pp. 1592–1604, Association for Computing Machinery, 2024.

[36] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable Code Actions Elicit Better LLM Agents," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 50208–50232, PMLR, 2024.

[37] C. S. Xia and L. Zhang, "Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 819–831, ACM, 2024.

[38] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 2188–2200, 2025.

[39] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652, Curran Associates, Inc., 2023.

[40] A. Zirak and H. Hemmati, "Improving Automated Program Repair with Domain Adaptation," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 3, pp. 1–43, 2024.

[41] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, and Y. Zhang, "Training Software Engineering Agents and Verifiers with SWE-Gym," in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267 of *Proceedings of Machine Learning Research*, pp. 47717–47737, PMLR, 2025.

[42] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *ArXiv*, vol. abs/2107.03374, 2021.

[43] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[44] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, "CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 46701–46723, Curran Associates, Inc., 2023.

[45] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems," in *International Conference on Learning Representations*, pp. 47832–47850, 2024.

[46] A. Gu, B. Roziere, H. J. Leather, A. Solar-Lezama, G. Synnaeve, and S. Wang, "CRUXEval: A Benchmark for Code Reasoning, Understanding and Execution," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 16568–16621, PMLR, 2024.

[47] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," in *International Conference on Learning Representations*, pp. 15585–15606, 2024.

[48] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Advances in Neural Information Processing Systems*, vol. 37, pp. 52040–52094, Curran Associates, Inc., 2024.

[49] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," in *International Conference on Learning Representations*, pp. 9965–10017, 2025.

[50] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. Zebaze, X. Hong, W.-D. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, N. Jain, A. Gu, Z. Cheng, J. Liu, Q. Liu, Z. Wang, B. Hui, N. Muennighoff, D. Lo, D. Fried, X. Du, H. de Vries, and L. von Werra, "BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions," in *International Conference on Learning Representations*, pp. 66602–66656, 2025.

[51] D. Zan, Z. Huang, W. Liu, H. Chen, S. Xin, L. Zhang, Q. Liu, A. Li, L. Chen, X. Zhong, S. Liu, Y. Xiao, L. Chen, Y. Zhang, J. Su, T. Liu, R. Long, M. Ding, and L. Xiang, "Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[52] X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, C. Rane, K. Sampath, M. Krishnan, S. R. Kundurthy, S. M. Hendryx, Z. Wang, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler, "SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?," in *Proceedings of the 43rd International Conference on Machine Learning*, vol. 306 of *Proceedings of Machine Learning Research*, pp. 23917–23934, PMLR, 2026.

[53] J. Chen, X. Xu, H. Wei, C. Chen, and B. Zhao, "Swe-ci: Evaluating agent capabilities in maintaining codebases via continuous integration," *arXiv preprint arXiv:2603.03823*, 2026. Preprint.

[54] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. I. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code," in *International Conference on Learning Representations*, pp. 58791–58831, 2025.

[55] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, "SWE-bench Goes Live!," in *Advances in Neural Information Processing Systems 38*, pp. 163347–163372, 2025.

[56] X. Chen, M. Lin, N. Schaerli, and D. Zhou, "Teaching Large Language Models to Self-Debug," in *The Twelfth International Conference on Learning Representations*, pp. 8746–8825, 2024.

[57] R. Just, C. Parnin, I. Drosos, and M. D. Ernst, "Comparing developer-provided to user-provided tests for fault localization and automated program repair," in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 287–297, ACM, 2018.

[58] SWE-bench Team, "SWE-bench: Official evaluation harness and docker setup documentation." GitHub repository. Accessed: 2 October 2026.

[59] J. Yang, A. Prabhakar, K. Narasimhan, and S. Yao, "InterCode: Official implementation and quick start." GitHub repository. Accessed: 2 October 2026.

[60] Z. Guo, S. Cheng, H. Wang, S. Liang, Y. Qin, P. Li, Z. Liu, M. Sun, and Y. Liu, "StableToolBench: Towards stable large-scale benchmarking on tool learning of large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 11143–11156, Association for Computational Linguistics, 2024.

[61] P. Di Francesco, P. Lago, and I. Malavolta, "Architecting with microservices: A systematic mapping study," *Journal of Systems and Software*, vol. 150, pp. 77–97, 2019.

[62] P. Jamshidi, C. Pahl, N. C. Mendonça, J. Lewis, and S. Tilkov, "Microservices: The journey so far and challenges ahead," *IEEE Software*, vol. 35, no. 3, pp. 24–35, 2018.

[63] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Transactions on Software Engineering*, vol. 47, no. 2, pp. 243–260, 2021.

[64] R. Mehta, P. Katkhede, and M. Motwani, "Defects4rest: A benchmark of real-world defects to enable controlled testing and debugging studies for rest apis," in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, pp. 1597–1608, ACM, 2026.

[65] L. Gregor, A. Hentschel, L. Kastner, and A. Pretschner, "A taxonomy of integration-relevant faults for microservice testing," in *2025 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pp. 138–149, IEEE, 2025.

[66] X. Kong, L. Zhang, W. E. Wong, and B. Li, "The impacts of techniques, programs and tests on automated program repair: An empirical study," *Journal of Systems and Software*, vol. 137, pp. 480–496, 2018.

[67] Q. Xin and S. P. Reiss, "Identifying test-suite-overfitted patches through test case generation," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 226–236, 2017.

[68] H. Ye, M. Martinez, and M. Monperrus, "Automated patch assessment for program repair at scale," *Empirical Software Engineering*, vol. 26, no. 2, 2021.

[69] H. Ye, M. Martinez, T. Durieux, and M. Monperrus, "A comprehensive study of automatic program repair on the QuixBugs benchmark," *Journal of Systems and Software*, vol. 171, p. 110825, 2021.

[70] S. Wang, M. Wen, B. Lin, H. Wu, Y. Qin, D. Zou, X. Mao, and H. Jin, "Automated Patch Correctness Assessment: How Far are We?," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, pp. 968–980, ACM, 2020.

[71] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, H. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang, "ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities," in *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 1160–1183, Association for Computational Linguistics, 2025.

[72] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala, and A. Narayanan, "Towards a Science of AI Agent Reliability," in *Proceedings of the 43rd International Conference on Machine Learning*, vol. 306 of *Proceedings of Machine Learning Research*, pp. 102769–102820, PMLR, 2026.

[73] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI Agents That Matter," *Transactions on Machine Learning Research*, 2025.

[74] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912, Association for Computational Linguistics, 2020.

[75] Y. Zhu, T. Jin, Y. Pruksachatkun, A. Zhang, S. Liu, S. Cui, S. Kapoor, S. Longpre, K. Meng, R. Weiss, F. Barez, R. Gupta, J. Dhamala, J. Merizian, M. Giulianelli, H. Coppock, C. Ududec, A. Kellermann, J. Sekhon, J. Steinhardt, S. Schwettmann, A. Narayanan, M. A. Zaharia, I. Stoica, P. Liang, and D. Kang, "Establishing best practices in building rigorous agentic benchmarks," in *Advances in Neural Information Processing Systems*, vol. 38, Curran Associates, Inc., 2025.

[76] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root Cause Localization of Performance Issues in Microservices," in *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–9, 2020.

[77] D. Liu, C. He, X. Peng, F. Lin, C. Zhang, S. Gong, Z. Li, J. Ou, and Z. Wu, "MicroHECL: High-Efficient Root Cause Localization in Large-Scale Microservice Systems," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 338–347, 2021.

[78] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, and X. Li, "MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments," in *Proceedings of the Web Conference 2021*, pp. 3087–3098, 2021.

[79] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinsky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou, "An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 3–18, 2019.

[80] C. S. Meiklejohn, A. Estrada, Y. Song, H. Miller, and R. Padhye, "Service-Level Fault Injection Testing," in *Proceedings of the ACM Symposium on Cloud Computing*, pp. 388–402, 2021.

[81] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API Fuzzing," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pp. 748–758, 2019.

[82] E. Viglianisi, M. Dallago, and M. Ceccato, "RESTTESTGEN: Automated Black-Box Testing of RESTful APIs," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, pp. 142–152, IEEE, 2020.

[83] S. I. Ross, F. Martinez, S. Houde, M. Muller, and J. D. Weisz, "The programmer's assistant: Conversational interaction with a large language model for software development," in *Proceedings of the 28th International Conference on Intelligent User Interfaces*, p. 491–514, Association for Computing Machinery, 2023.

[84] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang, "ClarifyGPT: A Framework for Enhancing LLM-Based Code Generation via Requirements Clarification," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2332–2354, 2024.

[85] A. Testoni and R. Fernández, "Asking the Right Question at the Right Time: Human and Model Uncertainty Guidance to Ask Clarification Questions," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 258–275, 2024.

[86] M. J. Zhang and E. Choi, "Clarify When Necessary: Resolving Ambiguity Through Interaction with LMs," in *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 5541–5558, 2025.

[87] T. Zhang, P. Qin, Y. Deng, C. Huang, W. Lei, J. Liu, D. Jin, H. Liang, and T.-S. Chua, "CLAMBER: A Benchmark of Identifying and Clarifying Ambiguous Information Needs in Large Language Models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10746–10766, Association for Computational Linguistics, 2024.

[88] S. Rao and H. Daumé, III, "Learning to Ask Good Questions: Ranking Clarification Questions using Neural Expected Value of Perfect Information," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2737–2746, Association for Computational Linguistics, 2018.

[89] M. J. Q. Zhang, W. B. Knox, and E. Choi, "Modeling Future Conversation Turns to Teach LLMs to Ask Clarifying Questions," in *The Thirteenth International Conference on Learning Representations*, pp. 60722–60742, 2025.

[90] S. Barke, M. B. James, and N. Polikarpova, "Grounded Copilot: How Programmers Interact with Code-Generating Models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 85–111, 2023.

[91] S. Nguyen, H. M. Babe, Y. Zi, A. Guha, C. J. Anderson, and M. Q. Feldman, "How Beginning Programmers and Code LLMs (Mis)read Each Other," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pp. 1–26, 2024.

[92] J. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and Q. Yang, "Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, pp. 1–21, 2023.

[93] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, "Using an LLM to Help With Code Understanding," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13, Association for Computing Machinery, 2024.

[94] R. Khojah, M. Mohamad, P. Leitner, and F. G. de Oliveira Neto, "Beyond Code Generation: An Observational Study of ChatGPT Usage in Software Engineering Practice," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1819–1840, 2024.

[95] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[96] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[97] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations*, 2023.

[98] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[99] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar, B. Newman, B. Yuan, B. Yan, C. Zhang, C. Cosgrove, C. D. Manning, C. Ré, D. Acosta-Navas, D. A. Hudson, E. Zelikman, E. Durmus, F. Ladhak, F. Rong, H. Ren, H. Yao, J. Wang, K. Santhanam, L. Orr, L. Zheng, M. Yuksekgonul, M. Suzgun, N. Kim, N. Guha, N. S. Chatterji, O. Khattab, P. Henderson, Q. Huang, R. A. Chi, S. M. Xie, S. Santurkar, S. Ganguli, T. Hashimoto, T. Icard, T. Zhang, V. Chaudhary, W. Wang, X. Li, Y. Mai, Y. Zhang, and Y. Koreeda, "Holistic evaluation of language models," *Transactions on Machine Learning Research*, 2023.

[100] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 69, no. 3, pp. 369–390, 2007.

[101] D. Keller, K. Kwegyir-Aggrey, R. Steed, A. K. Rao, J. L. Sharp, and A. S. Bergman, "Expanding the AI Evaluation Toolbox with Statistical Models," in *Proceedings of the 43rd International Conference on Machine Learning*, vol. 306 of *Proceedings of Machine Learning Research*, pp. 56704–56739, PMLR, 2026.

[102] J. Hemerik and J. Goeman, "Exact testing with random permutations," *TEST*, vol. 27, no. 4, pp. 811–825, 2018.

[103] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[104] R. Verdecchia, E. Engström, P. Lago, P. Runeson, and Q. Song, "Threats to validity in software engineering research: A critical reflection," *Information and Software Technology*, vol. 164, p. 107329, 2023.

[105] Y. Chen, C. Huang, Y. Deng, W. Lei, D. Jin, J. Liu, and T.-S. Chua, "STYLE: Improving Domain Transferability of Asking Clarification Questions in Large Language Model Powered Conversational Agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 10633–10649, Association for Computational Linguistics, 2024.

[106] D. I. K. Sjøberg and G. R. Bergersen, "Construct validity in software engineering," *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1374–1396, 2023.

[107] M. Lee, M. Srivastava, A. Hardy, J. Thickstun, E. Durmus, A. Paranjape, I. Gerard-Ursin, X. L. Li, F. Ladhak, F. Rong, R. E. Wang, M. Kwon, J. S. Park, H. Cao, T. Lee, R. Bommasani, M. S. Bernstein, and P. Liang, "Evaluating human-language model interaction," *Transactions on Machine Learning Research*, 2023.

[108] J. Siegmund, N. Siegmund, and S. Apel, "Views on internal and external validity in empirical software engineering," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1, pp. 9–19, 2015.

[109] D. Lakens, "Equivalence tests: A practical primer for t tests, correlations, and meta-analyses," *Social Psychological and Personality Science*, vol. 8, no. 4, pp. 355–362, 2017.

[110] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012.

[111] M. Shahin, M. Ali Babar, and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017.

[112] H. Mozannar, G. Bansal, A. Fourney, and E. Horvitz, "When to Show a Suggestion? Integrating Human Feedback in AI-Assisted Programming," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 9, pp. 10137–10144, 2024.

[113] A. Reuel, A. Hardy, C. Smith, M. Lamparth, M. Hardy, and M. J. Kochenderfer, "BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices," in *Advances in Neural Information Processing Systems*, vol. 37, Curran Associates, Inc., 2024.

[114] C. White, S. Dooley, M. Roberts, A. Pal, B. Feuer, S. Jain, R. Shwartz-Ziv, N. Jain, K. Saifullah, S. Dey, Shubh-Agrawal, S. S. Sandha, S. Naidu, C. Hegde, Y. LeCun, T. Goldstein, W. Neiswanger, and M. Goldblum, "LiveBench: A Challenging, Contamination-Limited LLM Benchmark," in *International Conference on Learning Representations*, pp. 91595–91631, 2025.

Appendix A: Detailed Task Case Studies in the Tiered System Benchmark

This appendix presents two representative TSB case studies. The complete benchmark contains 12 deterministic debugging tasks, with three tasks at each of the four architectural tiers. The examples below focus on Level 3 and Level 4 because they illustrate cross-service dependencies and the repair of architectural faults. Level 1 and Level 2 tasks are included in the benchmark and evaluate frontend, backend-routing, validation, and local-persistence faults; they are not described in detail in this appendix.

A.1 Level 3 Task: Cross-Origin Resource Sharing (CORS) Configuration

- **System Setup:** A containerised web application comprising a Vue.js front end exposed on port 8080 and a Python FastAPI back end exposed on port 8000.
- **Injected Fault:** The backend is deliberately configured without an appropriate `Access-Control-Allow-Origin` header[6], causing browser CORS policies to block frontend API requests.
- **Expected Agent Resolution:** The agent must autonomously diagnose the cross-origin restriction, identify the relevant backend configuration, configure an appropriate CORS policy for the frontend origin, and restart or reload the affected service where required. CORS is defined as a repairable technical fault and must not trigger evaluator assistance.
- **Evaluation Conditions:** Evaluator-only frozen tests verify that the specified frontend origin can access the required backend endpoint and that the backend remains healthy. An architectural invariant rejects an unnecessarily permissive wildcard policy when the task requires the named origin. The tests are stored outside the agent-accessible repository.

A.2 Level 4 Task: Database Schema Cascading Failure

- **System Setup:** A multi-container full-stack application consisting of a PostgreSQL database, a Python backend using an Object-Relational Mapping (ORM) layer, a REST API, and a frontend user interface.
- **Injected Fault:** The task begins from a deliberately inconsistent system state after a database migration removes the `age` column from the primary user table.
- **Expected Agent Resolution:** The agent must autonomously trace the cascading dependency and restore consistency across the system. This may require updating backend models, API validation or response schemas, database-access logic, and frontend data bindings. The evaluation assesses the final functioning system rather than requiring one predetermined file-edit sequence.

[6] https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Access-Control-Allow-Origin

- **Evaluation Conditions:** Evaluator-only frozen tests verify database compatibility, backend API behaviour, container health, and frontend–backend consistency after the schema change. A predefined architectural invariant requires the `age` column to remain absent, preventing a trivial rollback from satisfying the intended cascading-repair task. The run succeeds only after normal submission, with all applicable functional tests and invariants passing and no evaluator response.

Appendix B: FSM Implementation and Logic Gating Parameters

This appendix specifies the operational limits and clarification rules of the Finite State Machine (FSM) used in the Interaction Layer. Repairable technical faults must be resolved autonomously. One predefined Level 2 task deliberately contains one frozen non-technical business-requirement gap; the other 11 briefs are designed to include the information required to proceed.

B.1 Run Limits and Inference Configuration

To ensure reproducibility and prevent uncontrolled execution, the evaluation applies a common task-level wall-clock limit to all evaluated systems:

- **Task-Level Wall-Clock Limit (T_{task}):** Every run, regardless of the evaluated system, is subject to the same maximum elapsed time of 5,400 seconds from task-prompt submission to completion or suspension. If a run reaches this limit, no further agent actions are permitted and the run enters the `Suspension State`. The runner records the stopping reason, freezes the final system state, and transitions the run to the `Evaluation State`, where the same frozen hidden tests and architectural invariants used after normal submission provide diagnostic results. The limit-triggered stop remains unsuccessful even if every check passes.

- **Maximum Action Count (N_{max}):** The frozen API-wrapper maximum is 30 observable actions. One action is one accepted invocation of `execute_bash`, `read_file`, `patch_file`, or `request_human_clarification`; multiple tool calls in one model response count separately. The `<SUBMIT_FIX>` marker does not count, and provider retries that never reach the tool executor are logged but excluded. The pilot began at 15 actions; the amendment log records the single pre-freeze increase to 30 after the provisional threshold repeatedly stopped progressing GPT-5.4 runs. The frozen value is applied identically to both API agents. It is an API safety safeguard defined by this study, not a cross-system action-efficiency metric.

- **Command Timeout Threshold (T_{limit}):** Each command executed through the API-agent wrapper has a hard timeout of 180 seconds. This study-specific control prevents one command from indefinitely blocking a run; the exact 180-second threshold is a study design choice. The pilot did not justify changing the threshold. Cursor and Devin Desktop are governed primarily by the common task-level wall-clock limit because equivalent internal command controls are not exposed.

- **No-Progress Limit:** An API-agent run is suspended after three consecutive model responses that contain no accepted tool action, authorised clarification request, or valid submission marker. The counter resets whenever the response produces one of those events. This is a deterministic termination safeguard and is not reported as a cross-system quality metric.
- **Context Retention:** The wrapper retains at most 100,000 characters of interaction context under the frozen truncation policy. The selected Qwen pilot evidence reached at most 23,025 retained characters, so the boundary is retained as a conservative operational limit rather than claimed to be stress-tested.
- **API Sandbox:** API-agent actions are restricted to the assigned task repository. External network access and host-Docker control are disabled, while evaluator-only tests and infrastructure remain outside the agent-accessible sandbox.
- **Inference Configuration:** GPT-5.4 uses the OpenAI Responses API snapshot `gpt-5.4-2026-03-05`. Qwen2.5-Coder-7B-Instruct uses a pinned `Qwen/Qwen2.5-Coder-7B-Instruct` repository revision on a managed Hugging Face vLLM endpoint. The endpoint, repository commit, adapter version, temperature, context limit, system prompt, available tools, and retry policy are recorded and frozen before the main experiment. Cursor and Devin Desktop use the default agent model or mode and default settings observed at configuration freeze without manual override; the product version, displayed default where visible, settings, and evaluation date are documented.

Pilot runs are excluded from the 144-run main experiment. The schedule is blocked by task and repetition; within each task–repetition block, a seeded random permutation determines the four-system order. The seed and schedule are frozen before data collection. Evaluator mistakes and infrastructure failures outside the assigned task produce an invalid run whose evidence is retained and which is repeated under the same frozen configuration. Agent-caused service failures, tool errors, unsuccessful repairs, and execution-limit events remain valid outcomes. Protocol changes after freezing require an amendment entry.

B.2 Invalid Runs, Replacement Attempts, and Budget Interruptions

Attempt number `a01` is the default. A replacement attempt `a02`, `a03`, and so forth is permitted only after the preceding attempt is formally invalidated for a cause outside the evaluated agent's assigned task. The invalid attempt remains in the evidence store with its original identifier and is excluded from the success-rate denominator. The replacement retains the original phase, task, system, repetition, and seeded schedule position; only the attempt suffix changes. It begins from a fresh deterministic reset and uses the same frozen model or product mode, prompt, tools, limits, evaluator, and acceptance criteria.

Valid unsuccessful runs are never replaced. These include incorrect repairs, agent-caused tool or service failures, failure to submit, and stops at the task-time, API-action, command-timeout, or no-progress limit. By contrast, an evaluator malfunction, operator error, unavailable promised provider service, or account/billing control that prevents the configured run from executing may invalidate the attempt. A larger provider spending limit may be authorised prospectively to provide operational headroom, but it does not extend the frozen task clock or API action limit. If

resolving an invalidating cause changes any frozen experimental component, the amendment chain must record the cause, affected run identifiers, disposition, and linked replacement-freeze hash before execution resumes.

B.3 Clarification Rules and Technical Fault Categories

The evaluation does not use regular expressions to automatically convert technical failures into requests for human assistance. Instead, logs are retained as evidence of the agent's debugging process, and clarification requests are evaluated against the fixed task protocol.

- **Predefined Information-Gap Task:** For the single clarification-opportunity task, the omitted business-requirement field, accepted-request rule, and one permitted response are written before data collection. Any explicit clarification request enters the `Clarification State` and is evaluated against the frozen rule. A request that identifies the planned gap receives only the frozen response; a request on a complete brief, a question that does not identify the gap, or a request for debugging help receives no additional information. In both cases, the request, decision, applicable response identifier, and timestamp are logged before the run returns to the same `Execution State`. Four systems and three repetitions yield 12 opportunity runs.

- **CORS and API Configuration Faults:** Errors such as missing `Access-Control-Allow-Origin` headers, blocked CORS policies, incorrect API base paths, and service-port mismatches are repairable technical faults. Such faults are related to connection and configuration categories described in the microservice taxonomy [65], but they do not trigger the `Clarification State`. The agent must diagnose and repair them autonomously.

- **Authentication and Missing Credentials:** Messages such as `401 Unauthorized`, `Token expired`, or missing credentials do not automatically justify clarification. Main-experiment tasks do not intentionally omit required credentials. If the information is present in the repository, environment, or task brief, the agent must diagnose the fault autonomously. An essential credential unexpectedly absent from all promised sources is a task-specification or infrastructure failure handled by the invalid-run rule [64].

- **Database and Infrastructure Faults:** Errors such as `Connection refused`, database-host misconfiguration, schema migration failures, and ORM–database inconsistencies are repairable benchmark faults. Missing-resource and configuration faults are described in the integration taxonomy [65], and database-access defects appear in Defects4REST [64]. Within TSB, these repairable faults remain the agent's autonomous debugging responsibility.

B.4 Abstract State Transition Logic

The following pseudocode summarises the API-wrapper control flow, including submission precedence and response-level stopping checks. Each limit-enforcement helper enters the Suspension State, records the reason, stops further actions and freezes the final state when its limit is reached. The no-progress counter follows the response-level rule above, including responses with no accepted action. Native IDE runs follow the common task-time, clarification and final-assessment rules through their recorded interface protocol. The evaluator does not modify repository files or provide unsolicited debugging assistance.

```
State = EXECUTION_STATE

WHILE RunIsActive() DO:
    EnforceTaskTimeLimit()
    IF State = SUSPENSION_STATE THEN: BREAK

    Response = Agent.PlanNextMove(Context)
    EnforceTaskTimeLimit()
    IF State = SUSPENSION_STATE THEN: BREAK

    IF ResponseContainsValidSubmitMarker(Response) THEN:
        Transition To: SUBMIT
        StopFurtherAgentActions()
        FreezeFinalSystemState()
        BREAK  // Do not execute tools in this response.
    END IF

    Actions = ParseAcceptedActions(Response)
    FOR EACH Action IN Actions DO:
        IF Action Is ClarificationRequest THEN:
            Transition To: CLARIFICATION_STATE
            RecordClarificationRequest(Action, Timestamp)
            IF RequestIdentifiesFrozenLevel2Gap(TaskID, Action) THEN:
                Context.Append(RetrievePreWrittenResponse(TaskID))
                RecordClarificationDecision("Authorised", Timestamp)
                RecordEvaluatorResponse(ResponseID, Timestamp)
            ELSE:
                RecordClarificationDecision("Not authorised", Timestamp)
                // No additional information is provided.
            END IF
            Transition To: EXECUTION_STATE
        ELSE:
            Logs = Sandbox.Execute(Action)
            RecordAction(Action, Logs, Timestamp)
        END IF
```

```
        CountAcceptedAction(Action)
        EnforceActionCommandAndTaskLimits()
        IF State = SUSPENSION_STATE THEN: BREAK
    END FOR

    IF State = SUSPENSION_STATE THEN: BREAK
    UpdateNoProgressCounter(Response, Actions)
    EnforceNoProgressAndTaskLimits()
    Context = RetainAtMost100000Characters(Context)
END WHILE

EndedByNormalSubmission = (State = SUBMIT)
Transition To: EVALUATION_STATE
RecordFinalSystemState()
TestResults = RunFrozenAcceptanceTests()
InvariantResults = CheckArchitecturalInvariants()

IF EndedByNormalSubmission
AND AllApplicableTestsPass(TestResults)
AND AllApplicableInvariantsPass(InvariantResults) THEN:
    Transition To: SUCCESS
    RecordRunStatus("Successful")
ELSE:
    Transition To: FAILURE
    RecordRunStatus("Unsuccessful")
END IF
```

Appendix C: API-Agent Prompts and Action-Space Specifications

This appendix specifies the common prompt and action-space protocol used for the API-based agents in the TSB framework. GPT-5.4 is called through the OpenAI Responses API using `gpt-5.4-2026-03-05`. Qwen2.5-Coder-7B-Instruct is loaded from a pinned revision of `Qwen/Qwen2.5-Coder-7B-Instruct` through a managed Hugging Face Inference Endpoint using vLLM. Both are evaluated through the same wrapper, system prompt, task-brief format, logical tool definitions, execution limits, reset procedure, and sandbox policy. The live gate and 16-cell pilot are complete; endpoint and repository revisions, adapter versions, inference settings, and pilot-to-main execution limits have been recorded and refrozen for the main configuration.

Cursor and Devin Desktop are evaluated through their native IDE environments using the default agent model or mode and default settings available at the configuration-freeze date. No manual model override is applied. They receive the same task brief, repository state, acceptance criteria, and execution environment as the API-based agents; however, their internal system prompts, model parameters, and native tool implementations are not fully controlled or exposed. Their product version, displayed default model or mode where visible, available settings, and evaluation date are recorded before the main experiment.

The execution route is part of the evaluated end-to-end system and is frozen with it. Consequently, native IDE permission events are not converted into API tool calls, and API action counts are not used as proxies for unexposed native actions. Comparability instead rests on the controls observable across both routes: the same brief and initial state, a common task-level clock, the same clarification opportunity, a frozen final-state boundary, and the same hidden evaluator.

C.1 Standardized System Prompt for API-Based Agents

The API-based agent is initialized with a task-independent system prompt. Each task-specific brief is then supplied separately. The prompt is designed to provide consistent operational constraints without prescribing a particular debugging strategy. Provider-specific transport adapters validate model responses and map them to the same logical action schema; raw model responses and adapter decisions are retained for audit.

Standardized API-Agent System Prompt

```
You are an autonomous coding agent being evaluated on one task. The
task-specific brief is supplied separately as the user message and is the
only task-specific instruction you may use.

Work only within the supplied repository. Diagnose and repair the task while
preserving the stated invariants and existing architecture. Do not modify
tests or benchmark infrastructure. Evaluator internals, hidden tests,
researcher-only files, and external task solutions are outside your
information boundary.

You have exactly four logical tools: read_file reads one UTF-8 workspace
file; patch_file applies a standard unified git diff inside the workspace;
execute_bash runs one bounded Bash command in the isolated workspace sandbox
without network or host-Docker access; request_human_clarification submits
one observable clarification request under the frozen protocol. Use only
these tools and their declared schemas. Tool results are returned before
your next response.

The main experiment uses a maximum of 30 accepted logical tool actions.
Every accepted call to a known tool counts separately, including a call with
malformed arguments; provider retries and submission do not count. Each
execute_bash command has a maximum duration of 180 seconds. A separate task
wall-clock limit may also stop the run. Tool errors and non-zero command
exits are observable evidence; continue autonomously unless the runner stops
the run.

Request human clarification only when a material requirement is genuinely
unavailable from the brief or repository. A permission approval is not
clarification. Continue without asking when the task is fully specified.

When you believe the repository is in its final state, respond with the exact
marker <SUBMIT_FIX>. Submission is detected in observable assistant text,
counts as zero actions, and ends the run. If a response contains the marker,
tool calls in that same response are recorded but not executed. Do not emit
the marker until you intend to submit the final workspace.
```

C.2 Common Task Brief

Every evaluated system receives the same task-specific brief. The brief identifies the task objective, observable failure symptom, permitted repository, and promised environment. One predefined Level 2 clarification-opportunity task deliberately omits one non-technical business requirement, for which a single pre-written answer is available. The other 11 briefs contain the information required to proceed. Repeating the opportunity task three times for each of four systems produces 12 clarification opportunities. No brief reveals evaluator-only hidden tests, the oracle repair, or a required edit sequence.

The four pilot briefs, including the planned Level 2 gap, eligibility rule, and response card, were validated during the pilot. The other eight task packages subsequently completed implementation validation, and the full set of 12 briefs was frozen before main collection for reuse without modification across all three independent runs for each system. An unexpected essential omission outside the frozen clarification design is handled by the invalid-run and amendment rules.

C.3 Constrained Action Space for API-Based Agents

The API-agent wrapper provides a minimal action space for repository inspection, code modification, command execution, and authorised clarification. Every tool call, result, and timestamp is recorded. The 30-action limit, 180-second command timeout, three-response no-progress rule, 100,000-character context-retention boundary, and repository-only sandbox are enforced as described in Appendix B. External network access and host-Docker control are disabled.

- `execute_bash(command: str)` $\rightarrow$ `stdout/stderr`
  Executes a terminal command within the assigned project environment. Commands are subject to the T_{limit} timeout threshold defined in Appendix B. The wrapper records the command, output, exit status, duration, and timeout status.

- `read_file(filepath: str, start_line: int, end_line: int)` $\rightarrow$ `str`
  Retrieves the requested part of a file within the assigned repository. Outputs may be paginated to avoid exceeding the model context limit.

- `patch_file(patch: str)` $\rightarrow$ `tool result`
  Applies a standard unified git diff within the assigned repository. The wrapper validates the patch and affected paths, checks applicability with `git apply --check`, and then applies it with `git apply`. The tool returns a success message or an error result and records affected paths and patch size when applied.

- `request_human_clarification(query: str)` $\rightarrow$ `str`
  Submits and records a clarification request. On the predefined opportunity task, a request that identifies the frozen information gap returns only the corresponding pre-written answer. Other requests return no additional guidance. An unexpected omission outside the frozen design is handled under the task-specification invalid-run rule; the evaluator does not improvise debugging advice.

C.4 Native IDE Protocol for Cursor and Devin Desktop

Cursor and Devin Desktop receive the same fixed task brief and begin from the same reset repository and Docker environment as the API-based agents. No manual model or mode override is applied within the configuration bound to a run. Detected changes and replacement freezes follow the recorded amendment process. The evaluator does not modify files, provide unsolicited hints, or reveal hidden acceptance tests during a run.

Because the commercial IDEs may not expose their internal tool calls, token usage, or system prompts, these values are not treated as directly comparable primary metrics. The evaluation instead records the task start time, end time, final repository difference, frozen-test results,

container and database checks, clarification events, and available agent transcript. Where a transcript or model-selection record is available, it is retained as supplementary evidence.

For reproducible timing in the native IDEs, a local parameterised hotkey controller starts a monotonic recorder immediately before it submits the focused frozen brief; the primary elapsed-time value therefore does not depend on an IDE display duration. The controller stops at a product-specific visible completion state: How did the agent do? for Cursor, and a fully rendered final response with feedback and copy controls for Devin. The same controller records a frozen wall-clock limit and freezes the final workspace state before evaluator-only checks begin.

All visible IDE permission prompts are recorded verbatim with their observed decision and scope. They document routine product authorisation and are not treated as clarification or researcher assistance. The native timing and permission journals are hash-chained. Permission-event counts remain native-interface diagnostics and are never compared numerically with accepted API tool-action counts. Retained evidence includes the final workspace hash and diff, exported conversation where available, PII-free completion screenshots, evaluator probe logs, and the structured result bundle. This implementation preserves an auditable separation between the agent's autonomous execution, the locally observed timing boundary, and post-run evaluation.

C.5 Pilot and Main-Experiment Completion Status

The live gates and all 16 system–tier pilot cells were completed before main collection. The audit trail retains 27 attempts, including superseded and replacement attempts, while exactly one attempt is selected for each pilot cell. All 16 selected runs pass the common evidence validators, and all eight selected native-IDE runs pass the additional native evidence validator. Pilot-to-main execution decisions have been recorded and refrozen, including the increase from 15 to 30 API actions and retention of the other approved limits.

The complete main-experiment technical freeze was sealed before main collection. All 12 task packages and deterministic start commits, the seeded 144-run schedule and its hash, the 105-file main evaluation bundle, and the analysis plan are frozen and validated. Main collection was completed on 26 September 2026, with one selected valid terminal attempt for each of the 144 scheduled positions, including unsuccessful outcomes. Each system and each tier contributes 36 observations. Invalidated attempts and their adjudications remain in the provenance record but are excluded from performance denominators. Pilot evidence remains separate and descriptive. The selected analysis dataset and statistical results are complete. Section 5 reports reviewed outcomes and original-score sensitivity comparisons under the uniform evaluator-review rules. The completed main experiment forms the empirical basis of this thesis.

Appendix D: Evaluation Metrics and Reporting Schema

The TSB reports complementary outcomes separately. Binary run-level success is the primary outcome; no weighted aggregate or penalty score is used.

The evidence dataset retains one row per valid or invalidated attempt. Analysis selects one valid terminal attempt for each scheduled task–system–repetition slot; invalidated attempts remain in the audit record but are excluded from performance denominators. The frozen main data dictionary includes the run phase, run identifier, selected or replacement-attempt status where applicable, system, product or model version, provider endpoint, task, tier, repetition, seeded order position, configuration and prompt hashes, success indicator, C_{func}, C_{arch}, elapsed time, stopping reason, task-time and command-timeout indicators, action-limit and no-progress indicators, retained-context characters, clarification counts, native permission-event counts where observable, robustness eligibility, token usage and cost where available, failure category, and invalid-run reason. Raw logs remain linked by run identifier rather than embedded in the analysis table. The main data dictionary was sealed with the main evaluation bundle before data collection.

D.1 Evidence and Amendment Provenance

The evidence store uses the canonical identifier `<phase>-<task>-<system>-r<repetition>-a<attempt>`. It separates researcher-only task material, the agent-visible workspace, and evaluator-owned evidence. The run record links the frozen prompt and configuration hashes, immutable start commit, observable execution record, clarification and permission journals where applicable, final workspace state and diff, stopping transition, and hidden functional and architectural results. Missing commercial-IDE fields are marked unavailable rather than inferred.

Evidence writers have distinct authority: preparation creates the workspace and initial metadata; the evaluated system changes only the workspace; the API wrapper or native recorder appends observable execution events; the evaluator freezes and tests the final state; and the validator reads but does not repair the record. Native timing, permission, and clarification journals include the preceding entry hash, and their final chain heads are bound into the run record.

The configuration manifest hashes the exact bytes of every frozen component. Amendment entries are append-only, link to the preceding amendment hash, identify affected runs and their dispositions, and bind the replacement freeze that resolves the change. This produces the traceability chain

reported row → run ID → run evidence → configuration hash → freeze/amendment chain.

Invalid and superseded attempts remain in this chain even when excluded from performance analysis. The run-level configuration hash identifies a route-specific record: a canonical system record for a native IDE, or the API transport-configuration file for an API agent. It is not interchangeable with the hash of the complete experiment-freeze manifest. A matching manifest reference establishes provenance; restoring an earlier execution configuration additionally requires the corresponding component files. The latest configuration file must not be substituted for an earlier run-bound version.

D.2 Run-Level Success and Diagnostic Correctness

For system s, task i, and fresh repetition r:

$$Y_{s,i,r} = \begin{cases} 1, & \text{if a valid run submits normally and passes all applicable tests and invariants,} \\ 0, & \text{for any other valid terminal outcome.} \end{cases}$$

Each system contributed three selected valid observations for each of the 12 tasks, so the final rate is:

$$\text{SuccessRate}_s = \frac{1}{36} \sum_{i=1}^{12} \sum_{r=1}^{3} Y_{s,i,r}.$$

The `<SUBMIT_FIX>` marker declares normal API-agent completion; it does not establish correctness by itself. After submission or suspension, the evaluator runs tests stored outside the agent-accessible repository and records the final state. A valid execution-limit stop is unsuccessful even when the frozen checks pass; formally invalidated attempts are replaced and do not enter the denominator. The diagnostic quantities C_{func} and C_{arch} are the proportions of applicable functional tests and architectural invariants passed. They are reported separately and are not combined into a weighted score.

D.3 Execution Performance

For every run, the evaluator records end-to-end elapsed time:

$$\Delta T = T_{end} - T_{start},$$

together with the stopping reason, command failures, and timeout events. Observable API-agent action count, token usage, and monetary cost are supplementary where provider logs support them. Native IDE permission events are recorded separately where observable. Permission-event counts and accepted API tool-action counts are interface-specific diagnostics and are not combined or directly compared. Unavailable commercial-IDE data are reported as unavailable rather than estimated.

Completion time among successful runs and stopping time across all valid runs are reported separately. Runs stopped at T_{task} are flagged at the limit and are not treated as ordinary successful completions. No formal survival model is required for the primary analysis.

D.4 Clarification-Seeking Behaviour

For each run, the evaluator records:

- N_{req}: clarification requests submitted;
- N_{acc}: requests authorised by the predefined rule;
- N_{resp}: pre-written evaluator responses issued.

The run table also records whether the brief contains the frozen clarification opportunity. The single Level 2 opportunity task produces 12 runs overall and three per system. Clarification opportunity capture is the proportion of these runs with at least one authorised request. Request precision is N_{acc}/N_{req} when at least one request exists. The other 11 tasks produce 132 complete-brief runs overall and 33 per system; the unnecessary-request rate is the proportion of these runs containing any request. Submitting the initial prompt and approving routine IDE permissions are excluded. Technical faults such as CORS errors, database connection failures, migrations, and API-contract mismatches do not justify clarification.

D.5 Success Without Evaluator Clarification Responses

The prespecified robustness measure records success without an evaluator clarification response on Level 3 and Level 4 tasks:

$$R_s = \frac{S_{auto,s}}{18},$$

where $S_{auto,s}$ counts successful runs among the six applicable tasks, each repeated three times, that received no evaluator clarification response. Success requires normal submission and passage of all applicable assessed tests and invariants. A valid limit-triggered stop is unsuccessful. No eligible L3/L4 run received an evaluator response, so this measure equals upper-tier success in the present dataset and supplies no independent evidence of recovery or resilience.

D.6 Qualitative Failure-Trajectory Analysis

Observable transcripts, tool calls, final diffs, command outputs, and test results are reviewed for recurring trajectories such as premature completion, repeated ineffective commands, incorrect API or database assumptions, failure to trace cascading dependencies, unnecessary clarification requests, and failure to validate the final state. The analysis does not infer private reasoning or assign subjective reasoning points.

D.7 Statistical Analysis and Reporting

The scheduled task–system–repetition slot is the run-level analysis unit; the task is the resampling unit for comparative uncertainty. Success rates are described overall and by tier using Wilson 95% intervals, task-level results, and a task–system heat map. Wilson intervals describe run outcomes but do not account for task clustering. Pairwise system effects are risk differences with two-sided 95% percentile intervals from 10,000 task-level bootstrap resamples; each resampled task retains all four systems and all three repetitions. Two-sided exact paired task-label-swap tests over 2^{12} task-level assignments provide the six pairwise p-values, with Holm adjustment applied to this six-comparison family. Tier results remain primarily descriptive because each tier contains only three tasks. All comparative inference is exploratory.

Elapsed time is summarised using medians and interquartile ranges. Successful completion times, all-valid-run stopping times, and limit events are distinguished. Clarification opportunities are displayed in a table with explicit counts and denominators, while C_{func} and C_{arch} remain separate diagnostic views. The reproducible Python/Jupyter workflow reads the selected run table and produces the tables and figures for both scoring versions. The data dictionary, random seed,

exclusion rules and planned analyses were frozen before collection. Implementation scripts, package versions and the refined qualitative coding rules were recorded during post-collection analysis rather than retrospectively described as frozen main-experiment components. Pilot attempts remain in a separate provenance dataset and are excluded from the 144-run main analysis.

Reproduction Files and Evidence Boundaries. The public research-materials repository is `tiered-system-benchmark`, branch `supplement2`. The figure release described here is pinned to commit `05a65d4`. From the repository root, the following commands regenerate the nine quantitative figures from the released data:

```
python -m pip install -r analysis/requirements-statistics.txt
python analysis/plot_core_success.py
python analysis/plot_secondary_outcomes.py
```

The PDF, SVG, and PNG exports are provided under `results/figures/`. The results documentation maps thesis Figures 8–16 to the exported files and provides the figure captions and reproduction details.

Paths below are relative to the assembled research-materials release. The selected input is `data/main/analysis_dataset.csv`, with one row per scheduled slot and `run_id` as the evidence join key. `analysis/build_dataset.py` constructs this table from the tier-specific selected-run and attempt indexes, the main schedule, and `data/evaluation-review/original_scores.json` and `data/evaluation-review/reviewed_scores.json`. Reviewed columns use `public-contract-audit/2026-09-26`; original columns preserve the baseline scoring. The offline script `data/evaluation-review/reproduce_scores.py` checks the reviewed mapping against exported diagnostic evidence. It does not rerun the agents or the HTTP/database probes.

The scripts `analysis/core_success.py` and `analysis/secondary_outcomes.py` generate the quantitative tables under `results/tables/`. Their input hashes, software versions and analysis details are recorded in `results/core_success_manifest.json` and `results/secondary_manifest.json`. Failure codes are inputs in `results/tables/failure_coding.csv`; `analysis/failure_summary.py` validates coverage and reproduces category and stage counts, rather than independently reproducing the qualitative judgements. `results/failure_evidence_index.json` separates release-relative evidence paths from retained runtime-archive references.

The curated release is distinct from the complete operational archive. Full native transcripts are not uniformly available, and historical configuration files, amendment records and runtime evidence are not all included in the release. Reproducing the exported scores and statistical tables therefore does not establish complete recovery of every historical execution environment. Archived run identifiers and configuration hashes support tracing the retained evidence; missing historical component snapshots remain a limitation of execution reconstruction.

D.8 Uniform Evaluator Review and Score Amendments

After collection, all 12 tasks were reviewed against the public briefs, agent-visible repository contracts, scoring manifests, and evaluator implementations. The saved prompts for all 144 selected runs contain their corresponding public briefs. The review covered 34 functional scoring entries and 72 architectural scoring entries. In the original scoring, every run labelled successful also submitted normally; this condition is retained under every reviewed score version.

The review distinguishes implementation-specific checks from required behaviour. A variable name, source-code pattern, or internal implementation layer is not independently required unless it follows from the public task contract. A correction is applied to all 12 submissions for the affected task, including submissions that remain unsuccessful. Original scores, submissions, elapsed times, stopping reasons, and validity records are retained. The amendments below were made after outcomes were observed and are therefore reported as post-collection evaluation decisions, not as preregistered criteria.

L2-03: Storage behaviour and working directory. The original evaluator required variables named `PROJECT_ROOT` and `STORAGE_PATH`. It also restored the working directory after module import but before HTTP requests, which could miss failures in code that resolved relative paths during a request. All 12 submission copies were checked with the working directory held at the application directory during requests, followed by application reload from a different directory. The checks cover seed loading, import/export, the authoritative physical file, and preservation of saved notes after an invalid import. GPT-5.4 repetition 2 passes these behaviours despite omitting the expected variable name. Repetition 1 still fails actual storage behaviour: its functional score changes from 4/4 to 1/4, while its run remains unsuccessful. This diagnostic replay establishes application-level persistence; it does not establish Docker-volume persistence through container recreation.

L3-01: Public CORS boundary. The public brief requires access from the configured frontend without weakening policy for unrelated origins. It does not explicitly require removal of the development origin already allowed by the supplied repository. The reviewed interpretation permits retention of `http://127.0.0.1:5173`, requires `http://127.0.0.1:8080`, and rejects wildcards and newly permitted unrelated origins. All 12 submission copies were checked against this same rule using the actual middleware configuration and GET/preflight responses. The five affected normally submitted runs pass the reviewed boundary; the three Qwen runs still fail to enable the required frontend origin and remain unsuccessful.

L4-02: Default status and accepted values. The public brief requires omitted status to default to `active`, with only `active` and `inactive` accepted. It does not specify which internal layer must implement the default. The reviewed rule therefore checks HTTP creation, persisted status, and subsequent reads without requiring the Pydantic request model itself to supply the default. Explicit invalid values, including `null`, remain distinct from omission and must be rejected without a database write. The review uses the archived HTTP/database cases for all 12 submissions and removes the separate request-model gate. Cursor repetition 2 and Devin repetition 1 gain the omitted-default check but remain unsuccessful: the former accepts explicit null, while the latter ignores explicit inactive and accepts invalid inputs. Earlier source-pattern corrections for Cursor repetitions 1 and 3 and Devin repetitions 2 and 3 remain applicable.

Cursor L3-01 repetition 2 uses the selected replacement attempt a02; the other runs listed in Table 7 use a01. Task-level success counts change from 5/12 to 6/12 for L2-03, from 4/12 to 9/12 for L3-01, and from 3/12 to 7/12 for L4-02. Removing the default-layer restriction does not further change the existing 7/12 semantic result for L4-02. The complete versioned review table contains all 144 selected run identifiers, including unchanged outcomes and diagnostic-only

Table 7: Affected submitted runs: original and reviewed architectural checks and success. Repetitions grouped in a row have the same transition.

Task	System and repetition	C_{arch} counts	Y
L2-03	GPT-5.4, Rep 1	$5/6 \to 4/6$	$0 \to 0$
L2-03	GPT-5.4, Rep 2	$5/6 \to 6/6$	$0 \to 1$
L3-01	Cursor, Reps 1–3	$5/7 \to 7/7$	$0 \to 1$
L3-01	GPT-5.4, Reps 1–2	$5/7 \to 7/7$	$0 \to 1$
L4-02	Cursor, Reps 1 and 3	$4/6 \to 6/6$	$0 \to 1$
L4-02	Devin Desktop, Reps 2–3	$4/6 \to 6/6$	$0 \to 1$
L4-02	Cursor, Rep 2	$4/6 \to 5/6$	$0 \to 0$
L4-02	Devin Desktop, Rep 1	$4/6 \to 5/6$	$0 \to 0$

changes; it preserves the original scoring alongside the reviewed scoring and evidence references.

Coverage limits and sensitivity reporting. The review also identified syntax-dependent checks and incomplete behavioural coverage in other tasks. These are recorded as evaluator limitations rather than automatically treated as run-level misclassifications. Examples include incomplete checks of all preserved seed values, the absence of a duplicate-email probe in the customer task, and source-based approximations to layout and service-policy requirements. The review is not a claim that every final system has undergone exhaustive fresh testing. Section 5.7 compares original and reviewed scoring on the same 144 slots, including system success rates, diagnostic correctness and pairwise effects. The broad separation from Qwen persists, while the descriptive ordering among the other systems changes and their adjusted pairwise tests remain inconclusive. No agent execution is replaced by this analysis-layer review.

D.9 Failure Coding and Illustrative Evidence

The coding population comprises the 42 unsuccessful observations under reviewed scoring. Multiple planned category families may apply to one run. Codes were assigned from recorded outputs, accepted tool events, final diffs and evaluation evidence; no private reasoning was inferred. Operational coding decisions were made after observing outcomes and are descriptive. No independent second-coder agreement estimate is available.

Premature submission requires observable completion without executed repair, rather than simply any failed final check. Repeated ineffective commands require at least two executed, ineffective inspection attempts; repeated prose describing a command is not counted as execution. Incorrect API/database assumptions are coded through a visible contract-violating change, without attributing an internal belief. Missed cascading dependency denotes a local repair accompanied by a remaining or introduced linked-service failure. Unnecessary clarification uses the recorded request events on complete-brief tasks. Failed final-state validation requires explicit unsuccessful validation attempts followed by submission; it is not automatically assigned to every unsuccessful run. The other-supported category records 35 no-progress/protocol failures and two residual storage/configuration failures. Unobservable native behaviour remains unclassified at the trajectory level.

The supplementary stage description distinguishes no accepted tool action (18 Qwen runs), inspection attempts without a task-code diff (18 Qwen runs), and edited submissions with failing checks (six runs across the other systems). It describes observed execution progress, not an ordinal

capability score. Fifteen Qwen runs contain missing-path errors; all 36 have empty final diffs. Their unexecuted prose proposals are not treated as actual source changes.

API interaction examples. In Qwen L1-01 repetition 1, an accepted `read_file` call for `index.html` returned "Path does not exist: index.html". Subsequent responses proposed a file-search command as text without accepted execution, and the no-progress limit ended the run. In Qwen L3-02 repetition 3, a single response contained proposed JSON tool calls and `<SUBMIT_FIX>`; the adapter recorded submission with zero accepted actions and the final diff was empty. These two cases illustrate different observable routes to an unchanged submission. They are selected excerpts, not complete interaction transcripts.

Submitted changes and validation. Devin Desktop L2-03 repetition 3 changed import behaviour to merge existing and imported notes; the evaluator recorded four notes where the required replacement import contained two. Cursor L4-02 repetition 2 implemented a valid omitted-status default but also accepted explicit null. Devin Desktop L4-02 repetition 1 forced active status while ignoring explicit inactive and invalid values. These cases remain failures under the common public-contract review, independently of where a default is implemented. Their native transcripts are unavailable, so the evidence supports the final behaviour and diff, not claims about unseen decisions or testing.

GPT-5.4 L4-03 repetition 1 uses selected replacement attempt a02. Its diff corrected the database hostname and introduced frontend/proxy changes. Final browser evidence recorded `/api/dashboard` returning HTTP 404. The observable transcript also records "pytest: command not found" and "npm: command not found" before the submission marker. This supports a linked-service failure and unsuccessful pre-submission validation without requiring an inference about private reasoning.

Run-level codes, coverage labels and evidence references are retained in the accompanying `failure_coding.csv`; category and stage summaries are derived from that table. Original runtime evidence and original/reviewed scoring remain separate from these descriptive codes.

Appendix E: Supplementary Qwen Interface Experiment

E.1 Configuration and Execution Checks

The supplementary experiment was conducted after the main experiment on 26 October 2026. Its protocol and schedule were saved before new model calls, using schedule seed `qwen-interface-control-20261026-v1`. Arm A used `qwen-fenced-tool-json-v3`; Arm B changed only acceptance of physical line breaks within a JSON object. The single-call requirement, tool names, argument validation, residual-text handling and other rejection rules remained fixed.

Both arms used Qwen2.5-Coder-7B-Instruct revision `c03e6d358207e414f1eca0bb1891e29f1db0e242`, vLLM 0.26.0 and the same chat template. Temperature was 0 and the maximum output length was 4,096 tokens. Each run retained a 30-action budget, a 180-second command timeout, a 5,400-second task limit and termination after three consecutive responses without an accepted action. Task repositories, prompts, tools, sandbox restrictions and scoring rules were held fixed. The sandbox was not extended with additional commands during the experiment.

Guided preflight checks exercised actual reads, patch application and command execution in both arms. Their explicit step-by-step prompts were excluded from the benchmark runs and all reported outcome counts. The formal runs used the original benchmark prompts. Audit of the 36 task–repetition pairs confirmed matching first-request content, tool definitions and policy after excluding the bookkeeping attempt identifier. All 352 formal responses reported service fingerprint `vllm-0.26.0-607fd87d`; none contained a native structured tool call.

E.2 Task-Level Outcomes

Every task had three valid runs per arm and zero successful repairs in both arms. Table 8 reports tool progress. The three successful operations in Arm B were two directory listings and a file read in L2-02 repetition 3. All final diffs were empty. Passing checks on an unchanged starting state do not represent a partial repair.

Table 8: Tool progress by task in the supplementary experiment. Each row aggregates three repetitions per arm.

	Accepted calls		Successful operations	
Task	A	B	A	B
L1-01	3	3	0	0
L1-02	0	3	0	0
L1-03	3	3	0	0
L2-01	0	0	0	0
L2-02	3	7	0	3
L2-03	0	2	0	0
L3-01	0	0	0	0
L3-02	4	2	0	0
L3-03	0	3	0	0
L4-01	3	3	0	0
L4-02	5	5	0	0
L4-03	8	21	0	0
Total	29	52	0	3

E.3 Attempt Accounting

The 72 scheduled positions were executed in four groups of 18. At position 51 (Arm B, L4-03, repetition 1), an operator pause interrupted attempt a01 during its seventh model request, after six recorded responses. The incomplete attempt was retained and excluded from outcome counts. A fresh attempt, a02, reran that position under the same configuration, followed by the remaining scheduled positions in order. The preceding 14 completed runs in that group were retained without rerunning. The replacement manifest maps position 51 to a02, leaving exactly one valid attempt per scheduled position. Preflight checks and the interrupted attempt are separate from the 72 formal observations; the original 144 main-experiment observations remain intact.